

**Syrian Private University**  
**Faculty of Engineering**  
**Department of Software**  
**and Information Systems**



# **Integrated Event & Activity Management Platform**

A senior 2 project report- submitted to complete the requirements for obtaining a bachelor's degree in  
Informatics Engineering - Software Engineering and Information Systems

**Prepared by:**

Helpest Mohamad Mohamad      Marwa Ghassan Aljaouni      Hiba Ahmad Shawa

**Supervised by:**

Eng. Anas Abdulziz

2026

# منصة متكاملة لتخطيط الفعاليات والأنشطة

مشروع تخرج 2 - قدم استكمالاً لمتطلبات الحصول على درجة البكالوريوس في هندسة المعلوماتية  
هندسة البرمجيات ونظم المعلومات

إعداد:

هلبست محمد محمد

مروة غسان الجاعوني

هبة احمد شاوه

إشراف:

المهندس أنس عبد العزيز

## وَأَخِرُ دَعْوَاهُمْ أَنْ الْحَمْدُ لِلَّهِ رَبِّ الْعَالَمِينَ

رحلتَ عن الدنيا مبكراً، لكنك ما زلت حاضراً في قلبي وفي كل خطوة من حياتي.  
إلى من شاركني فرحة قبولي في الكلية ، وأكثر من تمنيت وجوده معي في يوم تخرجي.  
وما زلت أشتاق لصوتك وكلماتك التي كانت تطمئنني دائماً:  
“ما يينخاف عليها... هبوشة قدّها”.

ورغم غيابك، ما زالت دعواتك ومحبتك ترافقني في كل خطوة.  
هبوشتك الحلوة تخرجت يا بابا...  
رحمك الله يا حبيبي، وجعلني دائماً كما كنت تمنى.

والدي الغالي: أحمد شاوه

مررتي الأولى، وسندي في كل مراحل حياتي.  
إلى من حملت عني ثقل الحياة، وكانت لي الأب والأم معاً، والداعمة الأولى لكل أحلامي وخطواتي.  
إلى من كانت تُهَوِّن عليّ كل صعب، وترافقني في جميع تفاصيل حياتي، وتغمري بخنانها، وتحقق لي كل ما أتمنى.  
إلى من تعلّمتُ منها معنى القوة والصبر والحب، والرضا في أصعب الأوقات  
إلى من آمنت بي دائماً، ومنحتني القوة والثقة لأكمل طريقي.  
كل ما أنا عليه اليوم هو بفضل الله، ثم بفضل دعواتك وصبرك ومحبتك التي لم تنقطع يوماً.  
إلى من تُبَدِّل حياتي وقلبي ويبتنا بجها دائماً...

أمي الحنونة رشا حاج محمد

إلى من واجهنا الحياة معاً بكل مراحلها وصعوباتها، وشاركنا كل تفاصيل الأيام بجلوها وتعبها.

إلى من كانوا دائماً الجانب الأجل والأخف في حياتي، والضحكة التي تهون عليّ كل شيء.

شكراً لكل ضحكة، ولكل جلسة وحديثٍ امتدّ حتى آخر الليل، ولكل مرة خففتم عني ثقل الأيام والتعب.

كان وجودكما معي دائماً مصدراً للراحة والفرح، وبه اكتملت خفة هذه الرحلة وجمالها

إلى شريكنا دربي: تالا ورازة

إلى من كان بيتهم دائماً المكان الأقرب لراحتي، وفيه قضيت الكثير من ساعات الدراسة والتعب.

إلى من كانا يطمئنان عليّ في كل امتحان، ويدعوان لي في كل خطوة، ويشاركانني كل نجاح وكأنه نجاحهما.

شكراً لكل اهتمامٍ ودعوة صادقة، ولكل مرة احتفلتما بي وبناجحي بحبةٍ وفرحٍ لا ينسى.

شكراً للدلال والحنان الذي كنت أجده دائماً معكما، وللطمأنينة التي كانت ترافقني بوجودكما ودعائكما.

جدي الأستاذ عوني حاج محمد و جدتي الغالية فريال أبزاخ

إلى من شجّعني على دراسة الهندسة، ورافقني في يومي الأول إلى الجامعة، وكان إلى جانبي في كل خطوةٍ من هذه

الرحلة. إلى من كانت استشارته ورأيه يعينان لي الكثير في كل قرارٍ مررت به.

شكراً لأنك كنت دائماً السند القريب والحنون، ولكل دعمٍ وكلبةٍ خففت بها عني طوال هذه السنوات.

شكراً لوجودك الدائم بقربي.

خالي الغالي: المهندس أنس حاج محمد



إلى الحنان الدائم، وإلى من كانت الحياة معهم دائماً أخف وأجمل.

إلى من شاركوني تفاصيل أيامي كلها بحبة واهتمام، وكان فرحهنّ بنجاحي يشبه فرحتي وأكثر.

شكراً لكل دعوة صادقة، ولكل اهتمام وسؤال واحتواء، ولكل مرة شعرتُ فيها أنني محاطة بقلوب تحبني بصدق.

أحبكنّ كثيراً، وأحب كل اللحظات التي جمعتني بكنّ، وسيبقى لكنّ دائماً مكاناً كبير في قلبي

خالاتي الغاليات: وفاء وشذى وراما

إلى من عشتُ معهم طفولتي، ووكبرنا معاً وسط الضحكات واللعب وذكريات لا تُنسى.

إلى من كانت جمعات العائلة معهم دائماً الأجل، وكانت قصصنا وضحكاتنا تُهَوِّنُ تعب الأيام مهما كان.

شكراً لأنكم كنتم دائماً بقربي، ولكل ضحكة ولحظة جميلة ودعم جعل هذه السنوات أخف وأجمل.

إلى إخوتي وأخواتي:

عمر يحيى ، حمزة يحيى، معاذ يحيى ، أحمد نغوي ، عبد الرحمن نغوي

سارة صوقار ، سدره صوقار ، لين صوقار ، ستناي حاج محمد

إلى من كانت جمعات العائلة بوجودهم أجمل دائماً، وإلى من أضافوا لكل مناسبة فرحاً ودفقاً لا يُنسى.

شكراً لكل تشجيع ودعم وكلبة طيبة رافقتني خلال هذه الرحلة، ولكل محبة واهتمام شعرتُ بهما دائماً منكم.

وجودكم في حياتي وفي أجمل لحظاتي كان يعني لي الكثير، وسيبقى له مكاناً جميل في قلبي دائماً

هناء دوغوظ ، محمد نور يحيى ، إسلام نغوي ، زاهر صوقار

منذ يومنا الأول في الجامعة وحتى آخر لحظة في مشروع تخرجنا، كنّا معاً في كل شيء...

إلى من عشتُ معهما جميع تفاصيل هذه الرحلة؛ كبرنا معاً، وتغيّرنا معاً، وعشنا تفاصيل هذه السنوات بكل تعبها وضحكتها ولحظاتها التي لا تُنسى. كنتم دائماً الرفقة التي تهوّن عليّ كل شيء، والذين أشعر معهم أن التعب يخفّ مهما كان كبيراً. تشاركنا المحاضرات، والتسليمات، والتوتر، والضحك من القلب وسط كل هذا الضغط، حتى أصبحت هذه السنوات أجمل لأنكما كنتم فيها.

شكراً لأنكما كنتما السند والرفقة الأجل في هذه الرحلة، ولأن وجودكما جعل كل شيء أخفّ وأجمل.

### مروة الجاعوني وهلبست محمد

إلى من شاركوني تفاصيل خمسة أعوام لا تُنسى، بكل ما فيها من تعبٍ وضغطٍ، وضحكاتٍ كانت تهوّن ثقل الأيام. كنتم الرفقة التي جعلت لهذه المرحلة معنىً أجمل؛ تشاركنا صباحات الجامعة، وليالي الامتحانات، وتوتر المناقشات، والضحك الذي كان يسرقنا من كل هذا التعب، وكنتم دائماً أجمل ما في هذه الرحلة.

شكراً لكل جلسةٍ جمعتنا، ولكل حديثٍ عابرٍ أصبح اليوم ذكرى، ولكل ضحكةٍ أنستني تعب يومٍ كامل.

ستبقى أيامنا وتفاصيلنا الصغيرة من أجمل الذكريات التي سأحملها معي دائماً، فقد كان وجودكم يهوّن تعب الأيام ويجعل لهذه الرحلة جمالاً لا يُنسى.

ومهما أخذتنا الحياة إلى طرقٍ مختلفة، سيبقى أثركم في قلبي، وستبقى هذه السنوات من أجمل ما عشته معكم

إلى المهندسين: تامر الصياد ، أحمد الغوثاني ، عبد الله الزهنان ، عماد الخضراء ، أحمد قطوش

إلى المهندسات: هلبست محمد ، مروة الجاعوني ، دانه مزهر

إلى صديقات المدرسة، شريكات البدايات والأيام الجميلة...

كبرنا معاً وتقاسمنا الكثير من الذكريات والضحكات التي ستبقى دائماً قريبة من قلبي.

شكراً لكل دعم وكلمة صادقة، ولكل لحظة جميلة عشتها معكنّ

سراب خدام ، بيان رجب ، ندى حسين ، ريم سويد ، يارا الكيال، جودي بغدادي

إلى من رافقوني في طريق الجامعة، وجعلن أيام "دايموند" أجمل بما فيها من ضحكات ومواقف لا تُنسى.

معكنّ أصبح الطريق أخفّ، والأيام ألطف، وحتى التفاصيل البسيطة تحمل ذكرى جميلة.

شكراً لكل لحظة جمعتنا، ولكل حديث وضحكة جعلت هذه المرحلة مليئة بالفرح والذكريات الجميلة.

الصيدلانية ميس الدهش ، الصيدلانية: روان الكردي

إلى من جمعني بهم ممرات الجامعة والمحاضرات، وإلى من شاركوني الملخصات وتفاصيل الأيام البسيطة والجميلة...

شكراً لكل مساعدة، ولكل حديث وضحكة جعلت أيام الجامعة أخفّ وأقرب إلى القلب.

كانت أياماً مليئة بالتفاصيل الصغيرة التي صنعت ذكريات جميلة سأحتفظ بها دائماً.

نور نعان ، ماريا ابراهيم

شكراً لكل ما قدّمته لنا من علمٍ ودعمٍ وتوجيهٍ خلال سنوات الدراسة، ولكل جهدٍ رافقنا في المحاضرات والمقررات  
وخلال رحلة مشروعنا. كان لدعمك وتوجيهاتك أثرٌ جميل في رحلتنا، وتعلّنا منك الكثير خلال هذه السنوات

د.أنس عبد العزيز

إلى الذين كانوا دائماً جزءاً جميلاً من أيامي...

شكراً لكل ضحكةٍ وحديثٍ وجلسةٍ جعلت الأيام أخفّ وأجمل، ولكل لحظةٍ شعرت فيها بالراحة بقربكم.

أحبكم كثيراً، ووجودكم دائماً يجعل أيامي أجمل

الى العائلة الجميلة: توجان أبزاخ ، ماسة جركس ، جياغ جركس ، و ريتسا جركس

## **Abstract:**

In the absence of a unified digital platform serving the events sector in the Syrian market, our platform has emerged to fill this gap with a comprehensive solution that brings together everyone involved in creating, attending, and serving events under one digital roof. The platform allows attendees to browse events, book tickets, and pay fees in a few simple steps, receiving an instant digital ticket with a QR code. Organizers, on the other hand, gain access to a professional dashboard displaying bookings, revenue, and real-time performance indicators, with the ability to export reports. The platform also includes an integrated marketplace for service providers in photography, hospitality, and decoration. Most notably, this project integrates three powerful artificial intelligence applications: an intelligent event planner that analyzes user needs and recommends the most suitable event; a virtual assistant that answers inquiries instantly; and a personalized recommendation system based on user behavior—all built on a state-of-the-art technological infrastructure that ensures performance and future expansion.

## الملخص

في ظل غياب منصة رقمية موحدة تخدم قطاع الفعاليات في السوق السوري، جاءت منصتنا لتملأ هذا الفراغ بحلٍّ متكامل يجمع تحت سقف رقمي واحد كلَّ من يصنع الفعالية ويحضرها ويخدمها. تتيح المنصة للحضور تصفّح الفعاليات وحجز تذاكرها ودفع رسومها في خطوات فورياً، بينما يحصل المنظمون على لوحة تحكم QR بسيطة مع استلام تذكرة رقمية بكود احترافية تعرض الحجوزات والإيرادات ومؤشرات الأداء لحظةً بلحظة مع إمكانية تصدير التقارير. وتمتد المنصة لتشمل سوقاً مدمجاً لمزودي الخدمات من تصوير وضيافة وديكور، والأبرز في هذا المشروع هو دمج ثلاث تطبيقات ذكاء اصطناعي حقيقية: مخطط فعاليات ذكي يحلل احتياجات المستخدم ويوصي بالفعالية الأنسب، ومساعد افتراضي يجيب على الاستفسارات فوراً، ونظام توصيات شخصية مبني على سلوك المستخدم — كل ذلك مبني على بنية تقنية حديثة تضمن الأداء والتوسع المستقبلي

## List Of Content:

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>ABSTRACT:</b> .....                                                       | <b>9</b>  |
| <b>CHAPTER 1:</b> .....                                                      | <b>20</b> |
| <b>GENERAL INTRODUCTION</b> .....                                            | <b>20</b> |
| <b>1-1 PROJECT OVERVIEW AND BACKGROUND</b> .....                             | <b>21</b> |
| <b>1-2 PROBLEM STATEMENT AND ITS PRACTICAL AND ACADEMIC IMPORTANCE</b> ..... | <b>21</b> |
| <i>problem</i> .....                                                         | <i>21</i> |
| <i>Current situation in Syria</i> .....                                      | <i>21</i> |
| <i>impact</i> .....                                                          | <i>21</i> |
| <i>Special Syrian Gaps</i> .....                                             | <i>21</i> |
| <i>Practical Importance</i> .....                                            | <i>22</i> |
| <i>Academic Importance</i> .....                                             | <i>22</i> |
| <b>1-3 PROJECT OBJECTIVES (MAIN AND SUB OBJECTIVES)</b> .....                | <b>22</b> |
| <i>Main Objective</i> .....                                                  | <i>22</i> |
| <i>Sub Objectives</i> .....                                                  | <i>22</i> |
| <b>1-4 PROJECT SCOPE, CONSTRAINTS, AND ASSUMPTIONS</b> .....                 | <b>23</b> |
| <i>Project Scope</i> .....                                                   | <i>23</i> |
| <i>Constraints</i> .....                                                     | <i>23</i> |
| <i>Assumptions</i> .....                                                     | <i>23</i> |
| <b>1-5 PROJECT CONTRIBUTION TO KNOWLEDGE AND INDUSTRY</b> .....              | <b>23</b> |
| <b>1-6 RESEARCH METHODOLOGY AND SCIENTIFIC APPROACHES</b> .....              | <b>24</b> |
| <i>Supporting Methods</i> .....                                              | <i>24</i> |
| <b>1-7 ORGANIZATIONAL STRUCTURE OF THE THESIS</b> .....                      | <b>24</b> |
| <b>1-8 DEFINITIONS AND KEY CONCEPTS</b> .....                                | <b>25</b> |
| <b>CHAPTER 2 :</b> .....                                                     | <b>26</b> |
| <b>PROJECT DEVELOPMENT &amp; MANAGEMENT METHODOLOGY</b> .....                | <b>26</b> |
| <b>2-1 SELECTED DEVELOPMENT METHODOLOGY AND JUSTIFICATION</b> .....          | <b>27</b> |
| <b>2-2 PROJECT SCHEDULE AND TIMELINE</b> .....                               | <b>27</b> |
| <b>2-3 RISK MANAGEMENT</b> .....                                             | <b>28</b> |
| <b>2-4 TEAM STRUCTURE AND ROLE DISTRIBUTION</b> .....                        | <b>29</b> |
| <b>2-5 QUALITY MANAGEMENT</b> .....                                          | <b>29</b> |
| <i>Code Quality Standards</i> .....                                          | <i>30</i> |
| <i>Testing Strategy</i> .....                                                | <i>30</i> |
| <i>Version Control and Code Review</i> .....                                 | <i>30</i> |
| <b>2-6 PROJECT MANAGEMENT TOOLS</b> .....                                    | <b>30</b> |
| <b>2-7 MONITORING AND CONTINUOUS EVALUATION</b> .....                        | <b>31</b> |
| <b>2-8 SOFTWARE CONFIGURATION MANAGEMENT</b> .....                           | <b>31</b> |
| <b>CHAPTER 3:</b> .....                                                      | <b>33</b> |
| <b>LITERATURE REVIEW &amp; PREVIOUS STUDIES</b> .....                        | <b>33</b> |
| <b>3-1 REVIEW OF EXISTING EVENT MANAGEMENT SYSTEMS:</b> .....                | <b>34</b> |
| <b>1. Bizzabo:</b> .....                                                     | <b>34</b> |

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| 2.Splash:.....                                                    | 35         |
| 3. Luma: .....                                                    | 35         |
| 5.Faaliyeh App_ Syria: .....                                      | 38         |
| <b>2-3 TECHNOLOGIES AND MODERN FRAMEWORKS IN THE FIELD: .....</b> | <b>39</b>  |
| <b>3-3 COMPARATIVE ANALYSIS: .....</b>                            | <b>40</b>  |
| <b>3-4 RESEARCH GAP AND PROPOSED INNOVATION: .....</b>            | <b>41</b>  |
| <b>3-5 THEORETICAL FRAMEWORK:.....</b>                            | <b>42</b>  |
| <b>3-6 EMERGING TRENDS IN THE FIELD: .....</b>                    | <b>43</b>  |
| <b>CHAPTER 4: .....</b>                                           | <b>44</b>  |
| <b>REQUIREMENTS MODELING AND ANALYSIS .....</b>                   | <b>44</b>  |
| <b>4-1 FEASIBILITY STUDY .....</b>                                | <b>45</b>  |
| <i>Technical Feasibility .....</i>                                | <i>45</i>  |
| <i>Economic Feasibility.....</i>                                  | <i>45</i>  |
| <b>4-2 REQUIREMENTS GATHERING .....</b>                           | <b>45</b>  |
| <b>4-3 STAKEHOLDER ANALYSIS .....</b>                             | <b>45</b>  |
| <i>End Users (Attendees).....</i>                                 | <i>45</i>  |
| <i>Organizers.....</i>                                            | <i>46</i>  |
| <i>Suppliers .....</i>                                            | <i>46</i>  |
| <i>Developers/Maintainers.....</i>                                | <i>46</i>  |
| <b>4-4 FUNCTIONAL REQUIREMENTS.....</b>                           | <b>46</b>  |
| 1. User Account Management .....                                  | 47         |
| 2. Event Discovery .....                                          | 47         |
| 3. Booking and Payment .....                                      | 47         |
| 4. Event Management for Organizers .....                          | 48         |
| 5. Dashboard and Booking Management.....                          | 48         |
| 6. Service Providers .....                                        | 49         |
| 7. Ratings and Reviews .....                                      | 49         |
| 8. Notification System .....                                      | 49         |
| 9. Reports and Analytics .....                                    | 50         |
| <b>4-5 NON-FUNCTIONAL REQUIREMENTS.....</b>                       | <b>51</b>  |
| <b>4-6 PRIORITIZATION OF REQUIREMENTS (MoSCoW).....</b>           | <b>51</b>  |
| <i>Must Have .....</i>                                            | <i>51</i>  |
| <i>Should Have .....</i>                                          | <i>52</i>  |
| <i>Could Have .....</i>                                           | <i>52</i>  |
| <i>Won't Have .....</i>                                           | <i>52</i>  |
| <b>4-7 UML DIAGRAMS.....</b>                                      | <b>52</b>  |
| 4-7-1 Main Use Case Diagrams .....                                | 52         |
| 4.7.2 use case Descriptions:.....                                 | 59         |
| 4-7-3 Activity Diagrams.....                                      | 71         |
| 4-7-4 Sequence Diagrams.....                                      | 78         |
| <b>4-8 AGILE MODELING AND METHODOLOGY IMPLEMENTATION.....</b>     | <b>98</b>  |
| 4-8-1 Story Mapping .....                                         | 98         |
| 4-8-2 Product Backlog.....                                        | 102        |
| 4-8-3 User Stories .....                                          | 105        |
| <b>4-9 REQUIREMENTS ANALYSIS AND DOCUMENTATION .....</b>          | <b>126</b> |
| <b>4-9-1 REQUIREMENTS TRACEABILITY MATRIX (RTM) .....</b>         | <b>126</b> |
| <b>CHAPTER 5: .....</b>                                           | <b>139</b> |
| <b>SYSTEM ARCHITECTURE DESIGN .....</b>                           | <b>139</b> |



|                                                   |            |
|---------------------------------------------------|------------|
| <b>5-1- HIGH-LEVEL DESIGN:</b>                    | <b>140</b> |
| 5-1-1- <i>System Block Diagram:</i>               | 141        |
| 5-1-2- <i>Architectural Design Pattern Used:</i>  | 142        |
| 5-1-3- <i>System Architecture Design:</i>         | 142        |
| <b>5-2- DATABASE DESIGN:</b>                      | <b>144</b> |
| 5-2-1- <i>Backend Architecture Diagram:</i>       | 146        |
| 5-2-2- <i>Collections&amp; Documents:</i>         | 147        |
| 5-2-3- <i>Constraints and Keys:</i>               | 148        |
| <b>5-3- DESIGNING EXTERNAL INTERFACES (APIs):</b> | <b>149</b> |
| <b>5-4- SECURITY &amp; ACCESS CONTROL DESIGN:</b> | <b>150</b> |
| <b>CHAPTER 6 :</b>                                | <b>151</b> |
| <b>IMPLEMENTATION ENVIRONMENT &amp; TOOLS</b>     | <b>151</b> |
| 6-1 PROGRAMMING LANGUAGES AND FRAMEWORKS          | 152        |
| 6-2 DEVELOPMENT ENVIRONMENT AND PROJECT STRUCTURE | 153        |
| 6-3 LIBRARIES AND DEPENDENCIES                    | 154        |
| 6-4 DATABASE SYSTEM                               | 155        |
| 6-5 EXTERNAL SERVICE INTEGRATIONS                 | 156        |
| 6-7 BUILD PROCESS AND DEPLOYMENT                  | 158        |
| 6-8 TESTING ENVIRONMENT                           | 158        |
| 6-9 SECURITY CONSIDERATIONS                       | 159        |
| 6-10 PERFORMANCE CONSIDERATIONS                   | 160        |
| 6-11 LESSONS LEARNED AND FUTURE ENHANCEMENTS      | 160        |
| <b>CHAPTER 7:</b>                                 | <b>162</b> |
| <b>SOFTWARE IMPLEMENTATION</b>                    | <b>162</b> |
| 7-1 PROJECT FILE STRUCTURE                        | 163        |
| 7-2 MODULES IMPLEMENTATION                        | 164        |
| 7-3 API IMPLEMENTATION                            | 165        |
| 7-4 DATA ACCESS LAYER IMPLEMENTATION              | 165        |
| 7-5 BUSINESS LOGIC LAYER IMPLEMENTATION           | 166        |
| 7-6 UI IMPLEMENTATION                             | 166        |
| 7-7 SMART ALGORITHMS AND ML IMPLEMENTATION        | 178        |
| <b>CHAPTER 8:</b>                                 | <b>180</b> |
| <b>TESTING &amp; EVALUATION</b>                   | <b>180</b> |
| 8-1 TEST PLAN                                     | 181        |
| 8-1-1 <i>Testing Objectives</i>                   | 181        |
| 8-1-2 <i>Scope of Testing</i>                     | 181        |
| 8-1-3 <i>Testing Environment</i>                  | 182        |
| 8-2 TYPES OF TESTS                                | 183        |
| 8-2-1 <i>Unit Testing</i>                         | 183        |
| 8-2-2 <i>Integration Testing</i>                  | 183        |
| 8-2-3 <i>System Testing</i>                       | 184        |
| 8-2-4 <i>Acceptance Testing</i>                   | 184        |
| 8-2-5 <i>Performance Testing</i>                  | 184        |
| 8-2-6 <i>Security Testing</i>                     | 185        |
| 8-2-7 <i>Usability Testing</i>                    | 185        |
| 8-3 TEST CASES AND RESULTS                        | 186        |
| 8-3-1 <i>Test Case Structure</i>                  | 186        |
| 8-3-2 <i>Test Results Summary</i>                 | 187        |

|                                                      |     |
|------------------------------------------------------|-----|
| 8-4 TEST RESULTS AND ANALYSIS .....                  | 188 |
| 8-5 QUALITY METRICS .....                            | 188 |
| 8-6 USABILITY EVALUATION .....                       | 189 |
| CHAPTER 9: .....                                     | 191 |
| RESULTS AND ANALYSIS .....                           | 191 |
| 9-1- RESULTS OF IMPLEMENTING THE SYSTEM:.....        | 192 |
| 9-2 COMPARATIVE ANALYSIS WITH EXISTING SYSTEMS:..... | 193 |
| 9-3 OBJECTIVES ACHIEVEMENT: .....                    | 194 |
| 9-4 STRENGTHS AND WEAKNESSES ANALYSIS .....          | 194 |
| 9-5 KEY PERFORMANCE INDICATORS (KPIs):.....          | 195 |
| CHAPTER 10: .....                                    | 196 |
| CONCLUSION & RECOMMENDATIONS.....                    | 196 |
| 10-1 PROJECT SUMMARY AND MAIN ACHIEVEMENTS: .....    | 197 |
| 10-2 CHALLENGES AND DIFFICULTIES: .....              | 198 |
| 10-3 FUTURE DEVELOPMENT SUGGESTIONS: .....           | 198 |
| 10-4 RECOMMENDATIONS: .....                          | 199 |
| CHAPTER 11: .....                                    | 200 |
| APPENDICES & REFERENCES.....                         | 200 |
| APPENDIX A: .....                                    | 201 |
| APPENDIX B: .....                                    | 201 |
| B-1-4 APIS DESIGN .....                              | 203 |
| B-2 INTELLIGENT SERVICES DESIGN .....                | 206 |
| STATE MACHINE DIAGRAMS.....                          | 208 |
| D-2 Testing Summary .....                            | 212 |
| D-3 Sample Test Cases.....                           | 213 |
| D-4 Performance Testing .....                        | 214 |
| D-5 Security Testing.....                            | 214 |
| D-6 AI Features Testing .....                        | 214 |
| APPENDIX E — USER MANUAL .....                       | 214 |
| E-1 System Access .....                              | 215 |
| E-2 Browsing Events .....                            | 215 |
| E-3 Booking an Event .....                           | 216 |
| E-4 Managing Bookings.....                           | 216 |
| E-5 AI Features .....                                | 216 |
| E-6 Organizer Dashboard.....                         | 217 |
| E-7 Supplier Management.....                         | 217 |
| E-8 Reviews and Ratings.....                         | 217 |
| E-9 Logout .....                                     | 218 |
| REFERENCES .....                                     | 218 |
| ARABIC AND FOREIGN REFERENCES .....                  | 218 |
| WEBSITES AND TECHNICAL DOCUMENTATION.....            | 218 |
| RESEARCH PAPERS AND SCIENTIFIC ARTICLES .....        | 218 |

## List Of Table

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| Table (1): Problem Statement and Its Practical and Academic Importance ..... | 21 |
| Table (2): Project Contribution to Knowledge and Industry.....               | 24 |
| Table (3): Risk Management .....                                             | 29 |
| Table (4): Bizzabo .....                                                     | 34 |
| Table (5): splash.....                                                       | 35 |
| Table (6): Luma .....                                                        | 36 |
| Table (7): Hopin.....                                                        | 37 |
| Table (8): Faaliyeh App_ Syria .....                                         | 38 |
| Table (9): Technologies and Modern Frameworks in the Field.....              | 39 |
| Table (10): Comparative Analysis .....                                       | 40 |
| Table (11): Research Gap and Proposed Innovation .....                       | 41 |
| Table (12): Theoretical Framework .....                                      | 42 |
| Table (13): Emerging Trends in the Field .....                               | 43 |
| Table (14): Non-Functional Requirements .....                                | 51 |
| Table (15): Use Case Specification 1 .....                                   | 59 |
| Table (16): Use Case Specification 2 .....                                   | 60 |
| Table (17): Use Case Specification 3 .....                                   | 60 |
| Table (18): Use Case Specification 4 .....                                   | 61 |
| Table (19): Use Case Specification 5 .....                                   | 61 |
| Table (20): Use Case Specification 6 .....                                   | 62 |
| Table (21): Use Case Specification 7 .....                                   | 62 |
| Table (22): Use Case Specification 8 .....                                   | 62 |
| Table (23): Use Case Specification 9 .....                                   | 63 |
| Table (24): Use Case Specification 10 .....                                  | 64 |
| Table (25): Use Case Specification 11.....                                   | 64 |
| Table (26): Use Case Specification 12 .....                                  | 64 |
| Table (27): Use Case Specification 13 .....                                  | 64 |
| Table (28): Use Case Specification 14 .....                                  | 65 |
| Table (29): Use Case Specification 15 .....                                  | 65 |
| Table (30): Use Case Specification 16 .....                                  | 65 |
| Table (31): Use Case Specification 17 .....                                  | 66 |
| Table (32): Use Case Specification 18 .....                                  | 66 |
| Table (33): Use Case Specification 19 .....                                  | 66 |
| Table (34): Use Case Specification 20 .....                                  | 67 |
| Table (35): Use Case Specification 21 .....                                  | 67 |
| Table (36): Use Case Specification 22 .....                                  | 68 |

|                                                    |     |
|----------------------------------------------------|-----|
| Table (37): Use Case Specification 23 .....        | 69  |
| Table (38): Use Case Specification 24 .....        | 69  |
| Table (39): Use Case Specification 25 .....        | 69  |
| Table (40): Use Case Specification 26 .....        | 70  |
| Table (41): Use Case Specification 27 .....        | 70  |
| Table (42): Use Case Specification 28 .....        | 70  |
| Table (43): User Personas .....                    | 98  |
| Table (44): Backbone Activities .....              | 99  |
| Table (45): MVP Release Planning.....              | 100 |
| Table (46): Product Backlog Summary .....          | 103 |
| Table (47): Backlog Statistics .....               | 104 |
| Table (48): Detailed User Stories 1.....           | 106 |
| Table (49): Detailed User Stories 2.....           | 106 |
| Table (50): Detailed User Stories 3.....           | 107 |
| Table (51): Detailed User Stories 4.....           | 108 |
| Table (52): Detailed User Stories 5.....           | 109 |
| Table (53): Detailed User Stories 6.....           | 110 |
| Table (54): Detailed User Stories 7.....           | 111 |
| Table (55): Detailed User Stories 8.....           | 112 |
| Table (56): Detailed User Stories 9.....           | 113 |
| Table (57): Detailed User Stories 10.....          | 114 |
| Table (58): Detailed User Stories 11 .....         | 115 |
| Table (59): Detailed User Stories 12 .....         | 116 |
| Table (60): Detailed User Stories 13 .....         | 117 |
| Table (61): Detailed User Stories 14.....          | 118 |
| Table (62): Detailed User Stories 15 .....         | 119 |
| Table (63): Detailed User Stories 16.....          | 120 |
| Table (64): Detailed User Stories 17.....          | 121 |
| Table (65): Detailed User Stories 18.....          | 122 |
| Table (66): Detailed User Stories 19 .....         | 123 |
| Table (67): Detailed User Stories 20 .....         | 124 |
| Table (68): User Story Format Reference .....      | 125 |
| Table (69): RTM .....                              | 134 |
| Table (70): Test Case.....                         | 138 |
| Table (71): Constraints and Keys .....             | 148 |
| Table (72): APIs .....                             | 149 |
| Table (73): Security & Access Control Design ..... | 150 |
| Table (74): Development Tools.....                 | 153 |
| Table (75): Backend Project Structure.....         | 154 |
| Table (76): frontend Project Structure .....       | 154 |
| Table (77): Backend Dependencies.....              | 154 |

|                                                             |     |
|-------------------------------------------------------------|-----|
| Table (78): frontend Dependencies .....                     | 155 |
| Table (79): Firestore vs Relational Databases.....          | 156 |
| Table (80): Layer and Tools .....                           | 159 |
| Table (81): Modules Implementation .....                    | 164 |
| Table (82): Testing Environment.....                        | 182 |
| Table (83): Performance Testing .....                       | 185 |
| Table (84): Test Case Structure .....                       | 187 |
| Table (85): Test Results Summary .....                      | 187 |
| Table (86): Quality Metrics .....                           | 188 |
| Table (87): Comparative Analysis with Existing Systems..... | 193 |
| Table (88): Objectives Achievement .....                    | 194 |
| Table (89): KPIs .....                                      | 195 |
| Table (91): D-2 Testing Summary .....                       | 213 |
| Table (92): D-3 Sample Test Cases.....                      | 213 |
| Table (93): AI Chatbot Test.....                            | 213 |
| Table (94): D-4 Performance Testing.....                    | 214 |
| Table (95): D-6 AI Features Testing .....                   | 214 |

## List Of Figure

|                                                              |    |
|--------------------------------------------------------------|----|
| Figure 1:project schedule and timeline .....                 | 27 |
| Figure2 :project schedule and timeline2 .....                | 28 |
| Figure3 :use case Event Management Module.....               | 53 |
| Figure4 :Use Case Booking Management System.....             | 54 |
| Figure5 :use case Booking Management System .....            | 55 |
| Figure6 :Use Case AI Integration Module .....                | 55 |
| Figure7 :Use Case Supplier& Servier Management Module.....   | 56 |
| Figure8 :Use Case Analytics & Reporting Module.....          | 57 |
| Figure9 :Use Case Notification System Module .....           | 57 |
| Figure10 :Use Case Review System Module.....                 | 58 |
| Figure 11:Activity 1 .....                                   | 71 |
| Figure 12:Activity 2.....                                    | 72 |
| Figure 13:Activity 3.....                                    | 73 |
| Figure14 :Activity 4.....                                    | 74 |
| Figure 15:Activity 5.....                                    | 75 |
| Figure16 : Activity For Service Management System .....      | 76 |
| Figure17 :Activity For Provider Management System .....      | 77 |
| Figure 18:Sequence For AI Powered Event Planning .....       | 78 |
| Figure 19:Sequence For Create Event .....                    | 79 |
| Figure20 :Sequence For Update Event With Notifications.....  | 80 |
| Figure21 :Sequence For Create Booking .....                  | 81 |
| Figure22 :Sequence For Cancel Booking.....                   | 82 |
| Figure23 :Sequence For View User Bookings .....              | 83 |
| Figure 24:Sequence For Register as Supplier .....            | 84 |
| Figure25 :Sequence For Add Service .....                     | 85 |
| Figure26 :Sequence For Update Service Status .....           | 86 |
| Figure27 :Sequence For Browse Available Service .....        | 87 |
| Figure28 :Sequence For Submit Review .....                   | 88 |
| Figure29 :Sequence For View Event Reviews .....              | 89 |
| Figure30 :Sequence For Delete Review .....                   | 90 |
| Figure31 :Sequence For Chat With AI Assistant .....          | 91 |
| Figure32 :Sequence For Get Smart Recommendations .....       | 92 |
| Figure33 :Sequence For Send Attendee Notification .....      | 93 |
| Figure34 :Sequence For Automated Event Reminder .....        | 94 |
| Figure35 :Sequence For Update Notification Preferences ..... | 95 |
| Figure36 :Sequence For Export Dashboard Report .....         | 96 |

|                                                               |     |
|---------------------------------------------------------------|-----|
| Figure37 :Sequence For View Organization Dashboard .....      | 97  |
| Figure 38:System Architecture Design .....                    | 143 |
| Figure 39:ER Diagram .....                                    | 146 |
| Figure 40:Collections& Documents.....                         | 147 |
| Figure 41:Discover Events1 .....                              | 167 |
| Figure 42:Discover Events2 .....                              | 168 |
| Figure 43:Discover Events3 .....                              | 168 |
| Figure 44:Discover Events4 .....                              | 169 |
| Figure 45:Discover Events5 .....                              | 169 |
| Figure 46:Discover Events6 .....                              | 170 |
| Figure 47:Discover Events7 .....                              | 170 |
| Figure 48:Discover Events8 .....                              | 171 |
| Figure 49:My Bookings.....                                    | 171 |
| Figure 50:My Booking 2 Interface .....                        | 172 |
| Figure 51:Provider Directory Interface.....                   | 172 |
| Figure 52:Welcome To EventMarketPLace Interface.....          | 173 |
| Figure 53:Dashboard Organizer Panel Interface .....           | 173 |
| Figure 54: Recent Bookings Interface .....                    | 174 |
| Figure 55: Manage Events Organizer Panel Interface .....      | 174 |
| Figure 56:Bookings Management Organizer Panel Interface ..... | 175 |
| Figure 57:Provider Directory Interface2.....                  | 175 |
| Figure 58:Provider Directory Interface3.....                  | 176 |
| Figure 59:Dashboard Report Interface .....                    | 176 |
| Figure 60:Provider Panel Interface .....                      | 177 |
| Figure 61:EventMarketPlace Assistant Interface .....          | 177 |
| Figure 62:Add New Service Interface.....                      | 177 |
| Figure 63:AI Event Planner Interface .....                    | 178 |
| Figure64 : B-1 Class and Object Diagrams .....                | 202 |
| Figure65 : B-1-2 Component Diagrams .....                     | 202 |
| Figure66 : B-1-3 Modules Design .....                         | 203 |
| Figure 67: Context Diagram.....                               | 206 |
| Figure 68: Level 1 DFD .....                                  | 207 |
| Figure 69: Booking Level 2.....                               | 207 |
| Figure 70: AI Level 2.....                                    | 208 |
| Figure71 : Booking State Machine Diagrams.....                | 209 |
| Figure72 : Payment State Machine Diagrams.....                | 210 |
| Figure73 : Supplier State Machine Diagrams .....              | 211 |
| Figure74 : Service State Machine Diagrams.....                | 212 |

**Chapter 1:**  
**General Introduction**



## 1-1 Project Overview and Background

In recent years, the world has experienced rapid digital transformation, which has influenced many sectors, including event and activity management such as concerts, conferences, and weddings. As the number of these events continues to grow, the need for digital systems that simplify organization and coordination between attendees, organizers, and service providers has become increasingly important.

This led to the idea of the Integrated Event and Activity Management Platform, which aims to provide a unified system for managing events from discovery and booking to performance tracking.

The platform is specifically designed for the Syrian environment by offering a simple and effective digital solution that fits local needs, uses free technologies, works under limited internet conditions, and connects attendees, organizers, and service providers within one system.

## 1-2 Problem Statement and Its Practical and Academic Importance

Despite noticeable technological progress, the event management industry still suffers from major gaps that reduce efficiency and weaken user experience. In Syria, these challenges are more severe.

| problem                 | Current situation in Syria      | impact                                |
|-------------------------|---------------------------------|---------------------------------------|
| Fragmented systems      | WhatsApp, Facebook, printed ads | Time loss ,errors, difficult tracking |
| Lack of smart discovery | Manual searching for hours      | Missed opportunities, unmet needs     |
| Weak analytics          | Manual Excel sheets             | Poor decisions, lost revenue          |

*Table (1): Problem Statement and Its Practical and Academic Importance*

### Special Syrian Gaps

- **Lack of digital infrastructure:** There is no unified event platform covering Damascus, Aleppo, Latakia, and Homs.
- **Economic challenges:** There is a need for low-cost, high-impact solutions suitable for local capabilities.

- **Missed opportunities:** Syrian youth (around 60% under the age of 30) are interested in concerts, celebrations, and workshops, but no smart system currently connects them to these opportunities.

For these reasons, this project aims to close these gaps.

### **Practical Importance**

- Saves organizers time and effort.
- Helps users find suitable events easily.
- Opens direct channels for local service providers.

### **Academic Importance**

This project represents a unique case study of integrating Artificial Intelligence with event management in an emerging economy.

## **1-3 Project Objectives (Main and Sub Objectives)**

### **Main Objective**

To develop a smart event platform designed for the Syrian market that combines event management with selected Artificial Intelligence technologies, serving attendees, organizers, and service providers in one integrated system.

### **Sub Objectives**

- Develop a complete booking system that allows users to browse events and reserve tickets easily using QR electronic tickets.
- Design a dashboard that helps organizers manage events and monitor bookings clearly.
- Create a marketplace for local service providers and connect them with organizers.
- Provide initial event recommendations based on user interests.
- Offer a simple chatbot system to support users and answer inquiries.
- Display indicators that help measure event performance such as attendance and cancellation rate.
- Apply essential security measures such as role-based access control and data protection.

## 1-4 Project Scope, Constraints, and Assumptions

### Project Scope

The project includes developing a responsive web application that provides core event management functions such as:

- User management
- Event organization
- Booking system
- Communication between stakeholders

It also includes basic AI-related features such as event recommendations and a virtual assistant for answering common questions.

The project focuses on local Syrian events such as concerts, social occasions, and workshops.

### Constraints

- Development must be completed within one academic semester by a small team.
- Experimental data will be used instead of real production data.
- Dependence on free tools and services may create performance limitations.
- Payment processing will be simulated only.
- The system relies on cloud infrastructure without local servers.

### Assumptions

- Users have basic knowledge of internet usage and digital platforms.
- Acceptable internet connectivity is available.
- Organizers enter accurate event information.
- A mobile application is not required at the current stage because a responsive web app is sufficient.
- Service providers update their profiles regularly.

## 1-5 Project Contribution to Knowledge and Industry

This project aims to become the first AI-powered event platform in Syria through the following contributions:

| contribution   | Current situation in Syria | Proposed project          |
|----------------|----------------------------|---------------------------|
| AI integration | Not available              | Recommendations + Chatbot |

|                     |                         |                                    |
|---------------------|-------------------------|------------------------------------|
| Unified platform    | Multiple separate tools | Three user groups in one system    |
| Local technology    | Expensive solutions     | Free tools + slow internet support |
| QR Tickets          | Paper-based             | Secure digital tickets             |
| Service Marketplace | middlemen               | Direct connection with providers   |

*Table (2): Project Contribution to Knowledge and Industry*

The project also introduces a new economic model by connecting local services directly with customers and organizers. It can serve as a global case study for applying AI in an emerging economy.

## 1-6 Research Methodology and Scientific Approaches

This thesis adopts the Engineering Design Methodology as the main framework for organizing the research and documentation. It includes the following stages:

1. **Problem Analysis:** Studying the Syrian event management environment and identifying existing gaps.
2. **Solution Design:** Defining system requirements and architectural structure.
3. **Practical Implementation:** Building the prototype and integrating AI technologies.
4. **Evaluation and Analysis:** Testing performance and measuring quantitative and qualitative results.

### Supporting Methods

- **Descriptive Analytical Method:** Used for reviewing literature and studying available solutions.
- **Experimental Method:** Used for evaluating AI features such as accuracy, speed, and effectiveness.

This mixed approach ensures comprehensive research coverage and logical connection between chapters, beginning with the problem and ending with the proposed and evaluated solution.

## 1-7 Organizational Structure of the Thesis

- **Chapter One:** General Introduction
- **Chapter Two:** Project Development and Management Methodology
- **Chapter Three:** Literature Review and Previous Studies
- **Chapter Four:** Requirements Modeling and Analysis
- **Chapter Five:** System Architecture Design

- **Chapter Six:** Implementation Environment and Tools
- **Chapter Seven:** Software Implementation
- **Chapter Eight:** Testing and Evaluation
- **Chapter Nine:** Results and Analysis
- **Chapter Ten:** Conclusion and Recommendations

## **1-8 Definitions and Key Concepts**

- **Firebase Firestore:** Cloud-based NoSQL database for real-time data.
- **KPI:** Key Performance Indicators such as attendance and revenue.
- **LLM:** Large Language Model used in AI systems.
- **OpenRouter API:** Free gateway for accessing AI models.
- **Prompt Engineering:** Designing effective AI prompts and instructions.
- **QR Code:** Verification code used for electronic tickets.
- **RBAC:** Role-Based Access Control.
- **React:** Frontend library for interactive user interfaces.
- **REST API:** Communication architecture between frontend and backend systems.
- **Spring Boot:** Java framework for backend application development.

## **Chapter 2 :**

### **Project Development & Management Methodology**

## 2-1 selected development methodology and justification

The EventMarketPlace project employed a hybrid Agile methodology combining Scrum and Kanban frameworks. This approach was chosen due to the system's multi-component architecture which required both iterative development and continuous workflow. Scrum was used to organize and plan complex features such as the AI chatbot management. On the other hand, Kanban boards, ensuring structured sprint-based development, system limit work-in-progress, were used to visualize workflow across frontend and backend development. This hybrid approach was well-suited for progress and external integrations such as React frontend, coordinating the Spring Boot backend, allowing the system to evolve flexibly while maintaining Firebase and AI services development structure.

## 2-2 Project Schedule and Timeline

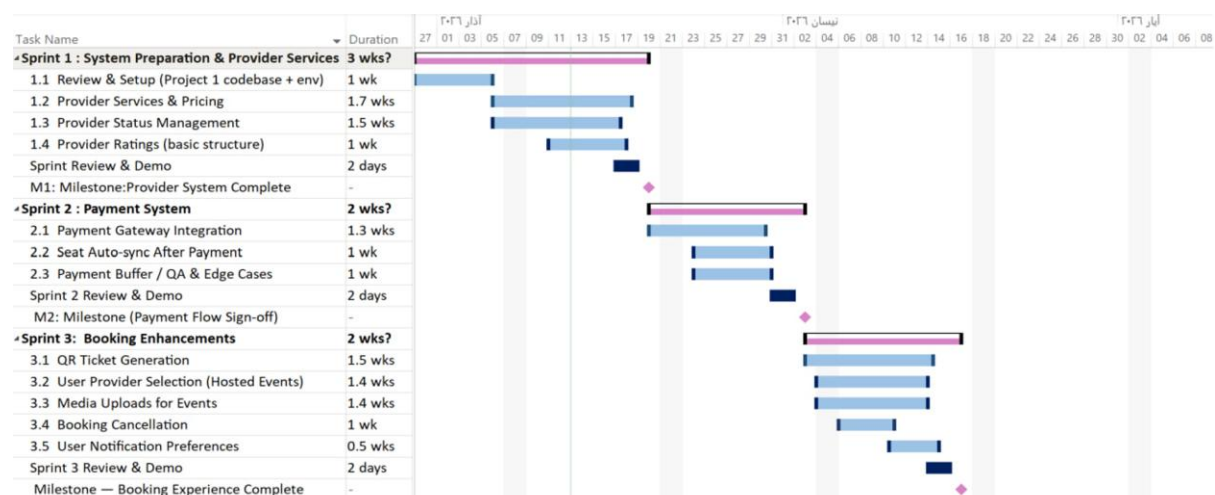


Figure 1:project schedule and timeline

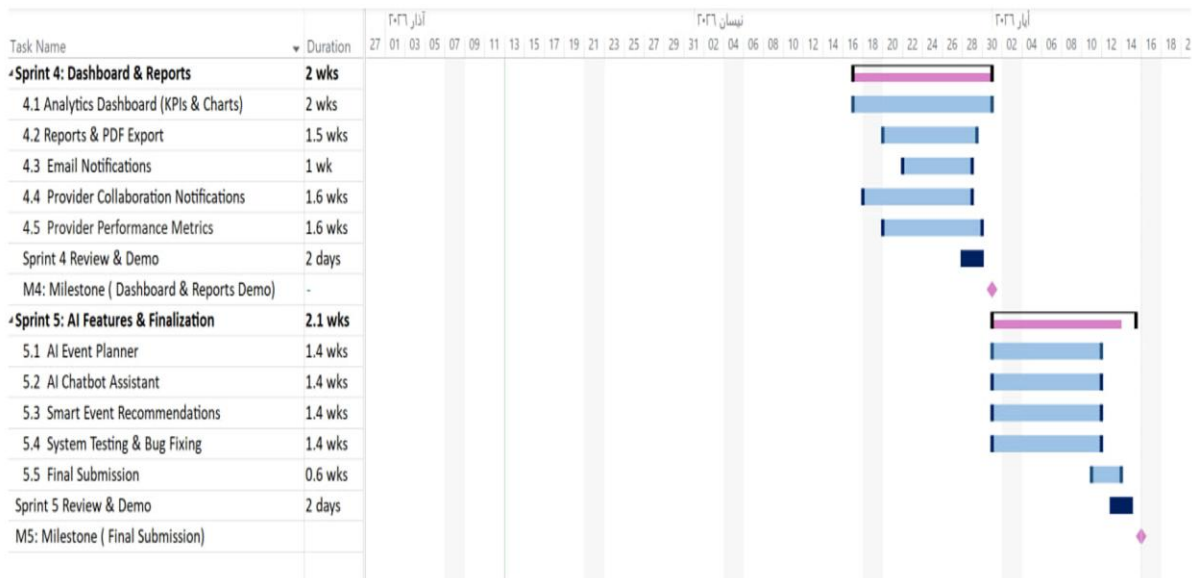


Figure2 :project schedule and timeline2

## 2-3 Risk Management

The risks were continuously reviewed throughout development, not only at the beginning of the project. At the end of each sprint, they were reassessed and mitigation strategies were adjusted when needed. This helped reduce unexpected technical issues, especially with Firebase and AI integrations.

| Risk                                                 | Probability | Impact | Mitigation Strategy                                                                             |
|------------------------------------------------------|-------------|--------|-------------------------------------------------------------------------------------------------|
| Firebase service unavailability or downtime          | Medium      | High   | Use local caching and fallback authentication to ensure core functionality during outages       |
| AI API rate limit exhaustion                         | High        | Medium | Implement request queuing, response caching, and graceful degradation when limits are reached   |
| Cross-browser and cross-device compatibility issues  | Medium      | Medium | Perform systematic testing across major browsers (Chrome, Firefox, Safari) and multiple devices |
| Data security vulnerabilities or unauthorized access | Low         | High   | Apply strict Firebase security rules, server-side validation, and role-based access control     |



|                                                         |        |     |                                                                                    |
|---------------------------------------------------------|--------|-----|------------------------------------------------------------------------------------|
| Dependency version conflicts between project components | Medium | low | Use Maven and npm lock files to maintain consistent and stable dependency versions |
|---------------------------------------------------------|--------|-----|------------------------------------------------------------------------------------|

Table (3): Risk Management

## 2-4 Team Structure and Role Distribution

The project was executed by a four-member team with clearly defined responsibilities:

### Leadership Roles:

- **Anas Abdulaziz (Product Owner):** Managed product backlog, defined acceptance criteria, and prioritized features based on business value
- **Hiba (Scrum Master):** Facilitated Agile ceremonies, removed impediments, and ensured adherence to Scrum-Kanban principles
- 

### Development Team:

- **Hiba (Frontend Developer):** Developed React components, implemented UI/UX using PrimeReact, and managed client-side Firebase integration
- **Helpest (Backend Developer):** Implemented Spring Boot REST APIs, data models, and Firebase Admin SDK integration
- **Marwa (Backend Developer):** Developed service layer components, repository patterns, and API endpoints

The frontend-backend collaboration was managed through daily synchronization meetings and shared Kanban boards, ensuring API compatibility and feature integration throughout the development lifecycle.

## 2-5 Quality Management

Quality assurance was embedded throughout the development lifecycle to ensure the reliability, maintainability, and consistency of the EventMarketPlace system. Rather than being treated as a final phase, quality was continuously enforced across all development stages.

## Code Quality Standards

The project followed structured coding practices to maintain clean and scalable code:

- A Generic Repository Pattern was implemented to ensure consistent and reusable data access across all entities.
- Lombok annotations were used to reduce boilerplate code and improve readability of backend classes.
- A unified exception handling mechanism and logging strategy were applied across all system layers to improve debugging and system reliability.

## Testing Strategy

A multi-level testing approach was adopted to ensure system correctness and stability:

- JUnit was used for backend unit testing to validate core application logic and service behaviour.
- React Testing Library was applied for frontend component testing to ensure correct UI rendering and interaction handling.
- Integration testing was conducted for REST API endpoints and database operations to verify end-to-end system functionality.

## Version Control and Code Review

Software quality was further ensured through disciplined version control practices:

- Git was used with a feature-branch workflow to isolate development work and maintain a stable main codebase.
- All pull requests were subject to peer code review to ensure code correctness, consistency, and adherence to standards.
- Maven Wrapper was used to guarantee consistent build environments across different machines, ensuring reproducible builds and reducing environment-related issues.

## 2-6 Project Management Tools

The development workflow of the EventMarketPlace project utilized several industry-standard tools to support implementation, testing, and deployment:

- **GitHub:** Version control, issue tracking, and pull request management.
- **VS Code:** Primary development environment for backend and frontend development.
- **Maven:** Backend dependency management and build automation.
- **npm:** Frontend dependency management and build scripts.
- **Firebase Console:** Database management, authentication, and backend services monitoring.

- **Postman:** REST API testing and validation.
- **OpenRouter:** External AI service integration used to support chatbot and recommendation features.

## 2-7 Monitoring and Continuous Evaluation

Progress was monitored through Agile practices combined with regular meetings with the project supervisor to ensure alignment with academic and technical requirements. Sprint planning and daily stand-ups were used to track progress and identify issues, while sprint reviews and retrospectives helped evaluate completed work and improve future development cycles.

Quality was assessed through code testing (JUnit and React Testing Library), API performance checks, and user feedback during testing phases. Continuous improvement was achieved through regular code refactoring, supervisor feedback integration, and tracking project progress using velocity and burndown charts.

## 2-8 Software Configuration Management

The project used Git as the main version control system with a structured branching strategy to support organized and continuous development.

The branching model included:

- **main:** stable and production-ready code
- **develop:** integration branch for completed features
- **feature/\*:** separate branches for developing new features
- **hotfix/\*:** urgent fixes applied to the production version

The system was divided into two main parts: a Spring Boot backend using Maven and a React frontend using npm. This separation helped keep the codebase clean and allowed both layers to be developed independently.

The project also followed semantic versioning, clear commit messages, and pull requests to ensure proper code review before merging changes. Maven Wrapper was used to maintain consistent build environments across all machines.

Overall, this approach helped maintain code stability, improve collaboration, and support a well-organized development process aligned with Agile practices.

**Chapter 3:**  
**Literature Review & Previous Studies**

### 3-1 Review of Existing Event Management Systems:

This section reviews five event management platforms selected for their relevance to the scope of this project — either due to their recent development activity or their operation in similar target markets. Each platform is evaluated on its core capabilities and limitations, with particular focus on AI integration, local market suitability, and dual-booking support.

#### 1. Bizzabo:

Description: Bizzabo focuses on enhancing attendee experience and engagement in professional events. It provides solutions for registration management and event marketing (virtual or in-person) while offering detailed reports to measure ROI and event impact.

| Advantages                                       | Disadvantages                                                                   |
|--------------------------------------------------|---------------------------------------------------------------------------------|
| 1. Well-designed and user-friendly interface.    | 1. High subscription cost.                                                      |
| 2. Advanced analytics and reporting tools.       | 2. Not suitable for small or local events.                                      |
| 3. Strong support for hybrid and virtual events. | 3. Requires time to learn advanced features                                     |
| 4. Effective networking features for attendees.  | 4. It does not support artificial intelligence or collaboration with suppliers. |

Table (4): Bizzabo

## 2.Splash:

Description: Splash is a platform specialized in designing visually appealing event pages and promoting them as part of marketing campaigns. It is used by major companies to organize brand events and manage invitations and marketing analytics.

| Advantages                                           | Disadvantages                                                       |
|------------------------------------------------------|---------------------------------------------------------------------|
| 1. Easy creation of visually attractive event pages. | 1. No ticketing or booking system.                                  |
| 2. Strong marketing and audience segmentation tools. | 2. Limited analytics for event management.                          |
| 3. Automated invitation management.                  | 3. Not suitable for non-marketing events.                           |
| 4. Integration with tools like CRM systems.          | 4. Paid features can be expensive and Limited operational features. |

Table (5): splash

## 3. Luma:

Description: Luma is a modern, simple platform designed for quick event creation, especially for communities and startups.

| <b>Advantages</b>                                      | <b>Disadvantages</b>              |
|--------------------------------------------------------|-----------------------------------|
| 1. Very simple and intuitive interface.                | 1. Limited customization options. |
| 2. Fast event creation process.                        | 2. No supplier integration.       |
| 3. Useful community management tools.                  | 3. Basic analytics only.          |
| 4. Supports recurring events and Free for free events. | 4. No AI features.                |

*Table (6): Luma*



#### 4.Hopin:

Description: Hopin is a platform specializing in organizing large virtual events .using advanced online environments.

| <b>Advantages</b>                           | <b>Disadvantages</b>                     |
|---------------------------------------------|------------------------------------------|
| 1. Comprehensive virtual event environment. | 1. High pricing and technically complex. |
| 2. Strong networking features.              | 2. Not suitable for small events.        |
| 3. Integration with external tools.         | 3. No supplier integration.              |
| 4. Detailed analytics.                      | 4. No AI features.                       |

*Table (7): Hopin*

### 5.Faaliyeh App\_ Syria:

Description: it is the official platform in Syria for organizing and managing exhibitions, projects, and events

It aims to support economic and cultural activities by bringing together government entities, organizers, and companies on a single digital platform.

It allows users to explore local events, register, and officially participate, while enabling organizers to manage requests and schedule exhibitions electronically, thus contributing to the digitization of the events sector and enhancing communication between participants and visitors.

| Advantages                                           | Disadvantages                                               |
|------------------------------------------------------|-------------------------------------------------------------|
| 1. Simple and accessible Arabic interface.           | 1. No ticketing or payment system.                          |
| 2. Covers a wide range of event types.               | 2. No user-generated events and No review or rating system. |
| 3. Simplifies application processes for exhibitions. | 3. Limited analytics capabilities.                          |
| 4. Supports visibility of local initiatives.         | 4. No AI integration.                                       |

*Table (8): Faaliyeh App\_ Syria*

## 2-3 Technologies and Modern Frameworks in the Field:

The platform is built on a unified technology stack designed to support real-time data access, scalable AI integration, and clear separation of concerns between components.

| Technology                | Layer / Role   | Function in the Project                                                                         |
|---------------------------|----------------|-------------------------------------------------------------------------------------------------|
| <b>Spring Boot</b>        | Backend        | Handles API requests, builds AI prompts, and communicates with OpenRouter API.                  |
| <b>React</b>              | Frontend       | Renders recommendation cards, ChatBot UI, and real-time AI responses.                           |
| <b>Firebase Firestore</b> | Database       | Stores and serves event/booking data in real time, feeding live data to AI features.            |
| <b>OpenRouter API</b>     | AI Gateway     | Routes requests from the backend to large language models (LLMs).                               |
| <b>LLMs</b>               | AI Engine      | Processes natural language and generates context-aware responses and recommendations.           |
| <b>REST Architecture</b>  | API Design     | Structures frontend-backend communication via defined endpoints (e.g., /api/events/plan).       |
| <b>Prompt Engineering</b> | AI Integration | Combines user inputs with live Firestore data into structured prompts before calling the model. |

*Table (9): Technologies and Modern Frameworks in the Field*

### 3-3 Comparative Analysis:

The table below evaluates all five reviewed platforms alongside the proposed system across eleven features spanning core event management functionality, local market suitability, and AI capabilities.

| Feature                         | Bizzabo | Splash | Faaliyeh | Luma | Hopin | Our System |
|---------------------------------|---------|--------|----------|------|-------|------------|
| User-friendly interface         | ✓       | ✓      | ✓        | ✓    | ✓     | ✓          |
| Ticket sales & confirmations    | ✓       | ✗      | ✗        | ✓    | ✓     | ✓          |
| Customizable hosting packages   | ✗       | ✗      | ✗        | ✗    | ✗     | ✓          |
| Private event creation          | ✗       | ✗      | ✗        | ✓    | ✓     | ✓          |
| Suitable for small/local events | ✗       | ✓      | ✓        | ✓    | ✗     | ✓          |
| Commercial event support        | ✓       | ✓      | ✗        | ✗    | ✓     | ✓          |
| Organizer dashboard             | ✓       | ✓      | ✗        | ✓    | ✓     | ✓          |
| Supplier directory              | ✗       | ✗      | ✗        | ✗    | ✗     | ✓          |
| AI Event Planner                | ✗       | ✗      | ✗        | ✗    | ✗     | ✓          |
| AI ChatBot (live data)          | ✗       | ✗      | ✗        | ✗    | ✗     | ✓          |
| Smart Recommendations           | ✗       | ✗      | ✗        | ✗    | ✗     | ✓          |

Table (10): Comparative Analysis

### 3-4 Research Gap and Proposed Innovation:

After reviewing the five platforms, it is clear that each one covers only one aspect of event management, and none of them uses AI to meet individual user needs. This project fills that gap by combining three things no existing platform offers together: private event hosting with flexible packages, a supplier directory, and AI tools that

| AI Feature                      | Technical Flow                                                                              | Gap Addressed                                                                  |
|---------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| AI Event Planner                | User enters their needs → system searches the database → AI shows the best matching events. | No platform could find the right event based on .all the user's needs at once  |
| AI ChatBot Assistant            | User asks a question → system gets live event data → AI .replies in the chat.               | No platform had a chatbot that reads live data and remembers the conversation. |
| Smart Recommendations (Planned) | System checks the user's past bookings → AI suggests events that match their interests.     | No platform showed each user a personalized list based on their own activity.  |

*Table (11): Research Gap and Proposed Innovation*

### 3-5 Theoretical Framework:

The AI features of this platform are grounded in the following well-established computational concepts, each having a direct and identifiable role in system functionality.

| Concept / The                | Application in This Project                                                                              |
|------------------------------|----------------------------------------------------------------------------------------------------------|
| REST Architecture            | A standard way for the frontend and backend to talk to each other using fixed web addresses.             |
| Large Language Models (LLMs) | AI programs that understand normal language and give useful answers — used here through OpenRouter API.  |
| Prompt Engineering           | Before calling the AI, the backend builds a message that combines the user's input with live event data. |
| Real-Time Databases          | Firebase Firestore updates data instantly, so the AI always works with the latest events.                |
| Context-Aware Computing      | The ChatBot sends the full conversation history with every request so its replies always make sense.     |
| Personalization Systems      | Smart Recommendations checks the user's past bookings and suggests events that match their interests.    |

*Table (12): Theoretical Framework*

### 3-6 Emerging Trends in the Field:

Event management platforms are moving toward systems that understand users, adapt to their needs, and handle repetitive tasks automatically. The table below shows how this project relates to the key trends shaping the field.

| Trend                        | Status in This Project | Details                                                                                                         |
|------------------------------|------------------------|-----------------------------------------------------------------------------------------------------------------|
| Large Language Models (LLMs) | Implemented            | The AI Event Planner and ChatBot are fully working and connected to live data through OpenRouter API.           |
| MLOps                        | Future Plan            | As more users join, the team will need tools to monitor and update the AI models to keep results accurate.      |
| Edge AI                      | Future Plan            | Running parts of the AI on the user's own device instead of a remote server — this would make responses faster. |
| Personalization at Scale     | In Development         | Smart Recommendations will show every user a different homepage based on their own activity in the system.      |

*Table (13): Emerging Trends in the Field*

## **Chapter 4:**

# **Requirements Modeling and Analysis**



## 4-1 Feasibility Study

### Technical Feasibility

The EventMarketPlace system demonstrates strong technical feasibility through the use of modern and well-integrated technologies. The backend is built using Spring Boot with Java, providing a solid and scalable foundation for developing RESTful APIs. The system also integrates Firebase for authentication and database services, which simplifies user management and data handling while ensuring reliability.

On the frontend, the system uses React with PrimeReact components to build a responsive single-page application. Role-based routing is implemented to control access based on user roles, ensuring proper navigation and security within the system.

In addition, the system incorporates artificial intelligence features through integration with an external AI service via OpenRouter, enabling intelligent chatbot functionality and enhanced user interaction.

### Economic Feasibility

The project is economically feasible for a student-level implementation due to its reliance on free or open-source technologies. Firebase provides a free tier suitable for development and testing, while AI services used in the system operate under cost-efficient or free usage plans. Furthermore, the use of open-source frameworks such as Spring Boot and React eliminates licensing costs entirely. This significantly reduces overall development expenses, making the system affordable and practical to implement without requiring additional financial resources.

## 4-2 Requirements Gathering

Requirements were gathered using a combination of analytical and semi-structured approaches. The process included informal interviews and discussions to understand user expectations and system needs at a high level. In addition, feedback and observations were considered during the design and development phases to refine the system requirements.

Alongside this, a detailed analysis of the system's implementation was conducted by reviewing backend services, frontend components, and overall system architecture. This helped in identifying functional requirements, user interactions, and business logic more accurately.

The combination of informal user feedback and technical system analysis ensured that the final requirements reflected both user needs and actual system capabilities.

## 4-3 Stakeholder Analysis

### End Users (Attendees)

**Needs:** Browse events, make bookings, manage personal profiles, receive notifications

**Expectations:** Intuitive interface, secure payments, timely confirmations

**Role:** Primary consumers who book events and interact with the platform through public routes and protected user dashboards

### **Organizers**

**Needs:** Create and manage events, track bookings and revenue, communicate with attendees

**Expectations:** Comprehensive analytics, easy event management tools, reliable booking system

**Role:** Content creators who utilize the organizer dashboard and event management

### **Suppliers**

**Needs:** List services, manage service offerings, track service bookings

**Expectations:** Visibility to organizers, easy service management, booking notifications

**Role:** Service providers who offer add-on services for events, managed through the supplier dashboard and service endpoints

### **Developers/Maintainers**

**Needs:** Modular codebase, clear documentation, scalable architecture

**Expectations:** Clean code patterns, comprehensive testing, version control

**Role:** Technical team responsible for system maintenance and enhancement, supported by the modular repository pattern

## **4-4 Functional Requirements**

The following functional requirements represent the complete scope of the system developed throughout both phases of the Graduation Project (Graduation Project 1 & Graduation Project 2). They include the core system requirements in addition to the advanced features and enhancements implemented during the second phase, such as artificial intelligence features, smart recommendations, and the virtual assistant.

## **1. User Account Management**

**FR1:** The system shall allow users to create a new account using their name, email address, password, and phone number.

**FR2:** The system shall allow users to log in using their registered email and password.

**FR3:** The system shall allow users to log out of their accounts at any time.

**FR4:** The system shall allow users to recover their passwords through email verification.

**FR5:** The system shall allow users to edit their account information, including name, email, phone number, and profile picture.

**FR6:** The system shall allow users to delete their accounts if they choose to do so.

**FR7:** The system shall manage different user roles and permissions, including users, organizers, and service providers.

## **2. Event Discovery**

**FR8:** The system shall allow users to browse all available events on the platform.

**FR9:** The system shall allow users to view full event details, including description, location, date, and price.

**FR10:** The system shall provide an easy-to-use event search feature.

**FR11:** The system shall allow users to filter events by date.

**FR12:** The system shall allow users to filter events by location.

**FR13:** The system shall allow users to filter events by price.

**FR14:** The system shall allow users to filter events by type, such as public events or host packages.

## **3. Booking and Payment**

**FR15:** The system shall allow users to book tickets for public events.

**FR16:** The system shall allow users to book ready-made host packages provided by organizers.

**FR17:** The system shall generate a unique reference number for each successful booking.

**FR18:** The system shall support online payment for bookings.

**FR19:** The system shall automatically update the number of available seats after payment confirmation.

**FR20:** The system shall prevent users from booking events when the maximum event capacity has been reached.

**FR21:** The system shall issue a digital ticket containing a QR code and booking information.

**FR22:** The system shall allow users to view their current and previous bookings.

**FR23:** The system shall allow users to cancel bookings before the event date.

**FR24:** The system shall send a confirmation email after a successful booking.

#### **4. Event Management for Organizers**

**FR25:** The system shall allow organizers to create new events on the platform.

**FR26:** The system shall allow organizers to edit their event details.

**FR27:** The system shall allow organizers to delete their events.

**FR28:** The system shall support two types of events: public events and host packages.

**FR29:** The system shall allow organizers to enter all event details, including title, description, location, date, time, capacity, price, images, and videos.

**FR30:** The system shall support different event statuses such as active, canceled, and completed.

**FR31:** The system shall allow organizers to enable or disable bookings for any event.

#### **5. Dashboard and Booking Management**

**FR32:** The system shall provide organizers with a dashboard to monitor the performance of their events.

**FR33:** The dashboard shall display the number of bookings for each event.

**FR34:** The dashboard shall display the number of remaining seats.

**FR35:** The dashboard shall display the total revenue generated from events.

**FR36:** The dashboard shall display attendance and cancellation rates.

**FR37:** The system shall allow organizers to view all bookings related to their events.

**FR38:** The system shall allow organizers to view attendee lists for each event.

**FR39:** The system shall allow organizers to monitor payment statuses for bookings.

## **6. Service Providers**

**FR40:** The system shall allow service providers to create accounts and public profiles containing their basic information.

**FR41:** The system shall allow service providers to add their services.

**FR42:** The system shall allow service providers to edit their services.

**FR43:** The system shall allow service providers to delete their services.

**FR44:** The system shall support different service statuses such as in progress, completed, and canceled.

**FR45:** The system shall allow service providers to view customer ratings and feedback.

**FR46:** The system shall allow users and organizers to search for service providers by service type or city.

## **7. Ratings and Reviews**

**FR47:** The system shall allow users to rate events they have attended.

**FR48:** The system shall allow users to submit ratings and reviews only for events they have attended.

**FR49:** The system shall allow users to write reviews or comments about events.

**FR50:** The system shall allow users to rate service providers.

**FR51:** The system shall display ratings and reviews publicly for all users.

## **8. Notification System**

**FR52:** The system shall allow users to enable or disable email notifications.

**FR53:** The system shall send a welcome email after successful account registration.

**FR54:** The system shall send a notification when a booking is confirmed.

**FR55:** The system shall send a notification when a booking is canceled.

**FR56:** The system shall send notifications whenever an event is updated.

**FR57:** The system shall send notifications when an event is canceled.

**FR58:** The system shall send notifications related to collaboration requests with service providers.

**FR59:** The system shall send notifications when a service status changes.

## **9. Reports and Analytics**

**FR60:** The system shall provide organizers with interactive reports and dashboards.

**FR61:** The system shall display ticket sales statistics for each event.

**FR62:** The system shall display reports showing revenue growth.

**FR63:** The system shall display information about service provider performance.

**FR64:** The system shall display the most popular events among users.

**FR65:** The system shall display the most requested and frequently used services.

**FR66:** The system shall allow organizers to export reports in PDF format.

## **10. Artificial Intelligence Features**

**FR67:** The system shall allow users to enter event preferences in order to receive suitable recommendations.

**FR68:** The system shall analyze event data using artificial intelligence techniques to provide the best recommendations.

**FR69:** The system shall display personalized event recommendations based on user interests and preferences.

**FR70:** The system shall provide an AI-powered assistant (ChatBot) to answer user questions related to events and services.

**FR71:** The system shall allow users to communicate with the AI assistant using natural language.

**FR72:** The AI assistant shall rely on real event data stored within the system.

**FR73:** The system shall display AI recommendations in interactive cards containing event details and direct booking links.

**FR74:** The system shall analyze users' booking history to provide future personalized recommendations.

**FR75:** The system shall display a “Recommended for You” section based on user interests and previous bookings.

#### 4-5 Non-Functional Requirements

| Category        | Requirement                                                                         |
|-----------------|-------------------------------------------------------------------------------------|
| Performance     | API response times under 2 seconds for standard operations                          |
| Security        | Firebase Authentication with role-based authorization                               |
| Usability       | Responsive React UI with PrimeReact components for consistent user experience       |
| Reliability     | Comprehensive error handling with try-catch blocks and user-friendly error messages |
| Maintainability | Generic repository pattern for consistent data access across all entities           |
| Scalability     | Cloud-based Firebase backend supporting horizontal scaling and real-time updates    |
| Integration     | RESTful API architecture enabling seamless frontend-backend communication           |

*Table (14): Non-Functional Requirements*

#### 4-6 Prioritization of Requirements (MoSCoW)

##### **Must Have**

Core authentication system, event creation and booking functionality, and basic dashboard analytics were classified as Must Have requirements. These form the foundation of the platform and were

implemented first, evidenced by the comprehensive **AuthService** and **BookingService** implementations.

### **Should Have**

AI chatbot integration, smart recommendations, and email notifications were categorized as Should Have. These features enhance user experience but are not critical for basic platform operation. The AI integration was implemented after core functionality, using the OpenRouter API.

### **Could Have**

Advanced analytics with PDF export, video uploads for events, and supplier management features were classified as Could Have. These provide additional value but have lower priority. The PDF export functionality using iText was implemented as an enhancement to the dashboard.

### **Won't Have**

Real-time chat between users, advanced payment gateway integration, and mobile applications were excluded from the current scope. The payment system uses a stub implementation, deferring real payment processing to future iterations.

This prioritization influenced development decisions by ensuring core functionality was delivered first, with advanced features added incrementally based on the hybrid Scrum-Kanban methodology. The modular architecture facilitated this approach by allowing features to be developed and integrated independently.

## **4-7 UML Diagrams**

UML diagrams are used to visually represent the system structure, user interactions, and workflow processes. They help simplify the analysis and design of the system by providing a clear understanding of its functionalities and component relationships before implementation.

### **4-7-1 Main Use Case Diagrams**

Main Use Case Diagrams illustrate the core functionalities and services provided by the system, as well as the relationships between different users and the system itself. These diagrams provide an overall view of the primary operations that can be performed within the platform, such as event management, booking, payment, and interaction with AI-powered features.

Use Case Diagram:



### Event Management Module - Use Case Diagram

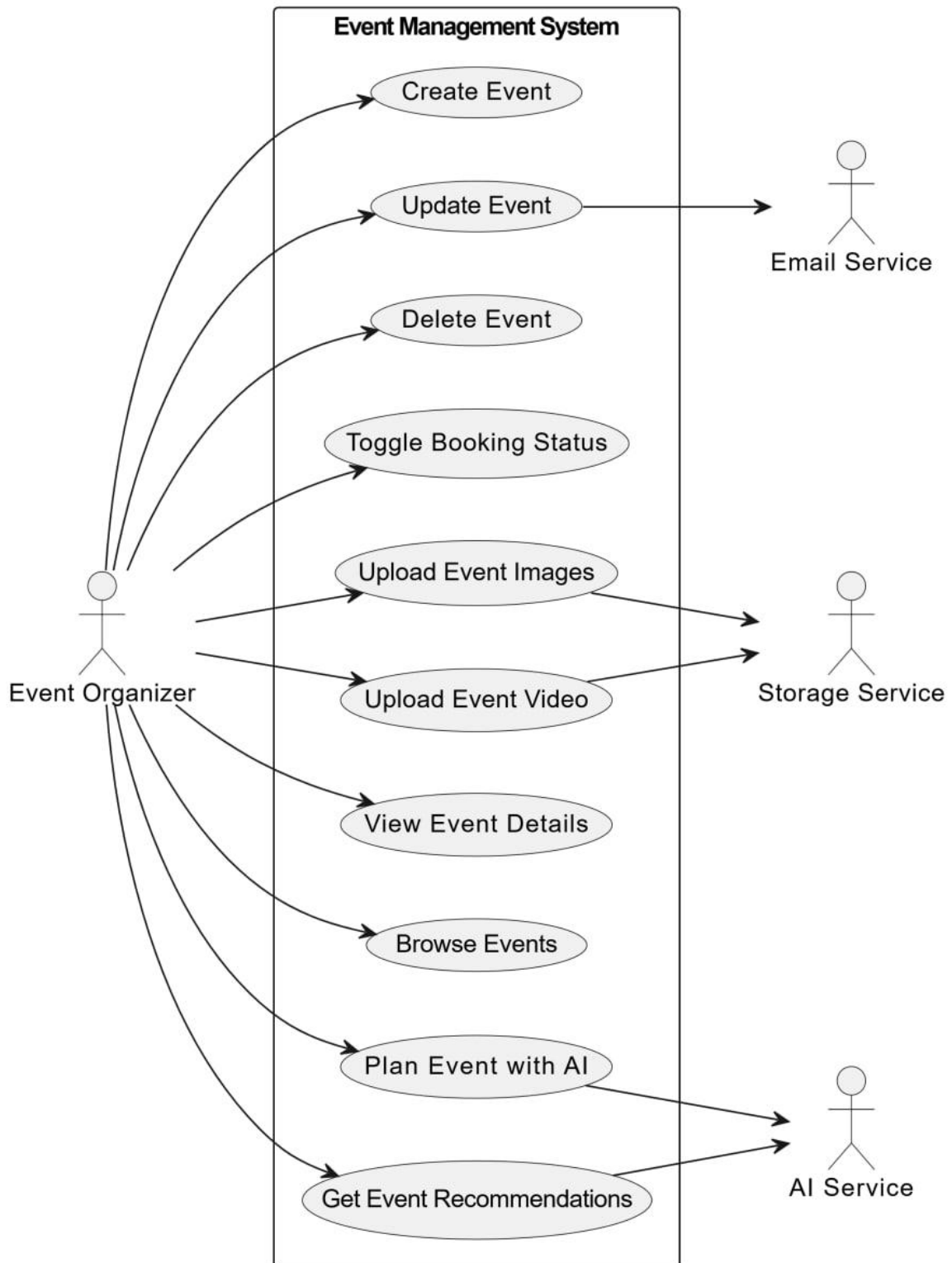


Figure3 :use case Event Management Module

## Booking Management System - Use Case Diagram

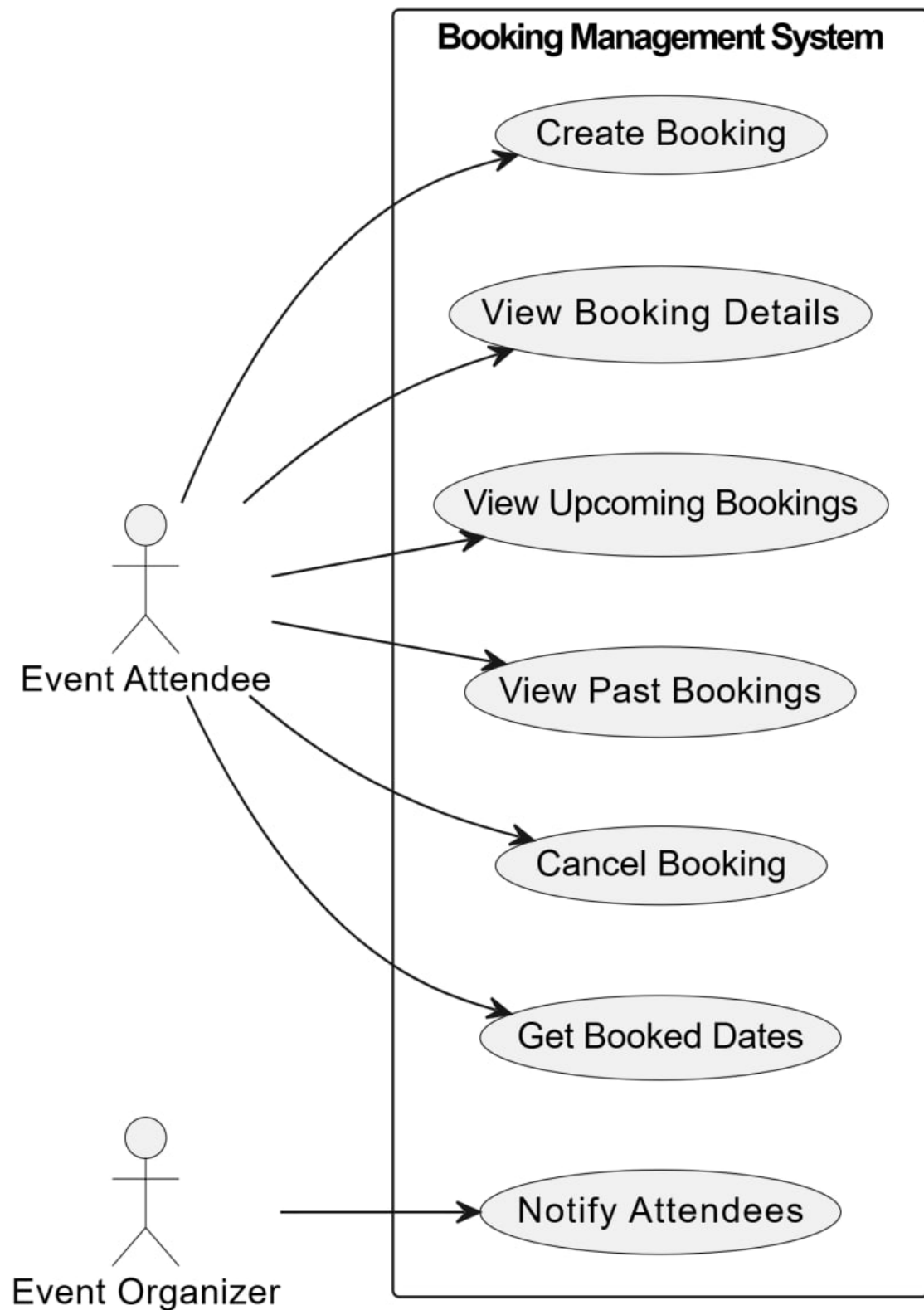


Figure4: Use Case Booking Management System

### Booking Management System - Use Case Diagram

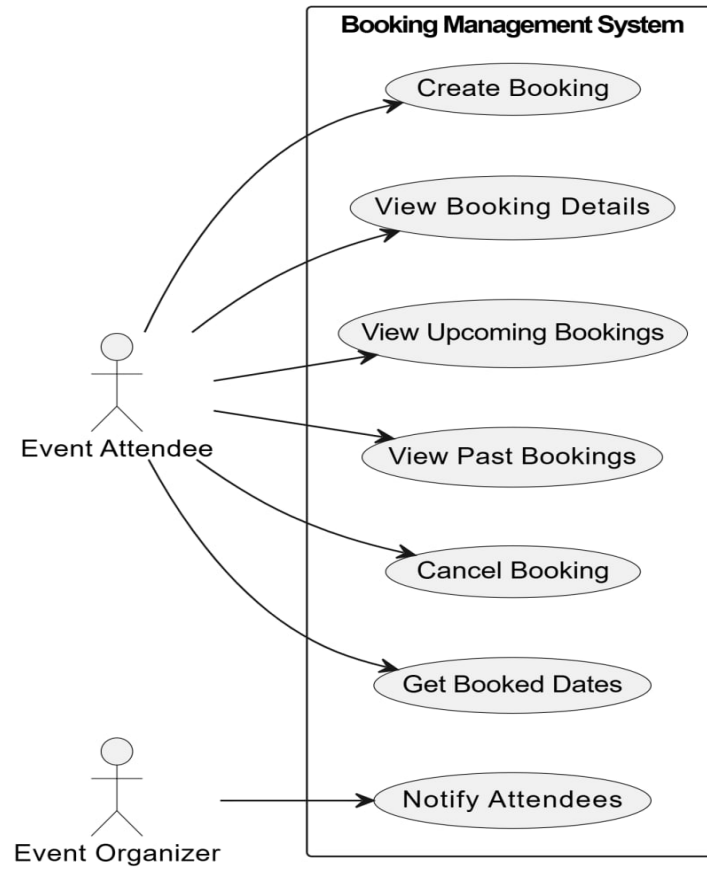


Figure5 :use case Booking Management System

### AI Integration Module - Use Case Diagram

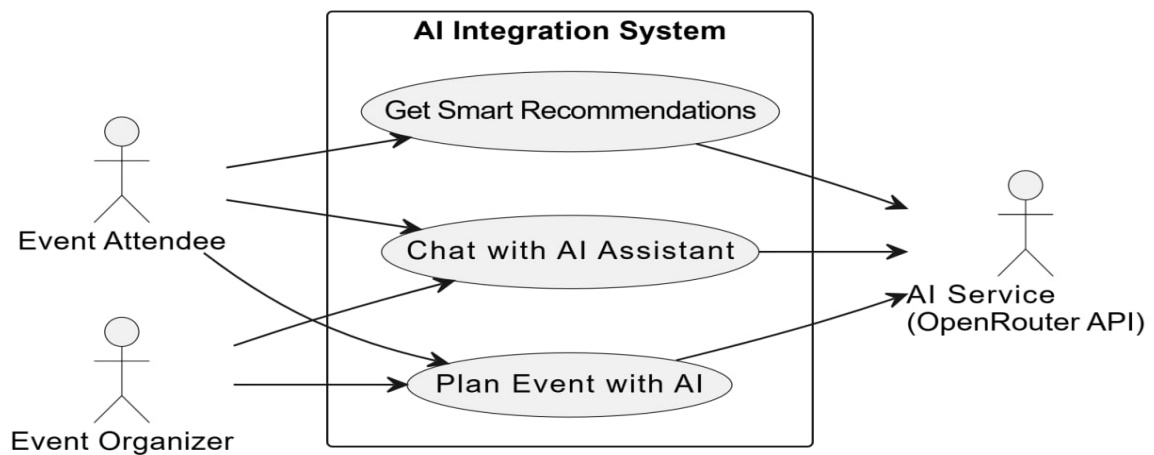


Figure6 :Use Case AI Integration Module

Supplier & Service Management Module - Use Case Diagram

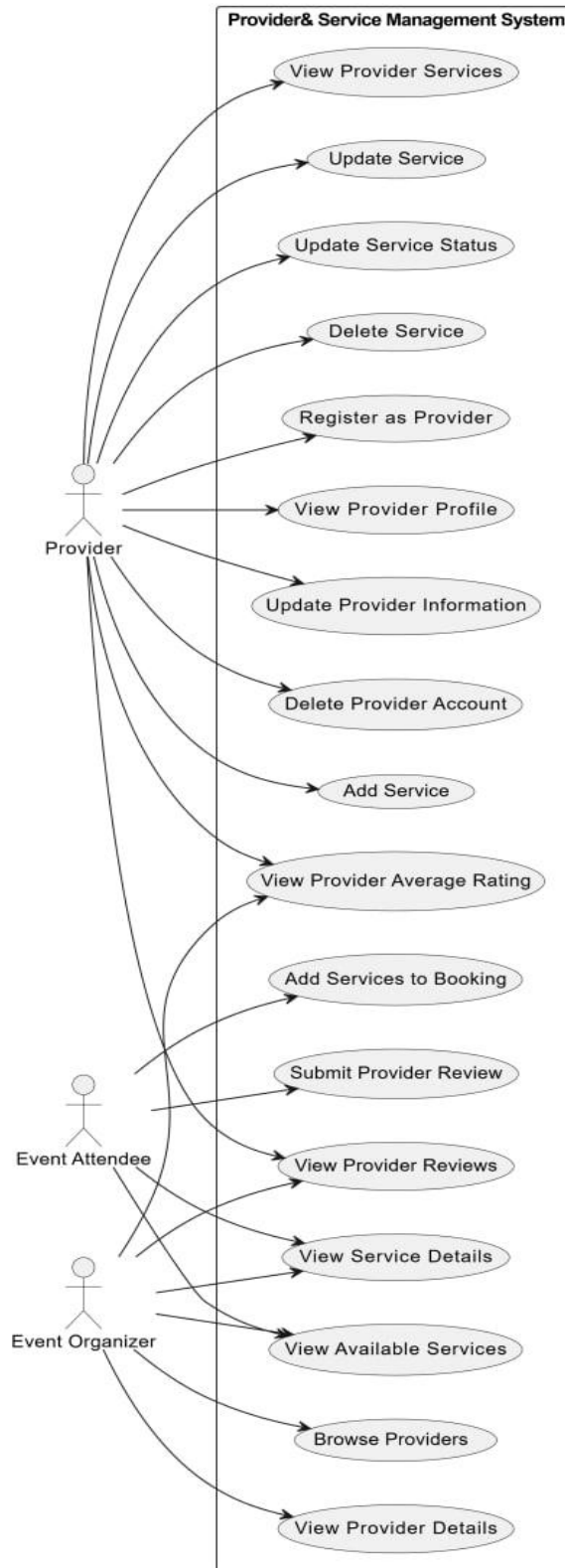
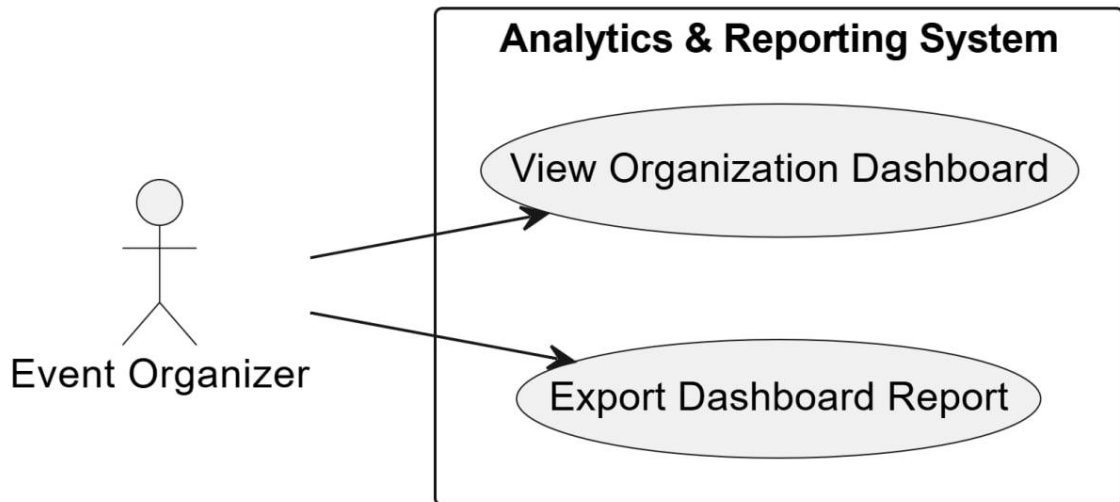


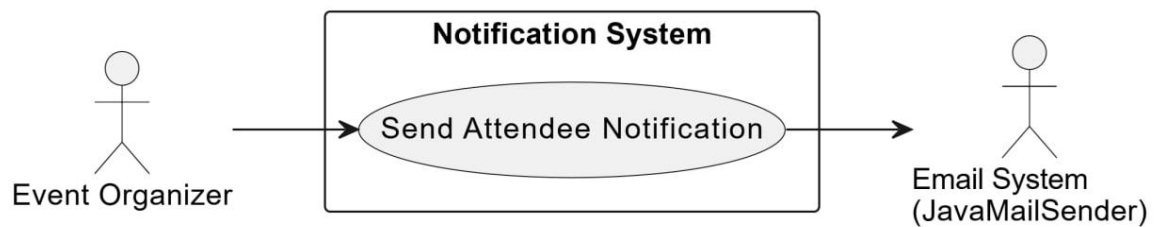
Figure 7: Use Case Supplier & Service Management Module

## Analytics & Reporting Module - Use Case Diagram



*Figure8 :Use Case Analytics & Reporting Module*

## Notification System Module - Use Case Diagram



*Figure9 :Use Case Notification System Module*

## Review System Module - Use Case Diagram

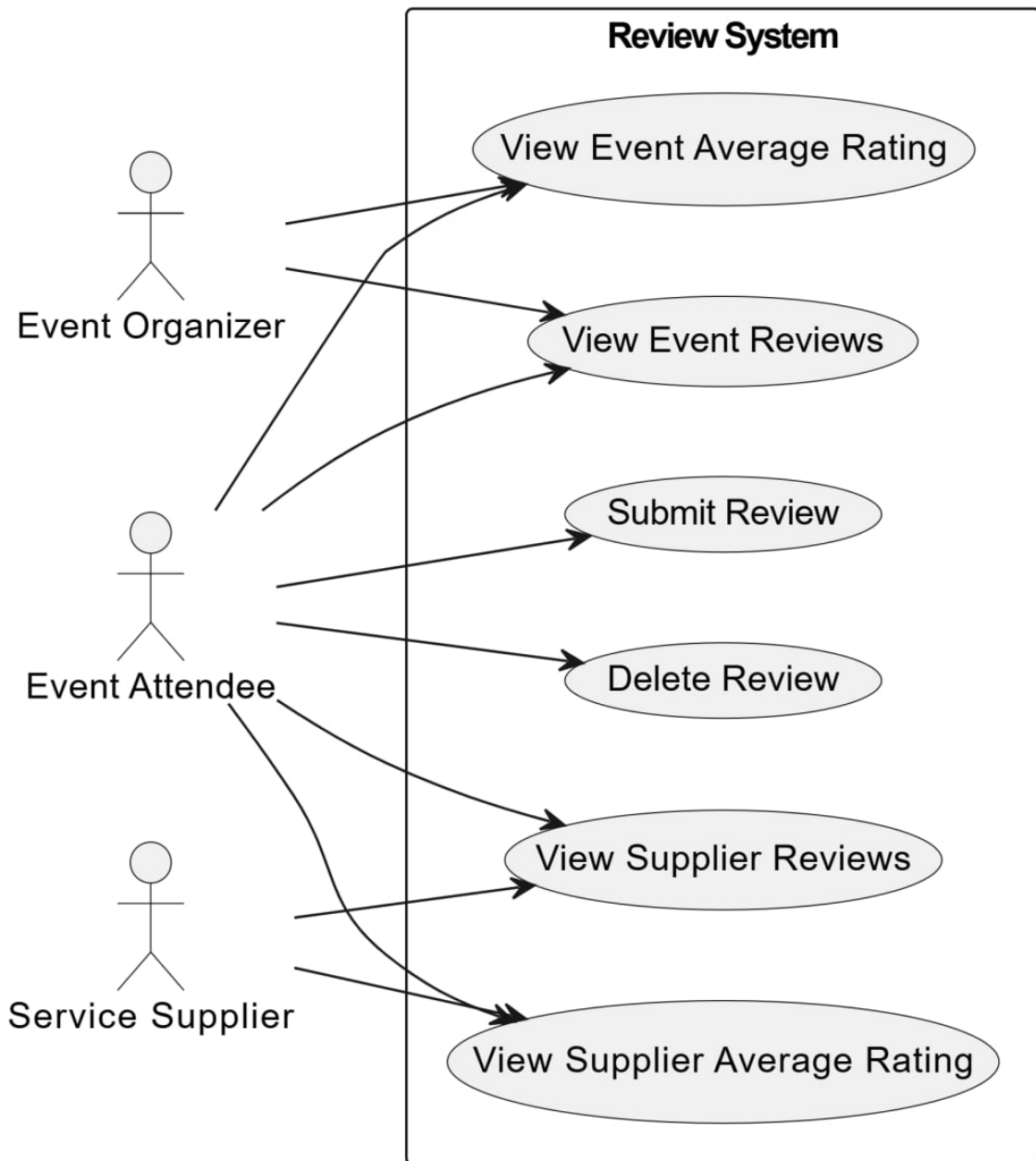


Figure 10 : Use Case Review System Module

#### 4.7.2 use case Descriptions:

Use Case Descriptions aim to provide a detailed explanation of each main use case in the system by identifying the use case name, the responsible actor, the main execution flow, and the possible alternative scenarios. This description helps in understanding how users interact with the system and how different functionalities are executed in a structured and accurate manner.

### 1.Event management module:

|                       |                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b> | <b>Create event</b>                                                                                                                                                                                                                                                                                                                                                         |
| <b>Actor(s):</b>      | Event Organizer                                                                                                                                                                                                                                                                                                                                                             |
| <b>Basic path:</b>    | <ol style="list-style-type: none"><li>1. Organizer selects Create Event.</li><li>2. System displays event creation form.</li><li>3. Organizer enters event details and submits.</li><li>4. System validates entered data.</li><li>5. System creates new event record.</li><li>6. System saves event to database.</li><li>7. System displays confirmation message.</li></ol> |
| <b>Alternative:</b>   | <ul style="list-style-type: none"><li>- 4a. Required fields are missing → System displays validation errors and highlights missing fields.</li><li>- 4b. Invalid data format (e.g., date, email) → System displays error and requests correction.</li></ul>                                                                                                                 |

Table (15): Use Case Specification 1

|                       |                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b> | <b>Update event</b>                                                                                                                                                                                                                                                                                                                                |
| <b>Actor(s):</b>      | Event Organizer , Email Service                                                                                                                                                                                                                                                                                                                    |
| <b>Basic path:</b>    | <ol style="list-style-type: none"><li>1.Organizer selects an event to update. System displays event details in editable form.</li><li>2.Organizer modifies event data and submits.</li><li>3.System validates updated data.</li><li>4.System updates event record and saves to database.</li><li>5.System checks for confirmed bookings.</li></ol> |

|                          |                                                                                                                                                                                                                                                                           |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          | 6.If confirmed bookings exist, System sends update notifications via Email Service.<br>7.System displays confirmation message to Organizer.                                                                                                                               |
| <b>Alternative path:</b> | <ul style="list-style-type: none"> <li>• 42a. Event not found → System displays error message.</li> <li>• 4a. Required fields missing / invalid format → System displays validation errors.</li> <li>• 6a. No confirmed bookings → System skips notifications.</li> </ul> |

Table (16): Use Case Specification 2

|                          |                                                                                                                                                                                                                          |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Delete event</b>                                                                                                                                                                                                      |
| <b>Actor(s):</b>         | Event Organizer                                                                                                                                                                                                          |
| <b>Basic path:</b>       | 1.Organizer selects an event to delete.<br>2.System displays delete confirmation dialog.<br>3.Organizer confirms deletion.<br>4.System deletes event from database.<br>5. System displays deletion confirmation message. |
| <b>Alternative path:</b> | <ul style="list-style-type: none"> <li>• 2a. Event not found → System displays error message.</li> <li>• 3a. Organizer cancels deletion → System aborts process and returns to event details.</li> </ul>                 |

Table (17): Use Case Specification 3

|                       |                                                                                                                                                                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b> | <b>Toggle Booking Status</b>                                                                                                                                                                                                                                            |
| <b>Actor(s):</b>      | Event Organizer                                                                                                                                                                                                                                                         |
| <b>Basic path:</b>    | 1. Organizer selects an event.<br>2. System displays current booking status (Open/Closed).<br>3. Organizer requests to toggle booking status.<br>4. System updates booking status.<br>5. System saves updated event record.<br>6. System displays confirmation message. |



|                          |                                                      |
|--------------------------|------------------------------------------------------|
| <b>Alternative path:</b> | 2a. Event not found → System displays error message. |
|--------------------------|------------------------------------------------------|

Table (18): Use Case Specification 4

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Upload Event Media</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Actor(s):</b>         | Event Organizer , Storage Service                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Basic path:</b>       | <ol style="list-style-type: none"> <li>1. Organizer selects an event and chooses to upload media.</li> <li>2. System displays the file upload dialog.</li> <li>3. Organizer selects files (images or videos) and clicks <b>Upload</b>.</li> <li>4. System validates:</li> <li>5. File type (Image / Video)</li> <li>6. File size (within allowed limits)</li> <li>7. System sends the files to the Storage Service.</li> <li>8. Storage Service stores the files and returns permanent URLs.</li> <li>9. System saves the URLs in the database</li> <li>10. System displays a confirmation message to the Organizer.</li> </ol> |
| <b>Alternative path:</b> | <ul style="list-style-type: none"> <li>• 4a. Unsupported file type → System displays an error and requests a valid file.</li> <li>• 4b. File size exceeds limit → System displays an error and requests a smaller file.</li> <li>• 6a. Storage Service failure → System displays an error and allows retry.</li> <li>• 3a. Organizer cancels upload → System aborts the operation and returns to the previous page.</li> </ul>                                                                                                                                                                                                  |

Table (19): Use Case Specification 5

|                       |                                                                                                                                                 |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b> | <b>View Event Details</b>                                                                                                                       |
| <b>Actor(s):</b>      | Event Organizer                                                                                                                                 |
| <b>Basic path:</b>    | <ol style="list-style-type: none"> <li>1. Organizer selects an event to view.</li> <li>2. System retrieves event data from database.</li> </ol> |

|                          |                                                                                                        |
|--------------------------|--------------------------------------------------------------------------------------------------------|
|                          | 3. System displays event details.                                                                      |
| <b>Alternative path:</b> | <ul style="list-style-type: none"> <li>2a. Event not found → System displays error message.</li> </ul> |

Table (20): Use Case Specification 6

|                          |                                                                                                                                                                                  |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Browse Events</b>                                                                                                                                                             |
| <b>Actor(s):</b>         | Event Organizer                                                                                                                                                                  |
| <b>Basic path:</b>       | <ol style="list-style-type: none"> <li>1. Organizer selects Browse Events.</li> <li>2. System retrieves all available events.</li> <li>3. System displays event list.</li> </ol> |
| <b>Alternative path:</b> | <ul style="list-style-type: none"> <li>2a. No events exist → System displays empty list message.</li> </ul>                                                                      |

Table (21): Use Case Specification 7

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Plan Event with AI</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Actor(s):</b>         | Event Organizer , AI Service                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Basic path:</b>       | <ol style="list-style-type: none"> <li>1. Organizer selects Plan Event with AI.</li> <li>2. System displays planning form and collects event requirements.</li> <li>3. Organizer submits planning request.</li> <li>4. System filters events by type and status.</li> <li>5. System builds AI prompt with event requirements.</li> <li>6. System sends request to AI Service (OpenRouter API).</li> <li>7. AI Service returns event recommendations.</li> <li>8. System maps AI response to event objects.</li> <li>9. System displays recommendations to Organizer.</li> </ol> |
| <b>Alternative path:</b> | <ul style="list-style-type: none"> <li>4a. No matching events → System uses active events as fallback.</li> <li>6a. API call fails → System displays error message.</li> <li>7a. No recommendations → System logs warning and displays empty result message.</li> </ul>                                                                                                                                                                                                                                                                                                         |

Table (22): Use Case Specification 8

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Chatbot</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Actor(s):</b>         | Event Organizer , AI Service                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Basic path:</b>       | <ol style="list-style-type: none"> <li>1. Organizer opens the Chatbot (text interface).</li> <li>2. Chatbot greets the Organizer and indicates available query topics</li> <li>3. Organizer types a question or request.</li> <li>4. Chatbot parses the text and queries the Event Database for relevant information.</li> <li>5. Database returns results .</li> <li>6. Chatbot formats the results as a text response and displays them to the Organizer.</li> <li>7. Chatbot suggests follow-up actions or additional queries .</li> </ol> |
| <b>Alternative path:</b> | <ul style="list-style-type: none"> <li>• 3a. Invalid or unrecognized question → Chatbot asks for clarification.</li> <li>• 4a. No matching data in the database → Chatbot returns a friendly message, e.g., “No events found matching your query.”</li> <li>• 5a. Database connection failure → Chatbot displays an error and suggests retry.</li> <li>• 3b. Organizer cancels query → Chatbot returns to main menu.</li> </ul>                                                                                                               |

Table (23): Use Case Specification 9

## 2. Booking Management Module

|                          |                                                                                                                                                                                                                                                                                             |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Create Booking</b>                                                                                                                                                                                                                                                                       |
| <b>Actor(s):</b>         | Event Attendee                                                                                                                                                                                                                                                                              |
| <b>Basic path:</b>       | <ol style="list-style-type: none"><li>1. Attendee selects an event and submits booking.</li><li>2. System validates event availability.</li><li>3. System creates booking record.</li><li>4. System saves booking to database.</li><li>5. Attendee receives booking confirmation.</li></ol> |
| <b>Alternative path:</b> | <ol style="list-style-type: none"><li>2a. Event is closed or full → System returns unavailability error.</li></ol>                                                                                                                                                                          |

Table (24): Use Case Specification 10

|                          |                                                                                                                                                                                                |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>View Booking Details</b>                                                                                                                                                                    |
| <b>Actor(s):</b>         | Event Attendee                                                                                                                                                                                 |
| <b>Basic path:</b>       | <ol style="list-style-type: none"><li>1. Attendee requests specific booking details.</li><li>2. System retrieves booking record.</li><li>3. System returns booking data to attendee.</li></ol> |
| <b>Alternative path:</b> | <ol style="list-style-type: none"><li>2a. Booking not found → System returns error.</li></ol>                                                                                                  |

Table (25): Use Case Specification 11

|                          |                                                                                                                                                                                                           |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>View Upcoming Bookings</b>                                                                                                                                                                             |
| <b>Actor(s):</b>         | Event Attendee                                                                                                                                                                                            |
| <b>Basic path:</b>       | <ol style="list-style-type: none"><li>1. Attendee requests list of upcoming bookings.</li><li>2. System retrieves all future bookings for attendee.</li><li>3. System returns list to attendee.</li></ol> |
| <b>Alternative path:</b> | <ol style="list-style-type: none"><li>2a. No upcoming bookings → System returns empty list.</li></ol>                                                                                                     |

Table (26): Use Case Specification 12

|                          |                                                                                                                                                                                              |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>View Past Bookings</b>                                                                                                                                                                    |
| <b>Actor(s):</b>         | Event Attendee                                                                                                                                                                               |
| <b>Basic path:</b>       | <ol style="list-style-type: none"><li>1. Attendee requests list of past bookings.</li><li>2. System retrieves all historical bookings.</li><li>3. System returns list to attendee.</li></ol> |
| <b>Alternative path:</b> | <ol style="list-style-type: none"><li>2a. No past bookings → System returns empty list.</li></ol>                                                                                            |

Table (27): Use Case Specification 13

|                          |                                                                                                                                                                                                                                                                               |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Cancel Booking</b>                                                                                                                                                                                                                                                         |
| <b>Actor(s):</b>         | <b>Event Attendee</b>                                                                                                                                                                                                                                                         |
| <b>Basic path:</b>       | <ol style="list-style-type: none"> <li>Attendee requests booking cancellation.</li> <li>System retrieves booking record.</li> <li>System updates booking status to cancelled.</li> <li>System saves changes.</li> <li>Attendee receives cancellation confirmation.</li> </ol> |
| <b>Alternative path:</b> | <ol style="list-style-type: none"> <li>2a. Booking not found → System returns error.</li> <li>3a. Booking already cancelled → System returns status error.</li> </ol>                                                                                                         |

*Table (28): Use Case Specification 14*

|                          |                                                                                                                                                                                                        |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Get Booked Dates</b>                                                                                                                                                                                |
| <b>Actor(s):</b>         | <b>Event Attendee</b>                                                                                                                                                                                  |
| <b>Basic path:</b>       | <ol style="list-style-type: none"> <li>Attendee requests their booked event dates.</li> <li>System retrieves all confirmed bookings.</li> <li>System extracts and returns date information.</li> </ol> |
| <b>Alternative path:</b> | 2a. No bookings exist → System returns empty list.                                                                                                                                                     |

*Table (29): Use Case Specification 15*

|                          |                                                                                                                                                                                                                                                              |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Notify Attendees</b>                                                                                                                                                                                                                                      |
| <b>Actor(s):</b>         | <b>Event Organizer</b>                                                                                                                                                                                                                                       |
| <b>Basic path:</b>       | <ol style="list-style-type: none"> <li>Organizer triggers attendee notification.</li> <li>System retrieves confirmed bookings for the event.</li> <li>System sends notifications to all attendees.</li> <li>Organizer receives send confirmation.</li> </ol> |
| <b>Alternative path:</b> | 2a. No confirmed bookings → No notifications sent                                                                                                                                                                                                            |

*Table (30): Use Case Specification 16*

#### 4. Supplier & Service Management Module

|                          |                                                                                                                                                                                                                                                                                                       |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Manage Provider Profile</b>                                                                                                                                                                                                                                                                        |
| <b>Actor(s):</b>         | Provider                                                                                                                                                                                                                                                                                              |
| <b>Basic path:</b>       | <ol style="list-style-type: none"> <li>1. Provider registers, views, updates, or deletes profile information.</li> <li>2. System validates the request.</li> <li>3. System processes and updates profile data.</li> <li>4. Confirmation or profile information is returned to the provider</li> </ol> |
| <b>Alternative path:</b> | If provider/user is not found → system displays an error message                                                                                                                                                                                                                                      |

Table (31): Use Case Specification 17

|                          |                                                                                                                                                                                                                                                                                      |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Manage Services</b>                                                                                                                                                                                                                                                               |
| <b>Actor(s):</b>         | Provider                                                                                                                                                                                                                                                                             |
| <b>Basic path:</b>       | <ol style="list-style-type: none"> <li>1. Provider creates, updates, deletes, views, or changes service status.</li> <li>2. System validates service data.</li> <li>3. System updates the service information.</li> <li>4. System saves changes and returns confirmation.</li> </ol> |
| <b>Alternative path:</b> | If service not found or required fields are missing → system displays an error message.                                                                                                                                                                                              |

Table (32): Use Case Specification 18

|                          |                                                                                                                                                                                                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Manage Provider Reviews</b>                                                                                                                                                                                                                                        |
| <b>Actor(s):</b>         | Provider , Event Organizer                                                                                                                                                                                                                                            |
| <b>Basic path:</b>       | <ol style="list-style-type: none"> <li>1. User submits or views provider reviews and ratings.</li> <li>2. System validates the request.</li> <li>3. System retrieves or stores review data.</li> <li>4. System returns review information or confirmation.</li> </ol> |
| <b>Alternative path:</b> | If no reviews exist or booking is invalid → system returns empty result or error message..                                                                                                                                                                            |

Table (33): Use Case Specification 19

|                          |                                                                                                                                   |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Browse Providers and Services</b>                                                                                              |
| <b>Actor(s):</b>         | Event Attendee, Event Organizer                                                                                                   |
| <b>Basic path:</b>       | 1.User browses providers or services.<br>2.System retrieves requested data.<br>3.System displays provider or service information. |
| <b>Alternative path:</b> | If no providers/services found → system returns empty result                                                                      |

*Table (34): Use Case Specification 20*

|                          |                                                                                                                                                                  |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Add Services to Booking</b>                                                                                                                                   |
| <b>Actor(s):</b>         | Event Attendee, Provider                                                                                                                                         |
| <b>Basic path:</b>       | 1.User requests to add services to a booking.<br>2.System validates booking and service data.<br>3.System updates the booking.<br>4.System returns confirmation. |
| <b>Alternative path:</b> | If booking or service not found → system displays an error message.                                                                                              |

*Table (35): Use Case Specification 21*

## 4. Review System Module

| Use Case Name                       | Actor(s)                           | Basic Path                                                                                                                                                            | Alternative Path                                                                                                           |
|-------------------------------------|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>View Event Average Rating</b>    | Event Organizer<br>Event Attendee  | 1. Actor requests event average rating.<br>2. System aggregates all event reviews.<br>3. System calculates and returns average score.                                 | 2a. No reviews exist → System returns zero or default.                                                                     |
| <b>View Event Reviews</b>           | Event Organizer<br>Event Attendee  | 1. Actor requests list of reviews for an event.<br>2. System retrieves all associated reviews.<br>3. System returns review list.                                      | 2a. No reviews found → System returns empty list.                                                                          |
| <b>Submit Review</b>                | Event Attendee                     | 1. Attendee submits a review (rating + comment).<br>2. System validates attendee attended the event.<br>3. System saves review.<br>4. Attendee receives confirmation. | 2a. Attendee has no booking → System rejects submission.<br>2b. Review already submitted → System returns duplicate error. |
| <b>Delete Review</b>                | Event Attendee                     | 1. Attendee requests deletion of their review.<br>2. System verifies attendee owns the review.<br>3. System deletes review.<br>4. Attendee receives confirmation.     | 2a. Review not found or not owned → System returns error.                                                                  |
| <b>View Supplier Reviews</b>        | Event Attendee<br>Service Supplier | 1. Actor requests reviews for a supplier.<br>2. System retrieves all reviews for supplier.<br>3. System returns review list.                                          | 2a. No reviews exist → System returns empty list.                                                                          |
| <b>View Supplier Average Rating</b> | Event Attendee<br>Service Supplier | 1. Actor requests average rating for a supplier.<br>2. System aggregates all supplier reviews.<br>3. System returns calculated average.                               | 2a. No reviews exist → System returns zero or default.                                                                     |

Table (36): Use Case Specification 22



## 5. Notification System Module

|                          |                                                                                                                                                                                                                                           |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Send Attendee Notification</b>                                                                                                                                                                                                         |
| <b>Actor(s):</b>         | Event Organizer<br>Email System (JavaMailSender)                                                                                                                                                                                          |
| <b>Basic path:</b>       | 1. Event Organizer triggers a notification.<br>2. System composes notification message.<br>3. System sends message via JavaMailSender.<br>4. Email System delivers notification to attendees.<br>5. Organizer receives send confirmation. |
| <b>Alternative path:</b> | 3a. Email service unavailable → System logs error and returns failure.<br>3b. Invalid attendee email → System skips that recipient and continues                                                                                          |

Table (37): Use Case Specification 23

## 6. Analytics & Reporting Module

|                          |                                                                                                                                                                                                                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>View Organization Dashboard</b>                                                                                                                                                                                                                                                      |
| <b>Actor(s):</b>         | Event Organizer                                                                                                                                                                                                                                                                         |
| <b>Basic path:</b>       | 1. Organizer navigates to the analytics dashboard.<br>2. System retrieves aggregate data (events, bookings, revenue, reviews).<br>3. System computes KPIs and metrics.<br>4. System renders dashboard with charts and summaries.<br>5. Organizer views organizational performance data. |
| <b>Alternative path:</b> | 2a. No data available → System displays empty state with guidance.                                                                                                                                                                                                                      |

Table (38): Use Case Specification 24

|                          |                                                                                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Export Dashboard Report</b>                                                                                                                                                                |
| <b>Actor(s):</b>         | Event Organizer                                                                                                                                                                               |
| <b>Basic path:</b>       | 1. Organizer requests export of current dashboard data.<br>2. System compiles data into report format.<br>3. System generates downloadable report file.<br>4. Organizer downloads the report. |
| <b>Alternative path:</b> | 2a. Data retrieval fails → System returns export error.<br>3a. Unsupported format requested → System returns format error.                                                                    |

Table (39): Use Case Specification 25

## 7. AI Integration Module

|                          |                                                                                                                                                                                                                                                                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Chat with AI Assistant</b>                                                                                                                                                                                                                                                                                                           |
| <b>Actor(s):</b>         | Event Attendee<br>Event Organizer<br>AI Service (OpenRouter API)                                                                                                                                                                                                                                                                        |
| <b>Basic path:</b>       | 1. User sends a chat message.<br>2. System loads conversation history.<br>3. System retrieves active events.<br>4. System builds event context.<br>5. System assembles AI context message.<br>6. System sends to AI Service via OpenRouter API.<br>7. AI returns response.<br>8. System formats response.<br>9. User receives AI reply. |
| <b>Alternative path:</b> | 3a. No active events → System returns empty list.                                                                                                                                                                                                                                                                                       |

|  |                                                                                                                                                                                      |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | 4a. No booking history → System sends prompt without history context.<br>6a. API call fails → System returns error.<br>8a. No matched events → System uses top 3 events as fallback. |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Table (40): Use Case Specification 26

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Get Smart Recommendations</b>                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Actor(s):</b>         | Event Attendee<br>AI Service (OpenRouter API)                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Basic path:</b>       | 1. Attendee requests personalized event recommendations.<br>2. System retrieves attendee's past bookings.<br>3. System retrieves active events.<br>4. System extracts past event titles for context.<br>5. System builds AI prompt with user history.<br>6. System sends request to AI Service.<br>7. AI processes and returns recommendations.<br>8. System parses response and maps to event objects.<br>9. Attendee receives personalized event list. |
| <b>Alternative path:</b> | 6a. API call fails → System returns error message to user.<br>7a. Empty response → System returns fallback message.                                                                                                                                                                                                                                                                                                                                      |

Table (41): Use Case Specification 27

|                          |                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name:</b>    | <b>Plan Event with AI</b>                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Actor(s):</b>         | Event attendee<br>Event Organizer<br>AI Service (OpenRouter API)                                                                                                                                                                                                                                                                                                                                            |
| <b>Basic path:</b>       | 1. Organizer submits event planning request with requirements.<br>2. System retrieves available events.<br>3. System filters by type and status.<br>4. System prepares event list and builds AI prompt.<br>5. System sends request to AI Service.<br>6. AI returns planning recommendations.<br>7. System parses recommendations and maps to event objects.<br>8. Organizer receives event recommendations. |
| <b>Alternative path:</b> | 3a. No matching events → System uses all active events as fallback.<br>5a. API call fails → System returns error.<br>7a. No results mapped → System logs warning                                                                                                                                                                                                                                            |

Table (42): Use Case Specification 28

### 4-7-3 Activity Diagrams

Activity Diagrams are used to represent the workflow within the system and to illustrate the sequence of activities and steps performed during the execution of a specific function. These diagrams help in understanding the workflow, data movement, and decision-making process throughout the various stages of the system's main operations.

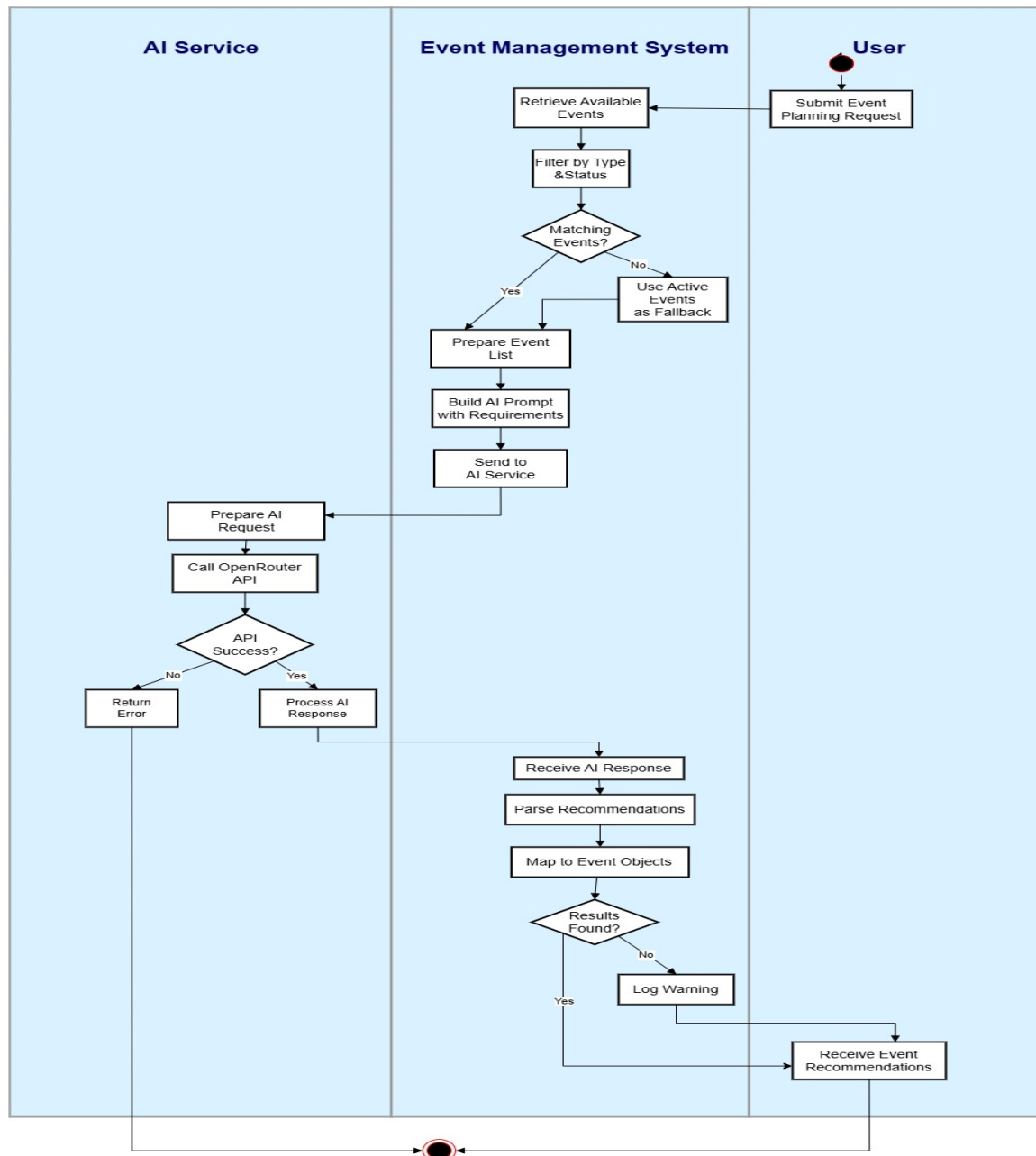


Figure 11: Activity 1

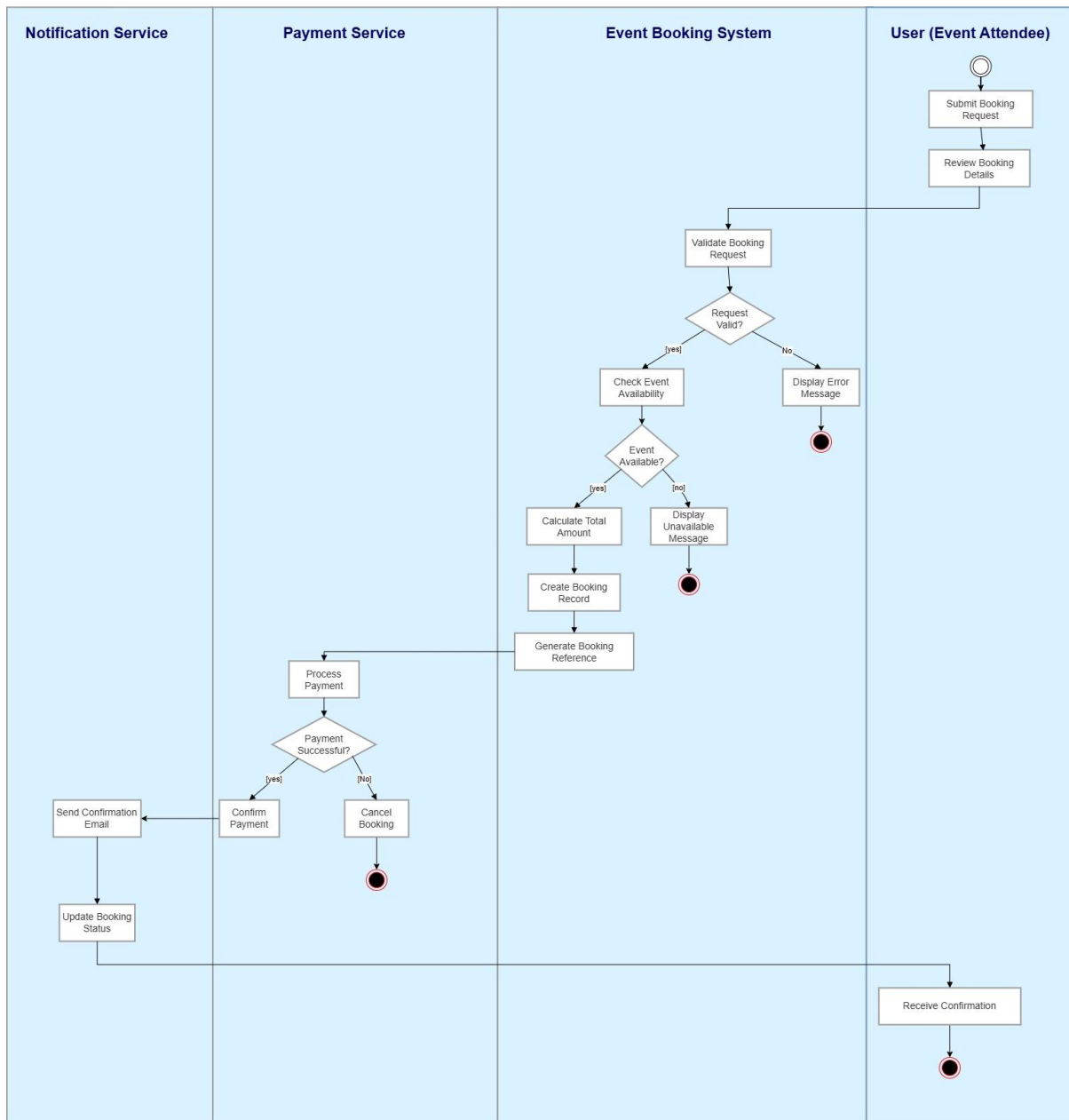


Figure 12:Activity 2

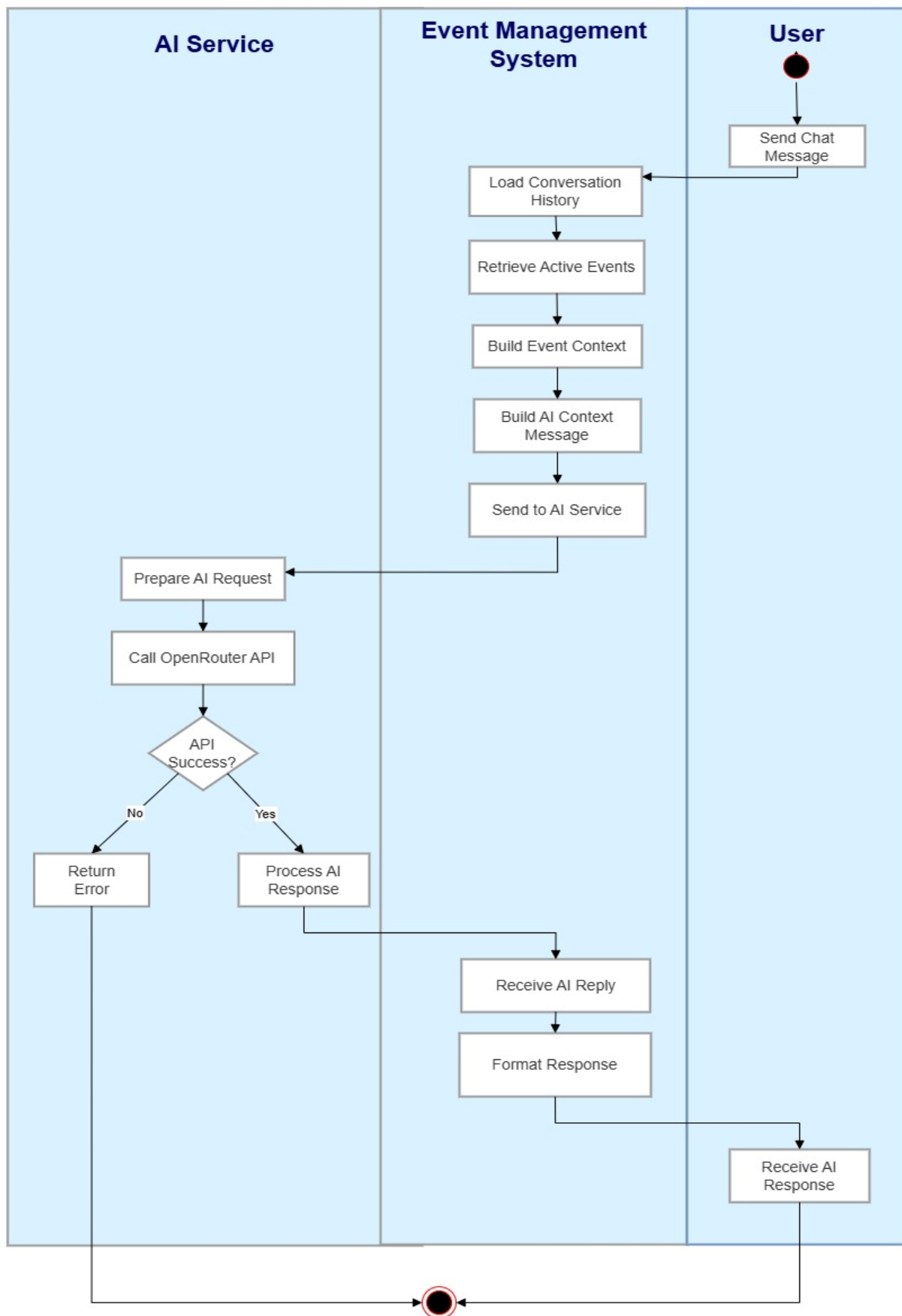


Figure 13:Activity 3

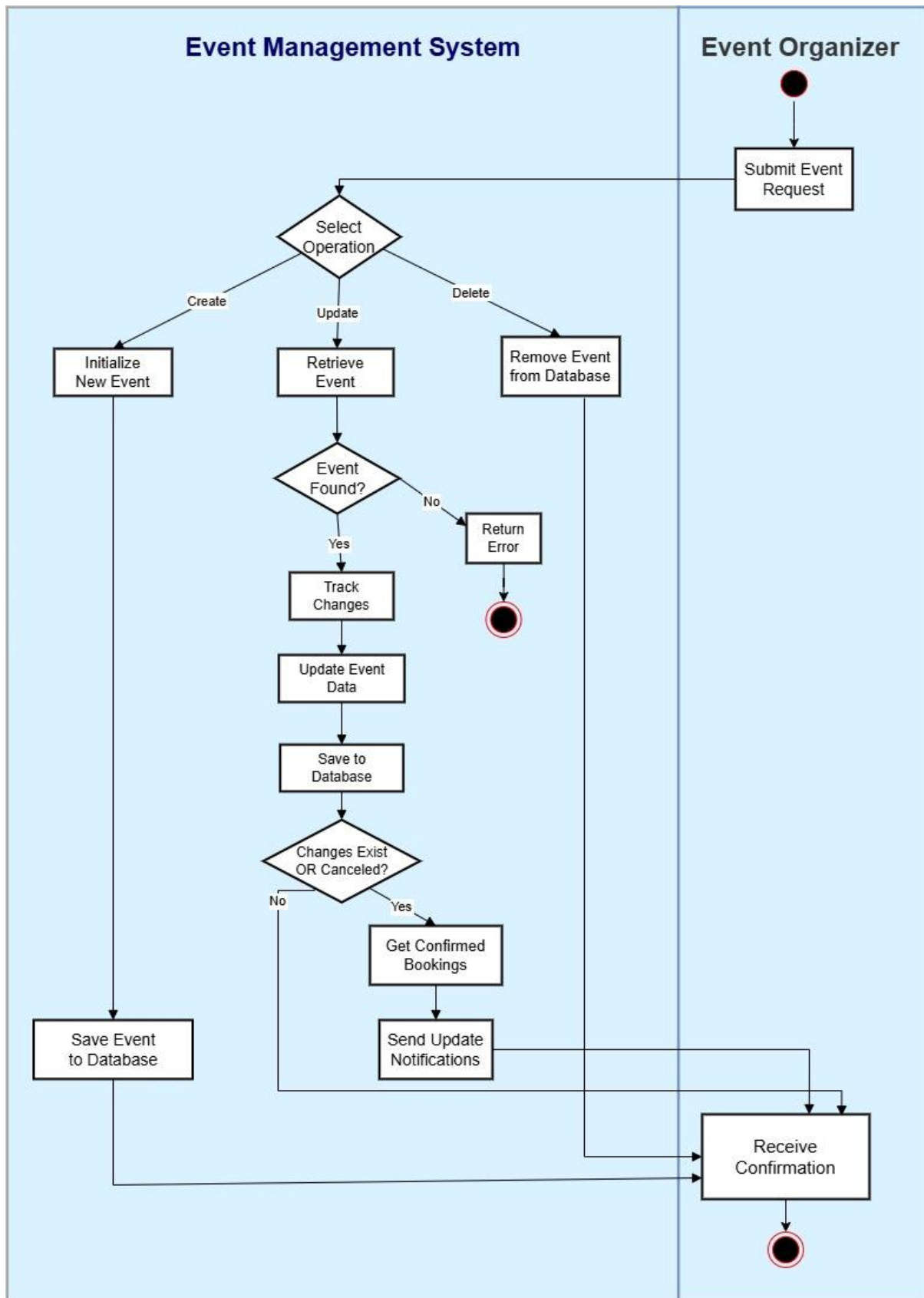


Figure14 :Activity 4

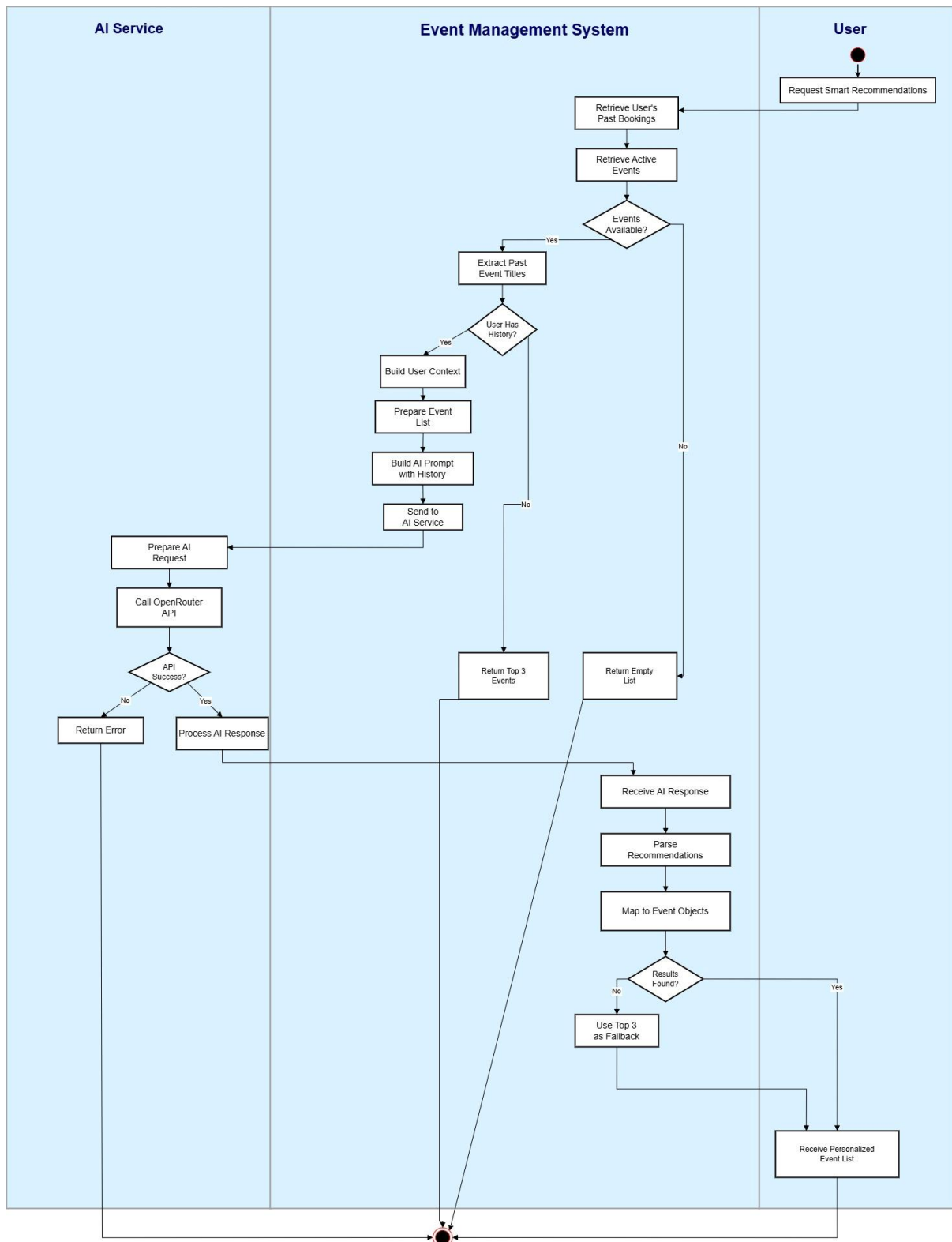


Figure 15: Activity 5

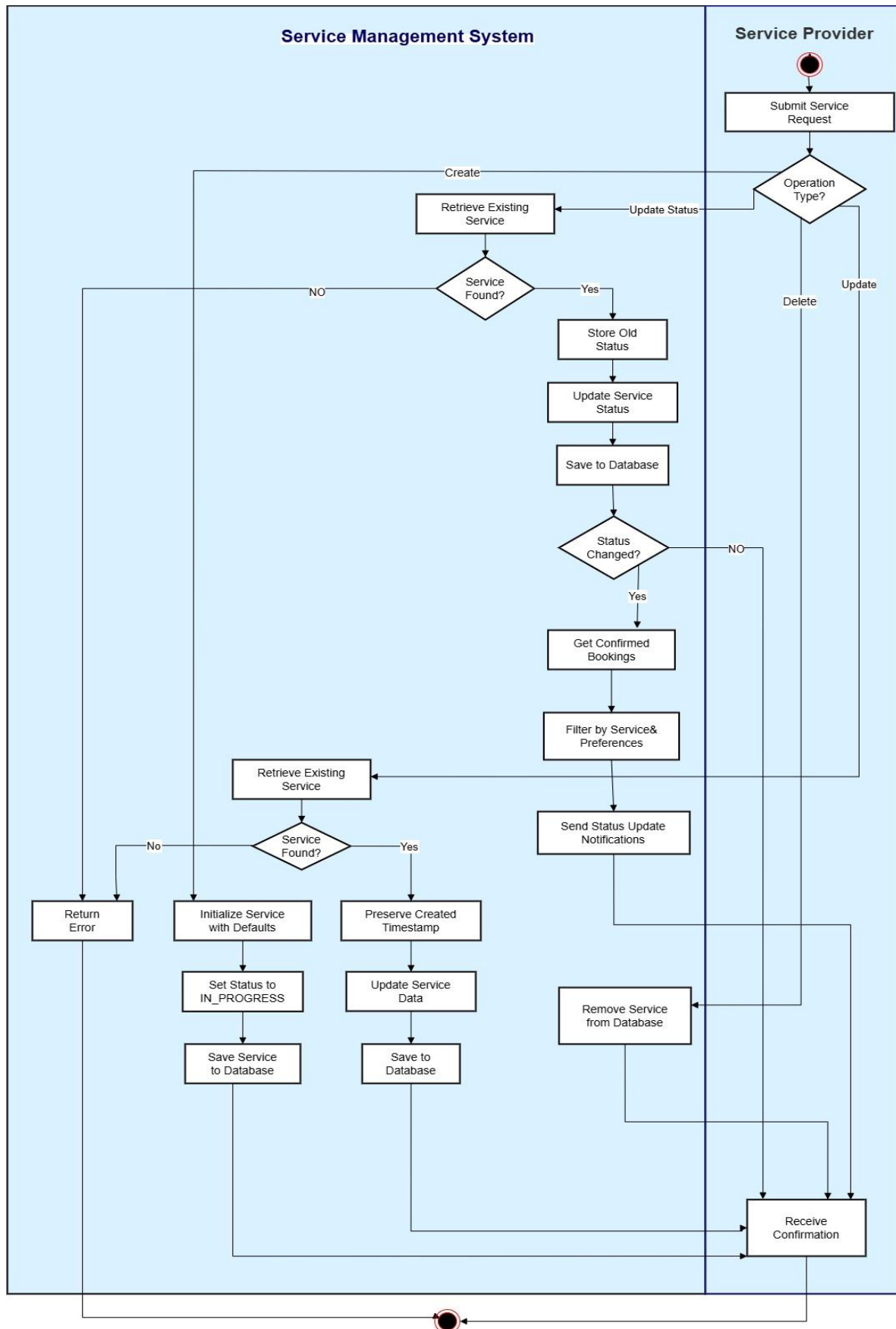


Figure 16 : Activity For Service Management System



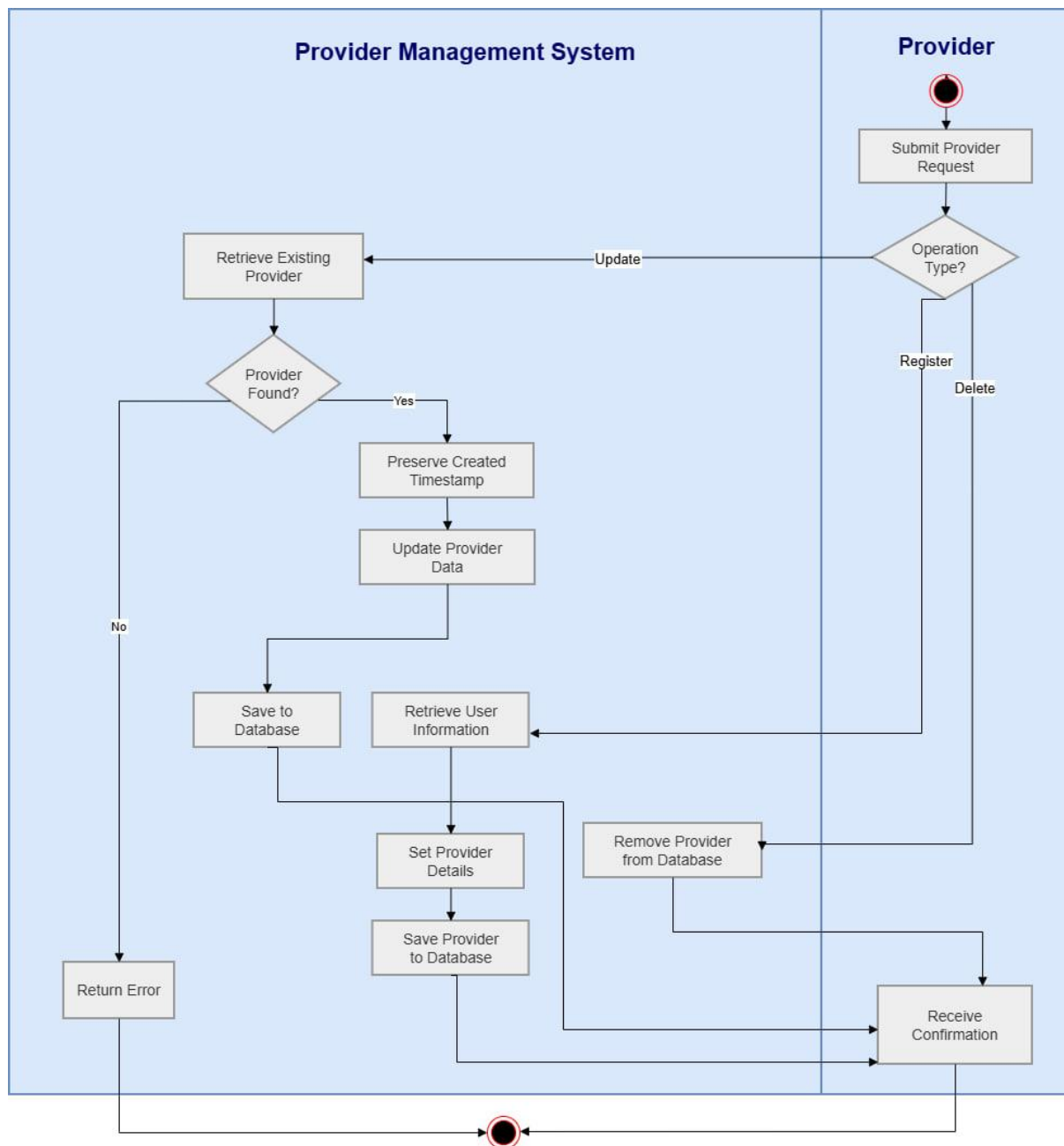


Figure 17: Activity For Provider Management System

#### 4-7-4 Sequence Diagrams

Sequence Diagrams illustrate how messages and interactions are exchanged between different system components over time while executing the main operations. These diagrams help explain the communication process between the frontend, backend, database, and AI services, providing a clear understanding of the logical sequence of operations within the system

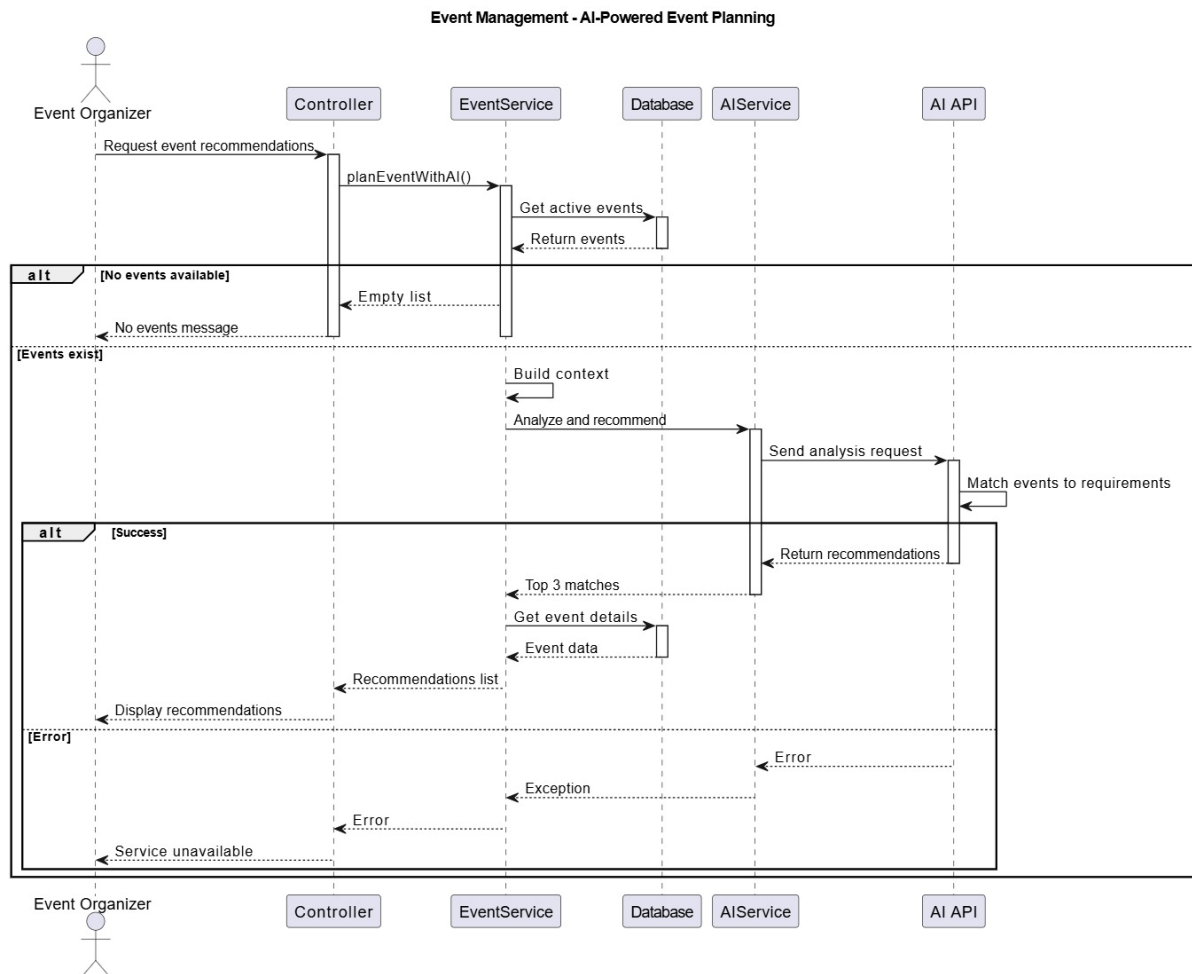


Figure 18: Sequence For AI Powered Event Planning

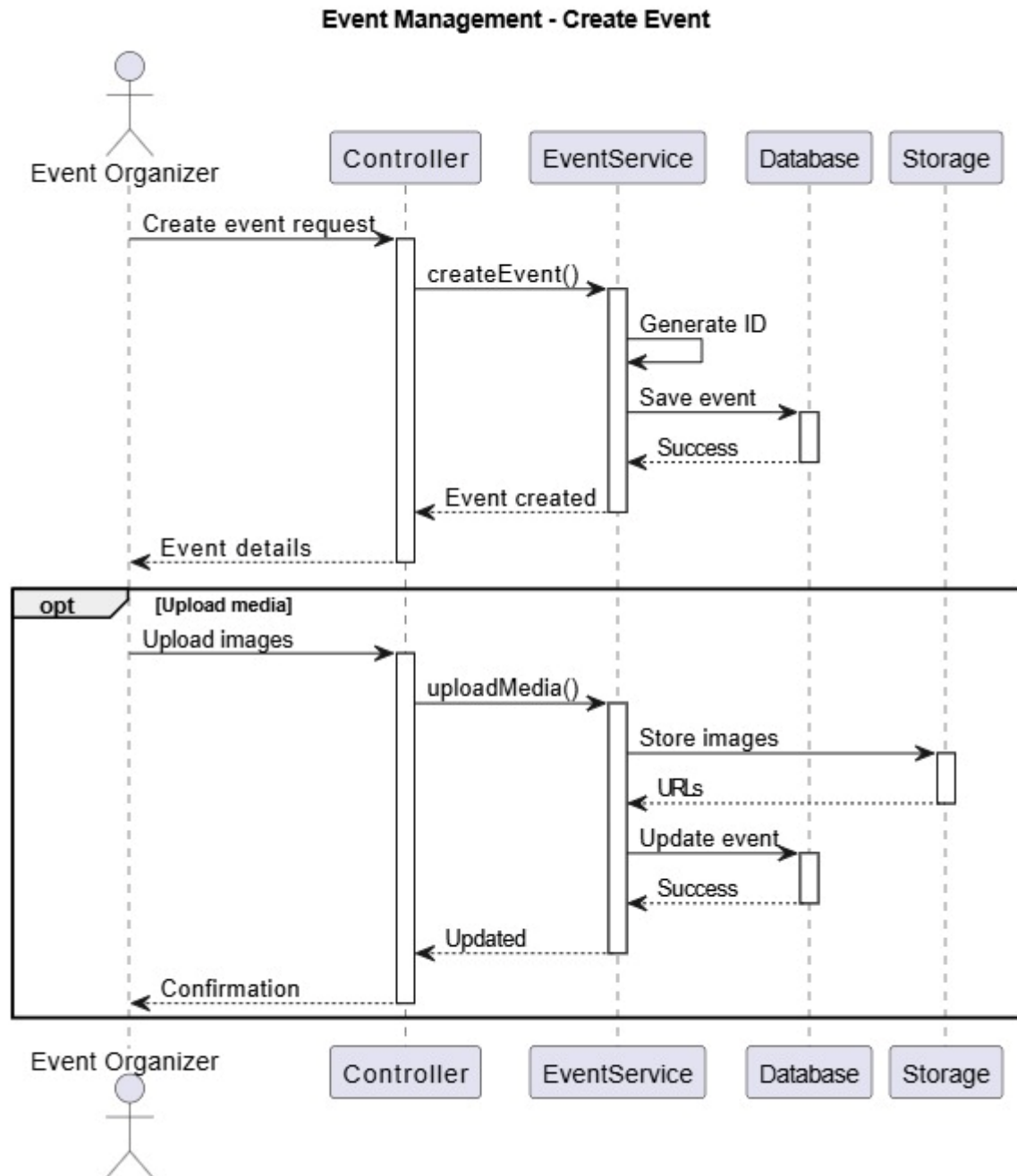


Figure 19:Sequence For Create Event

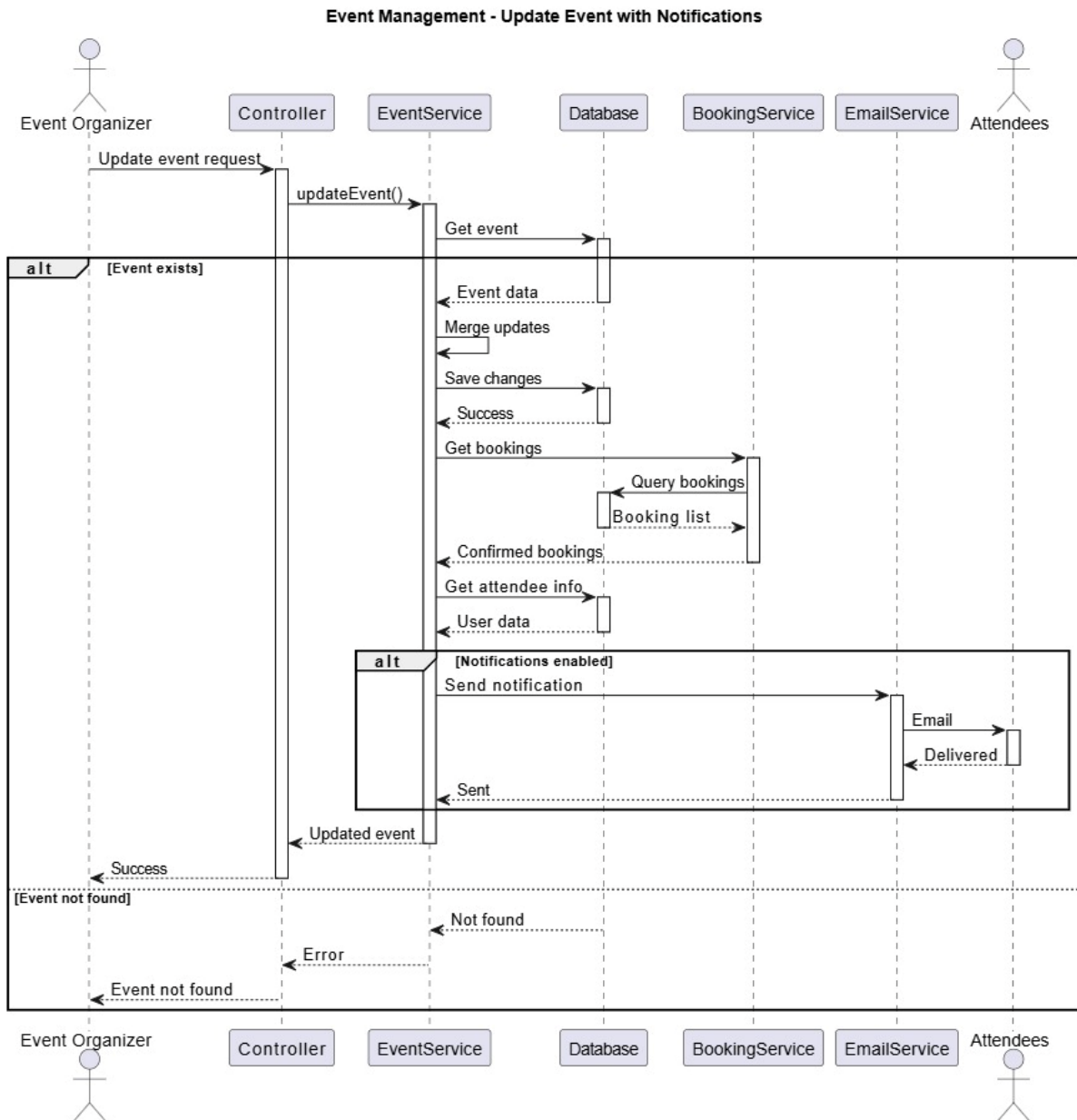


Figure20 :Sequence For Update Event With Notifications

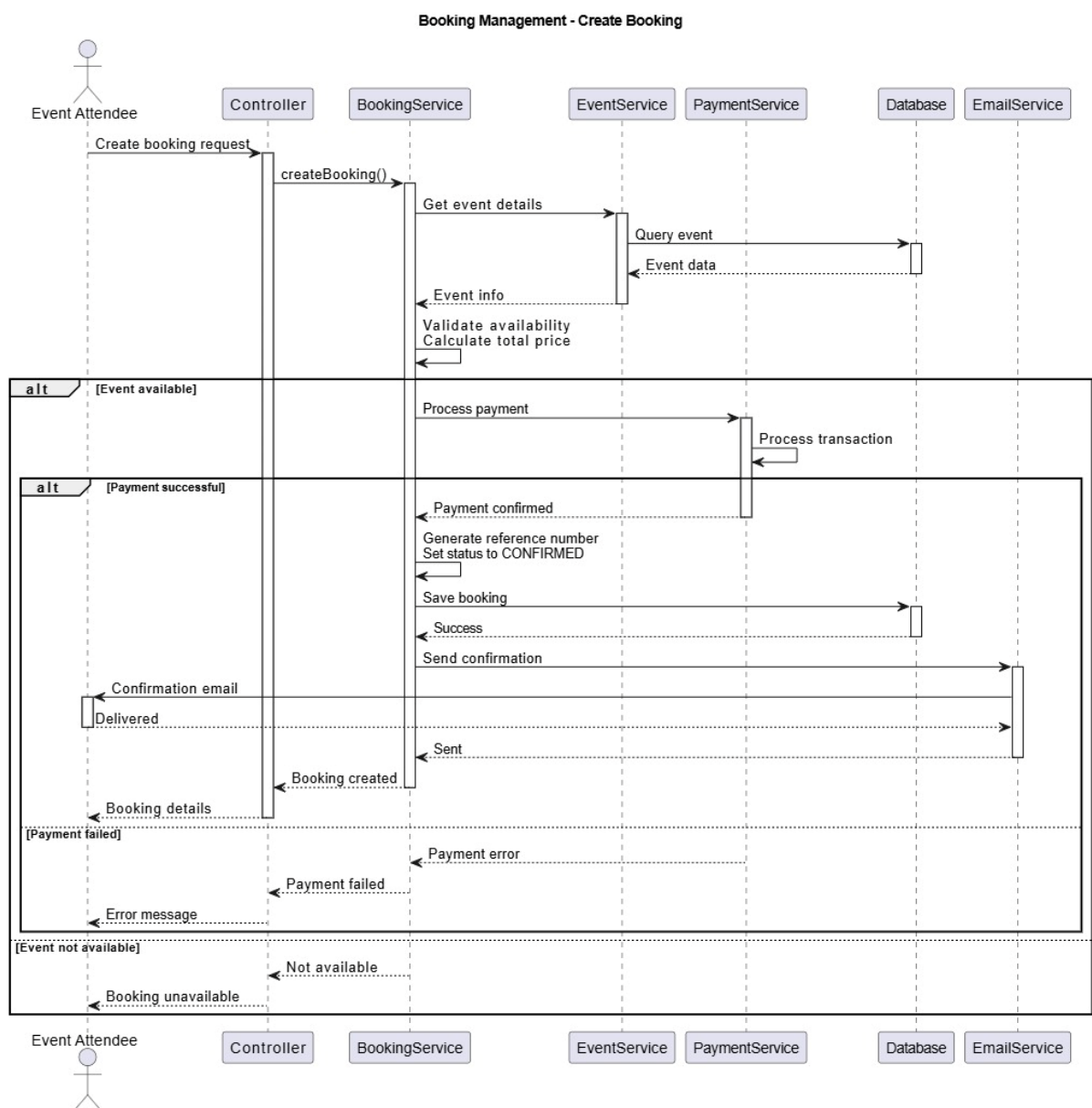


Figure21 :Sequence For Create Booking

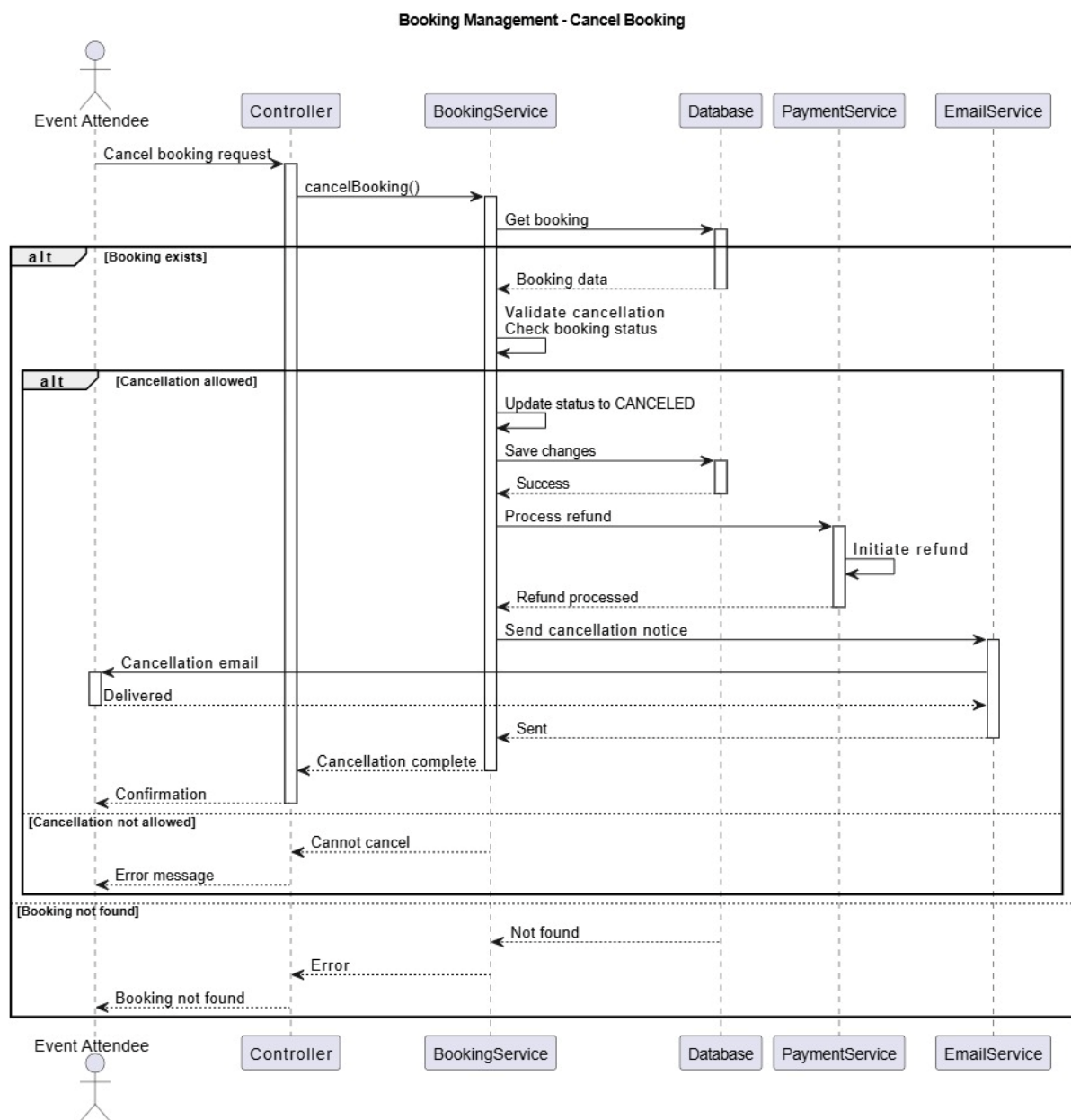


Figure22 :Sequence For Cancel Booking

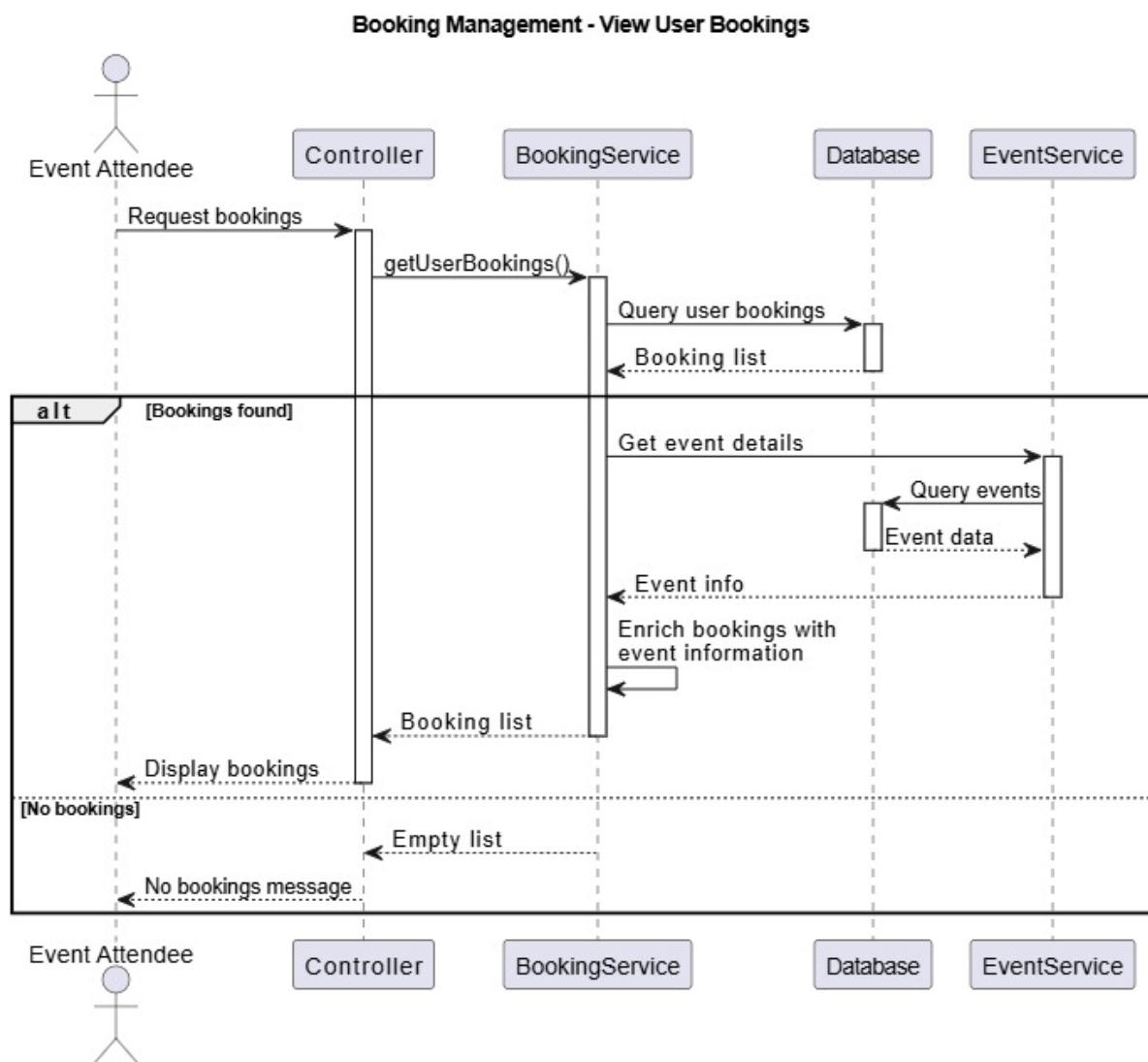


Figure23 :Sequence For View User Bookings

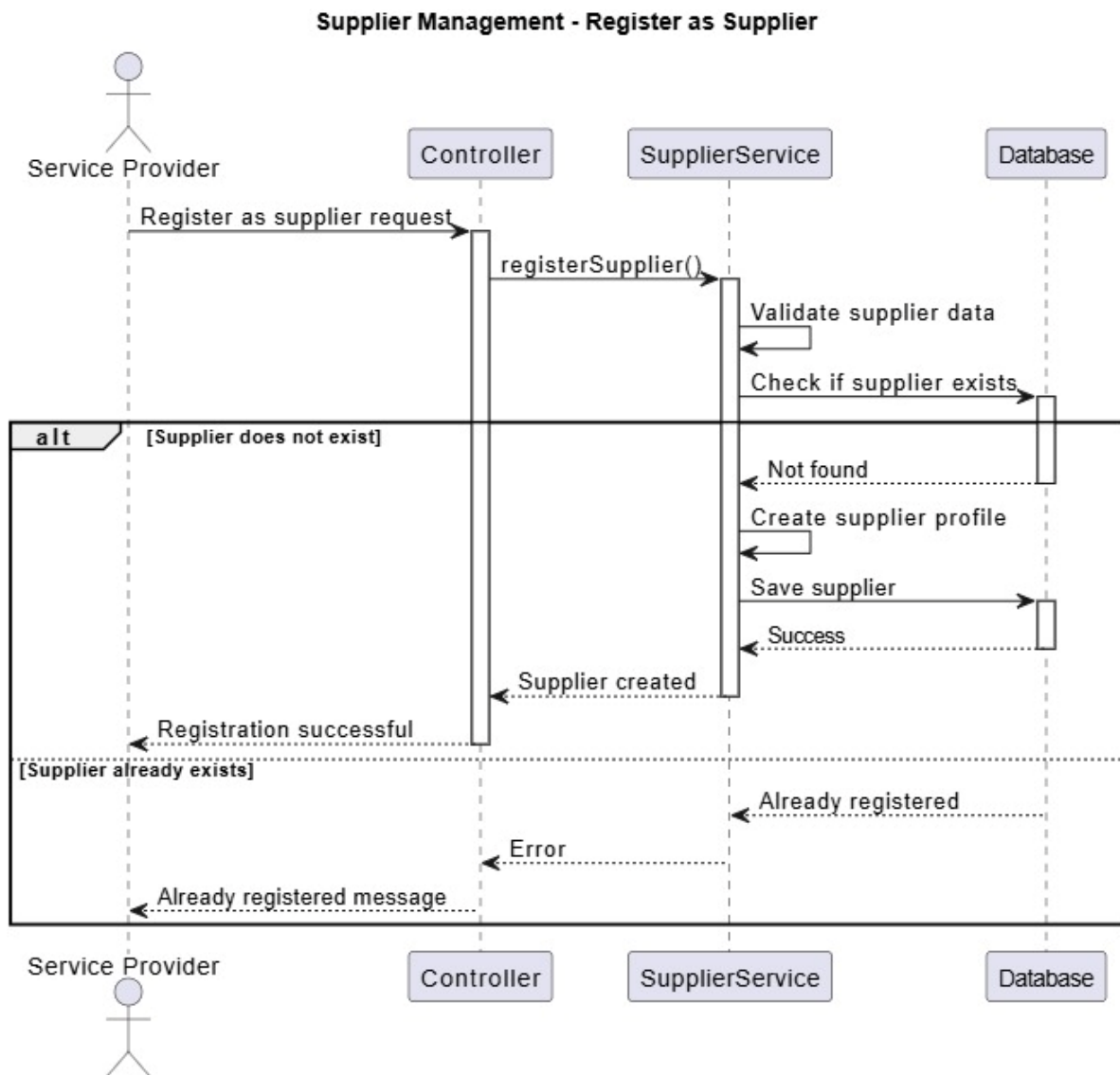


Figure 24:Sequence For Register as Supplier



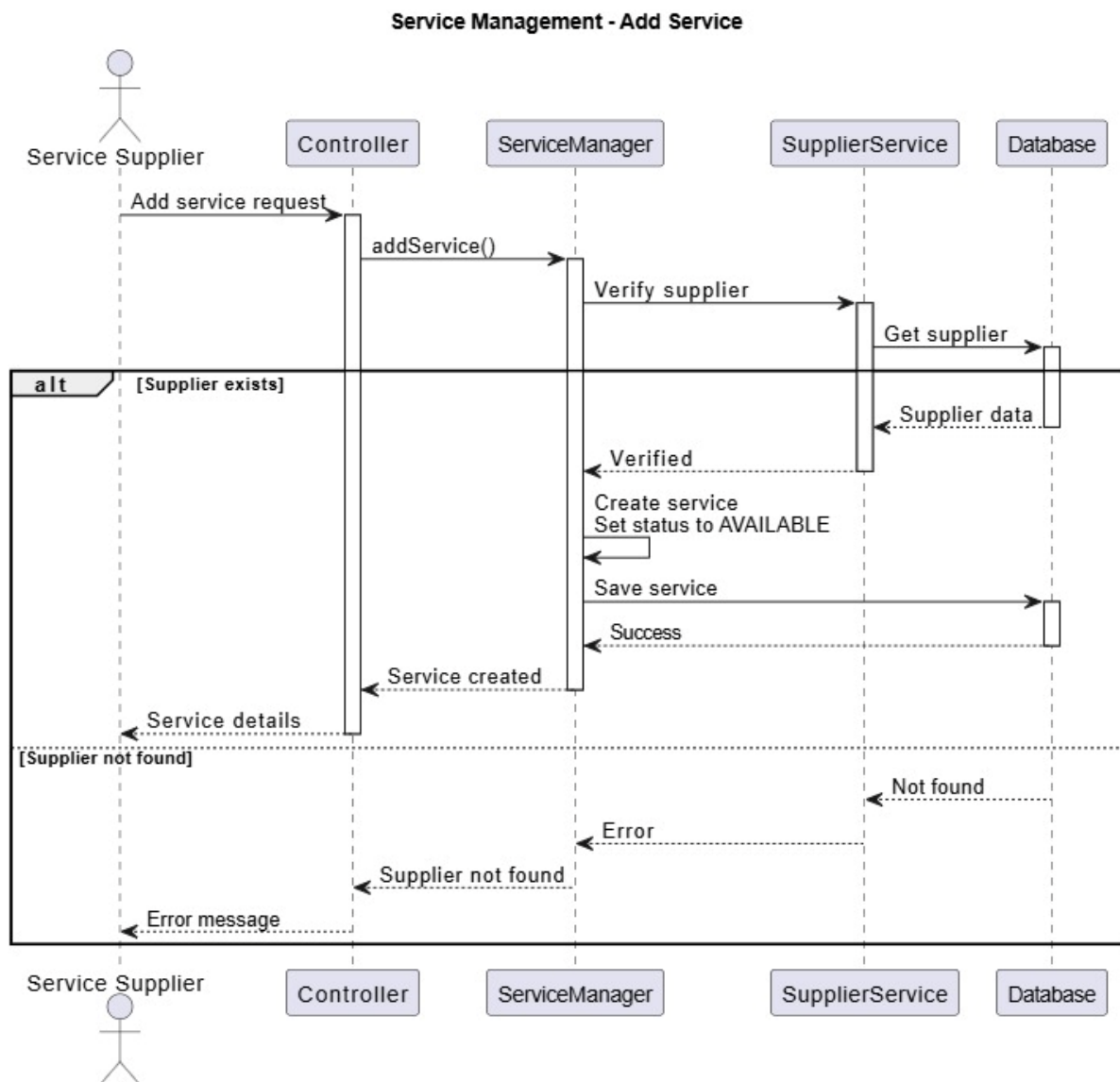


Figure25 :Sequence For Add Service

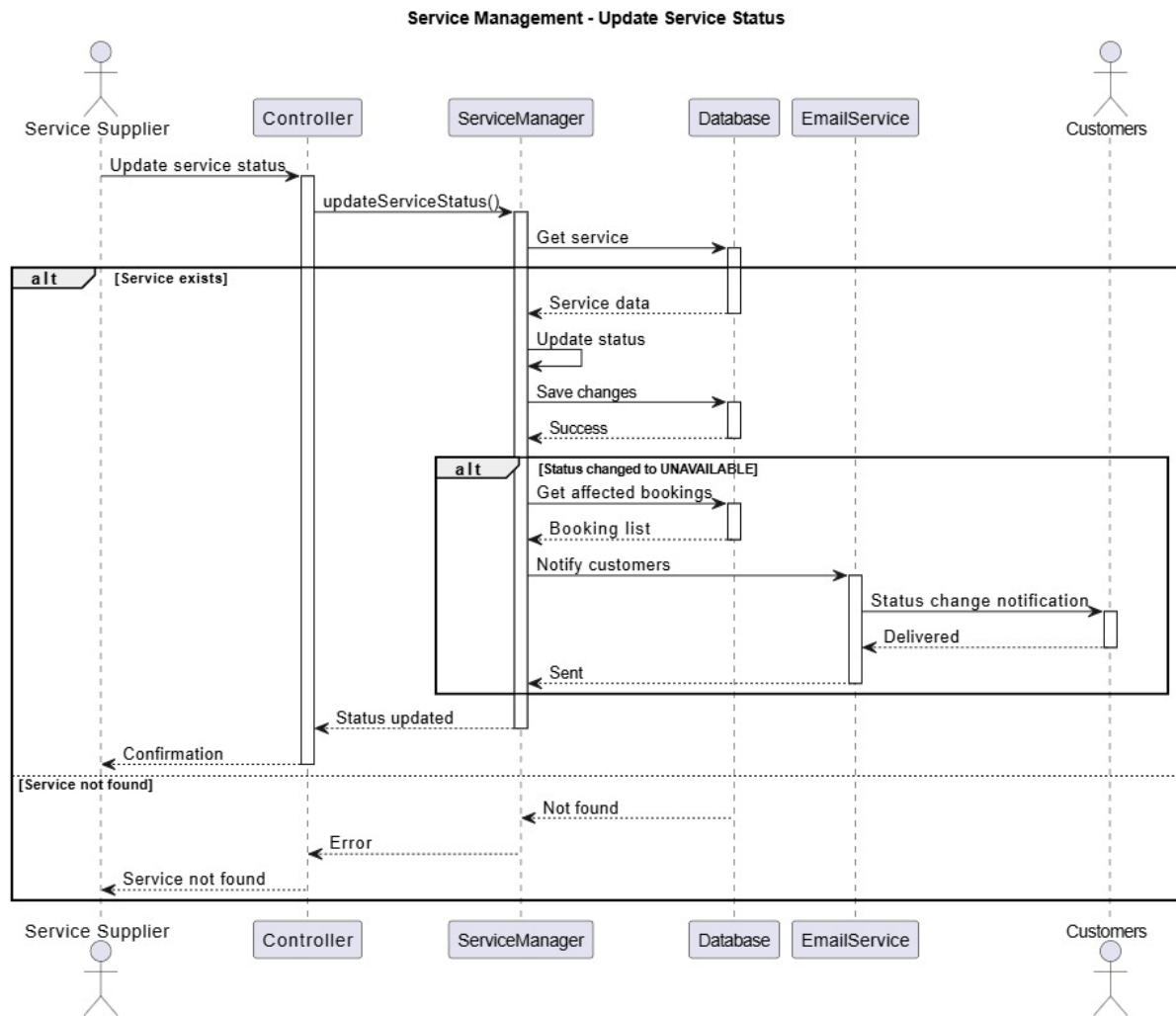


Figure26 :Sequence For Update Service Status

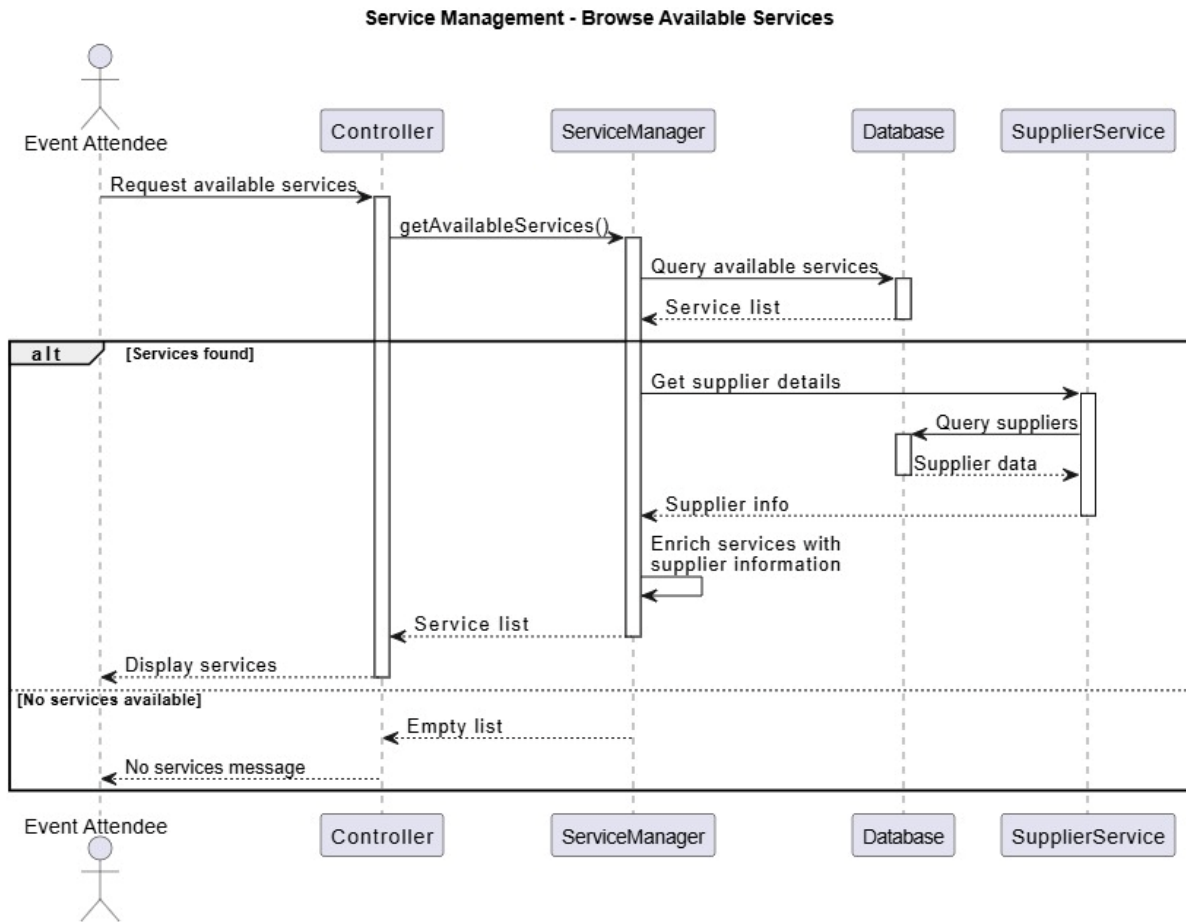


Figure27 :Sequence For Browse Available Service

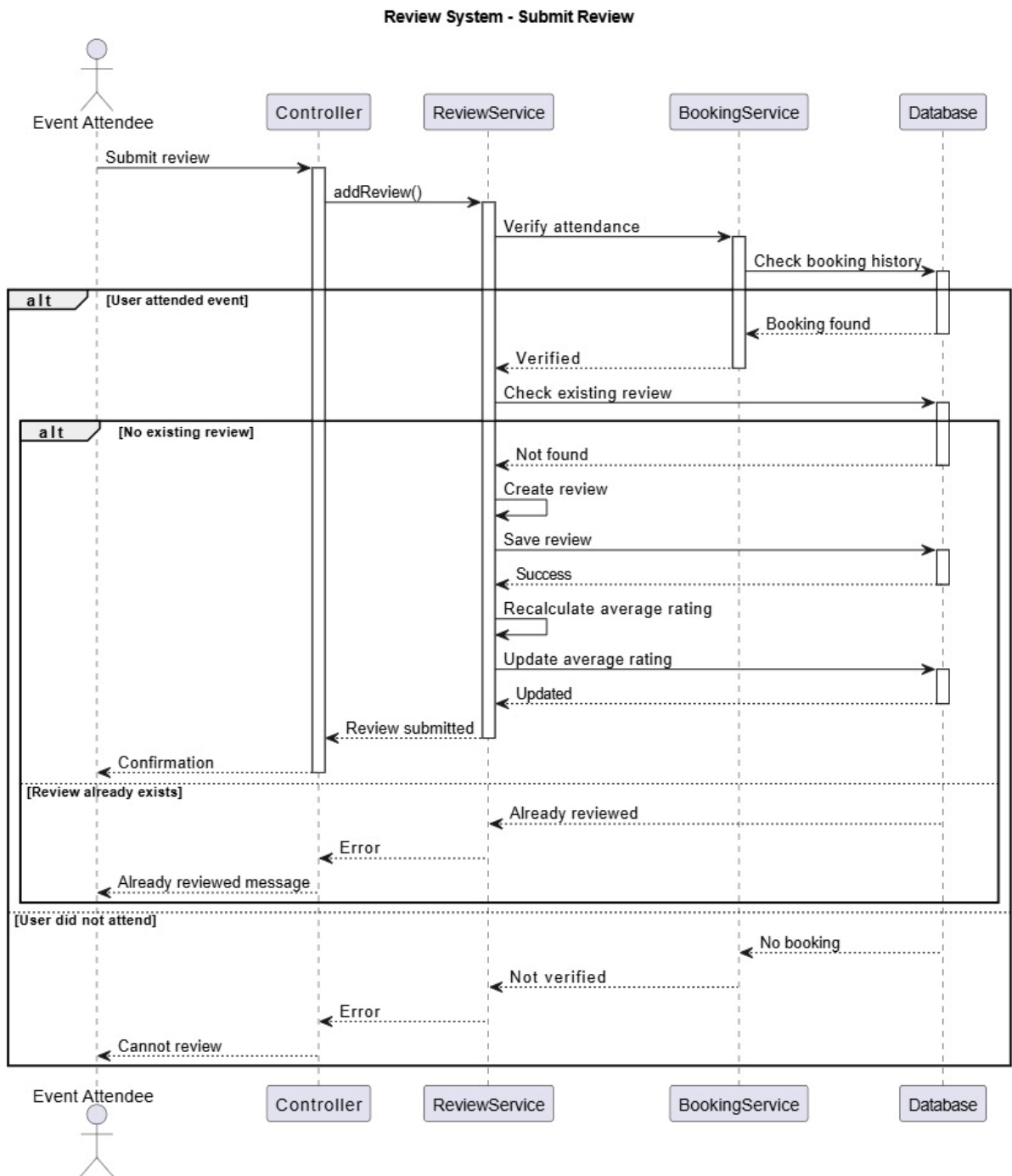


Figure28 :Sequence For Submit Review

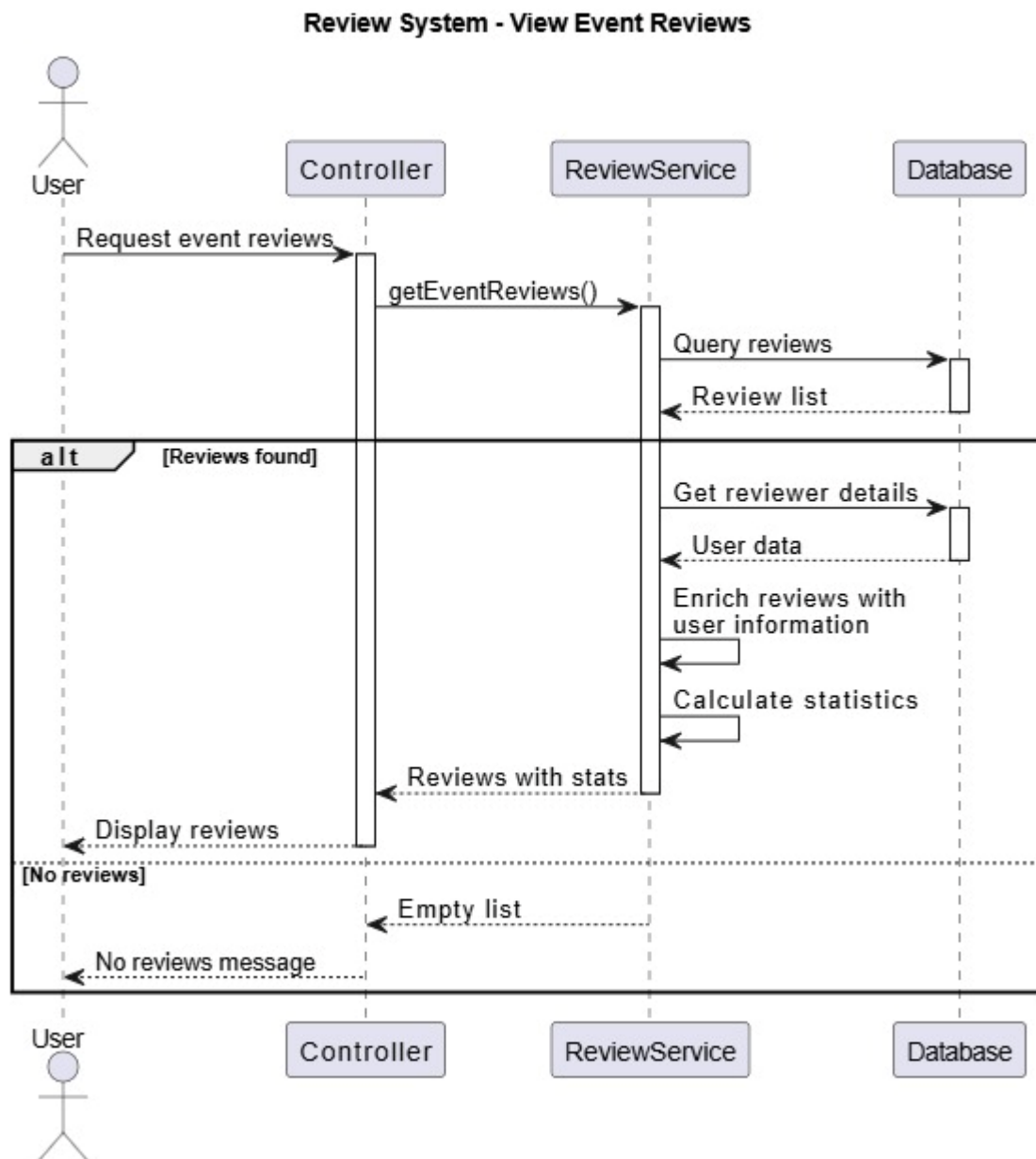


Figure29 :Sequence For View Event Reviews

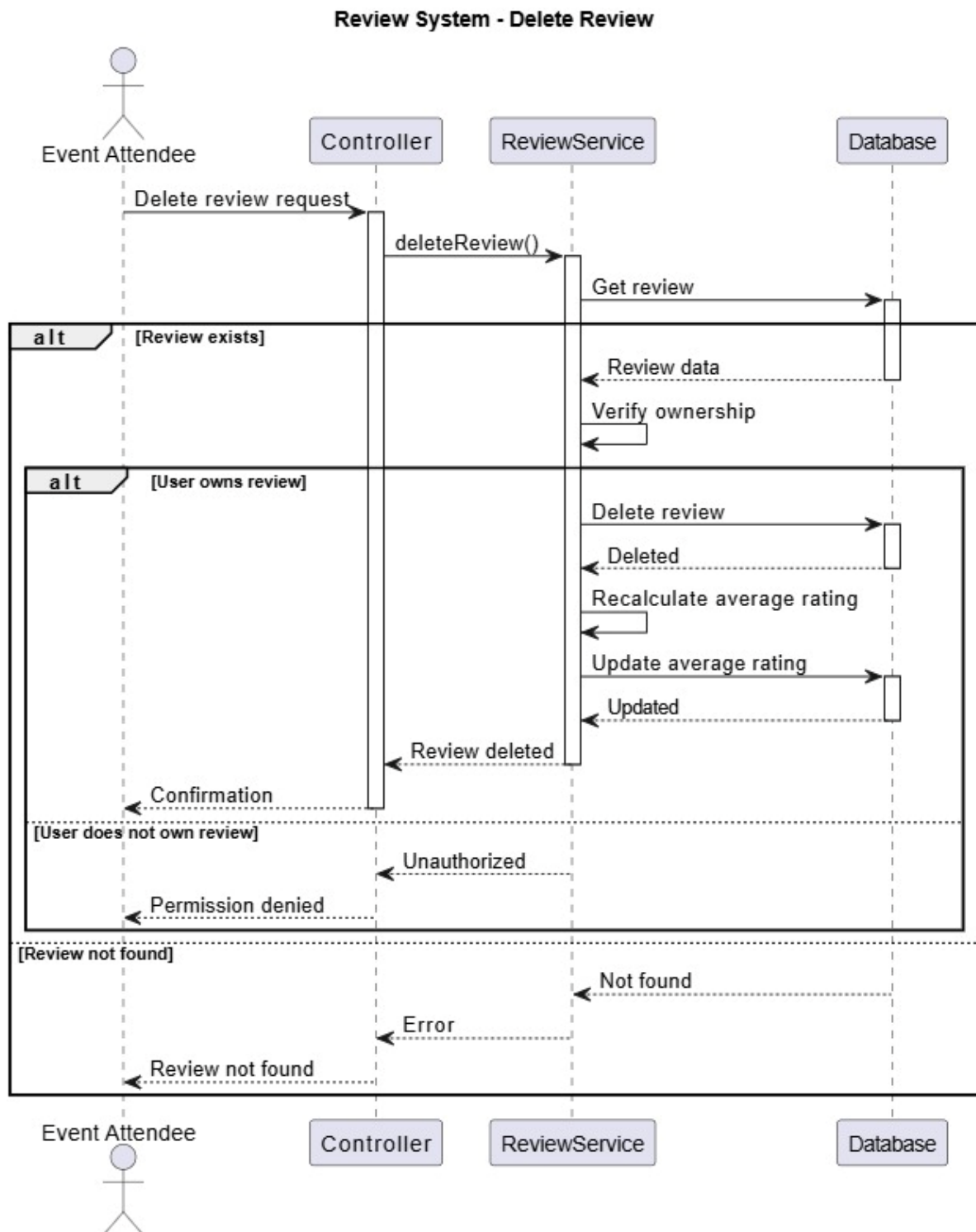


Figure30 :Sequence For Delete Review

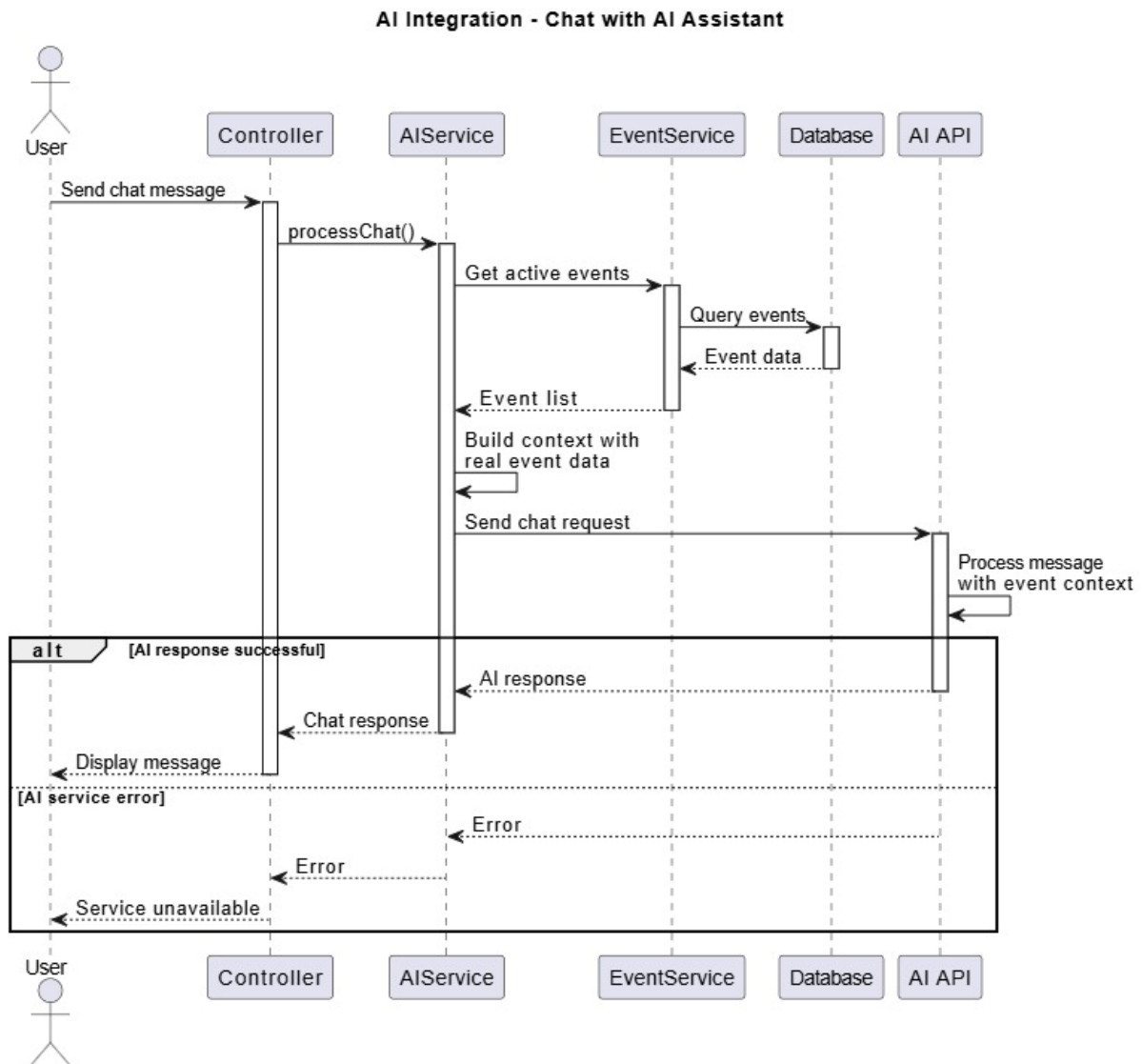


Figure31 :Sequence For Chat With AI Assistant

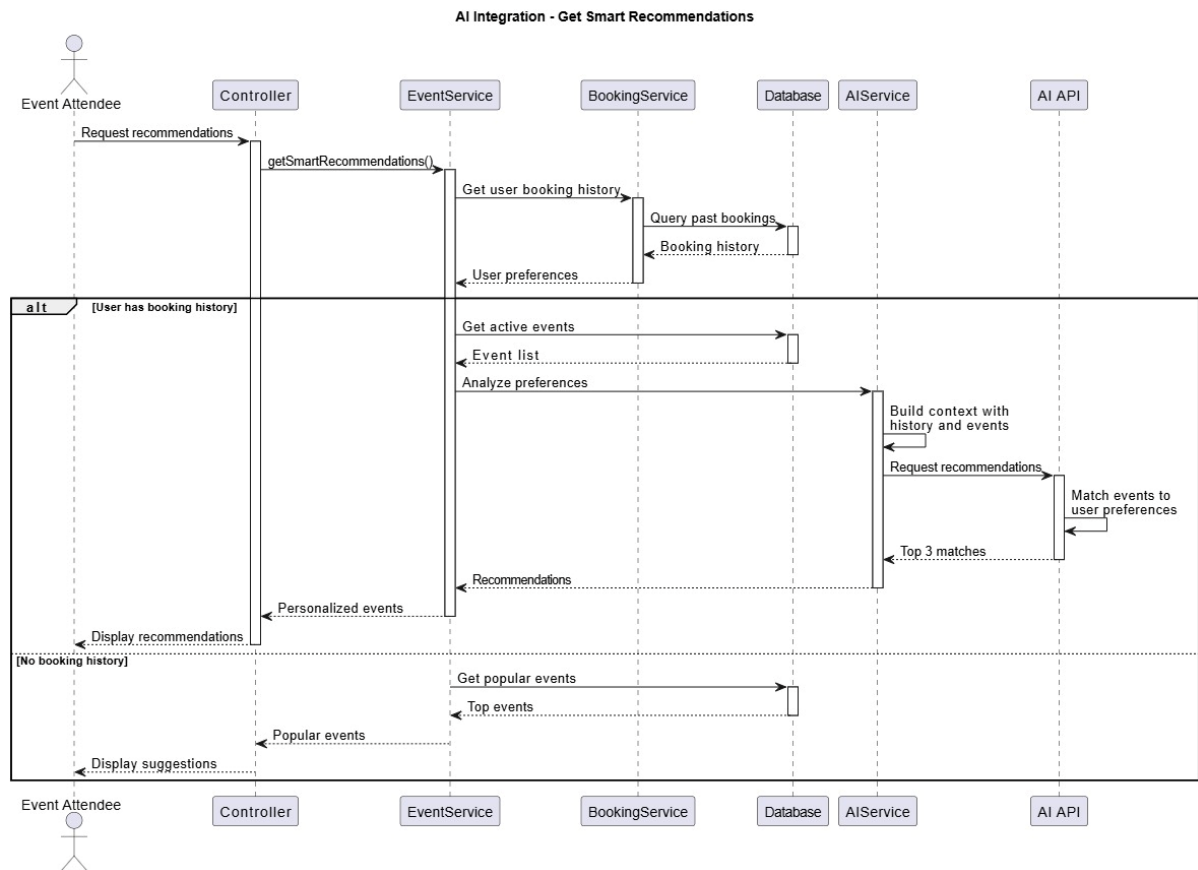
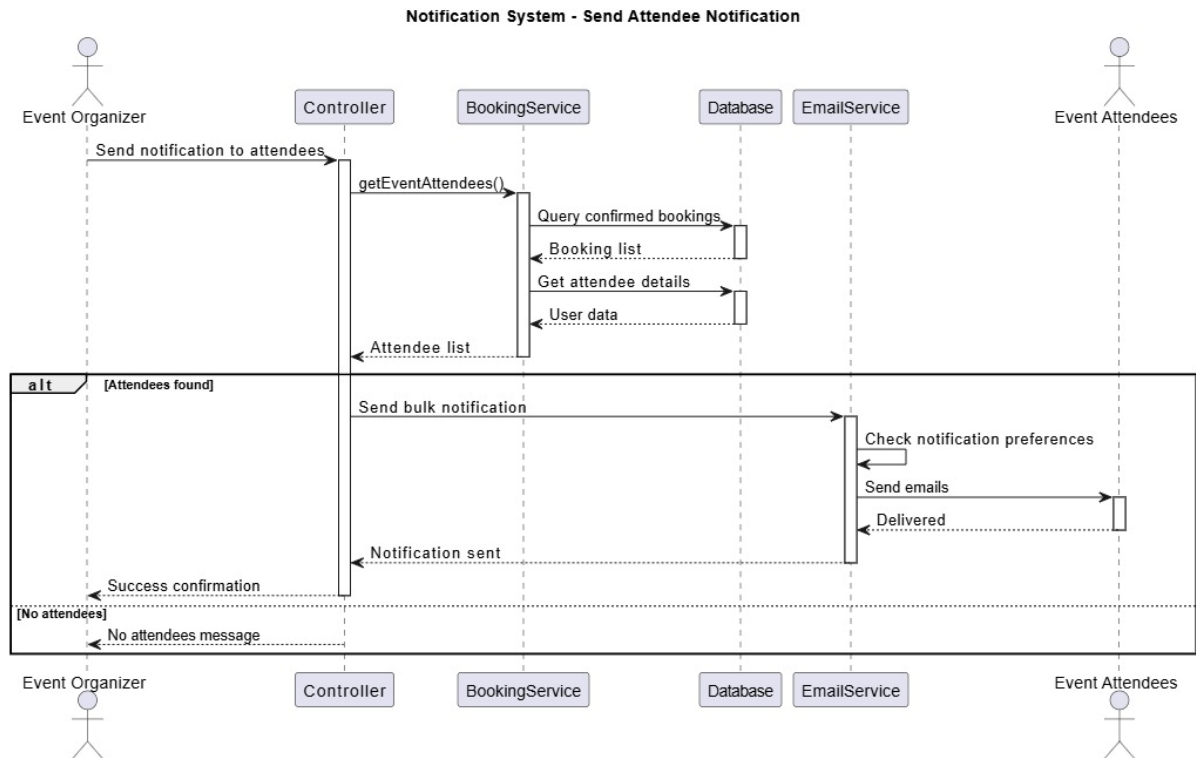


Figure32 :Sequence For Get Smart Recommendations





*Figure33 :Sequence For Send Attendee Notification*

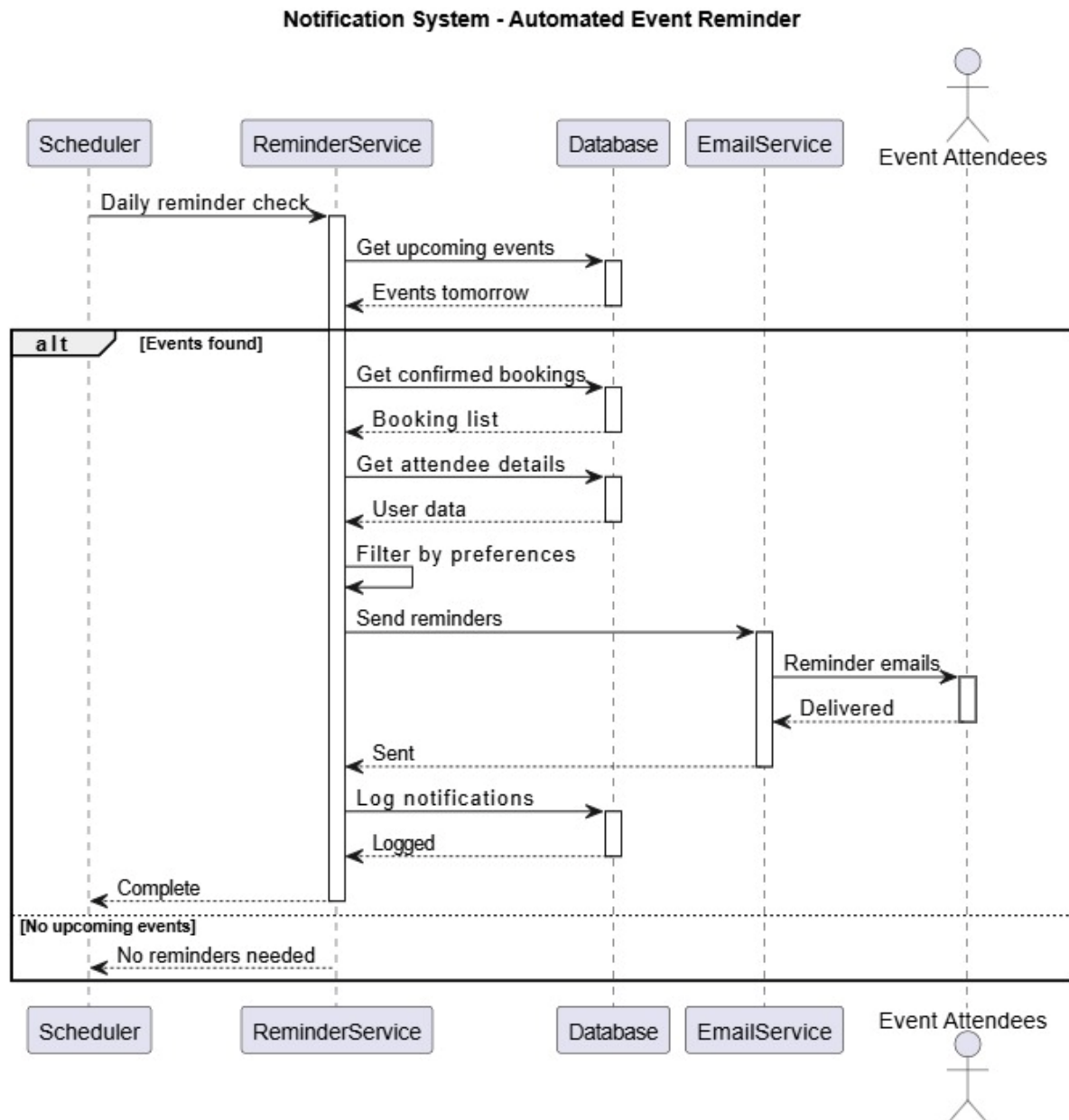


Figure34 :Sequence For Automated Event Reminder

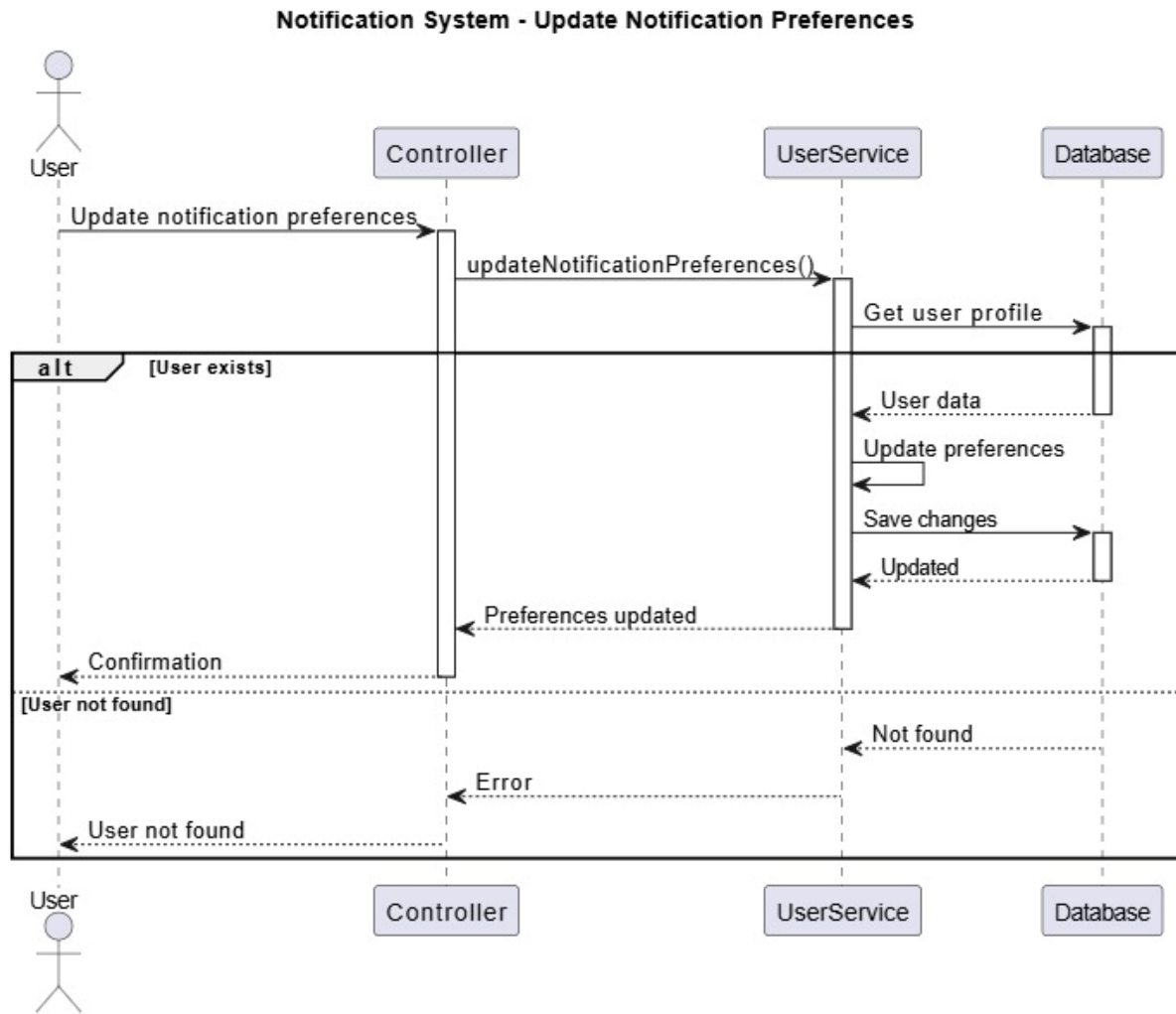


Figure.35 :Sequence For Update Notification Preferences

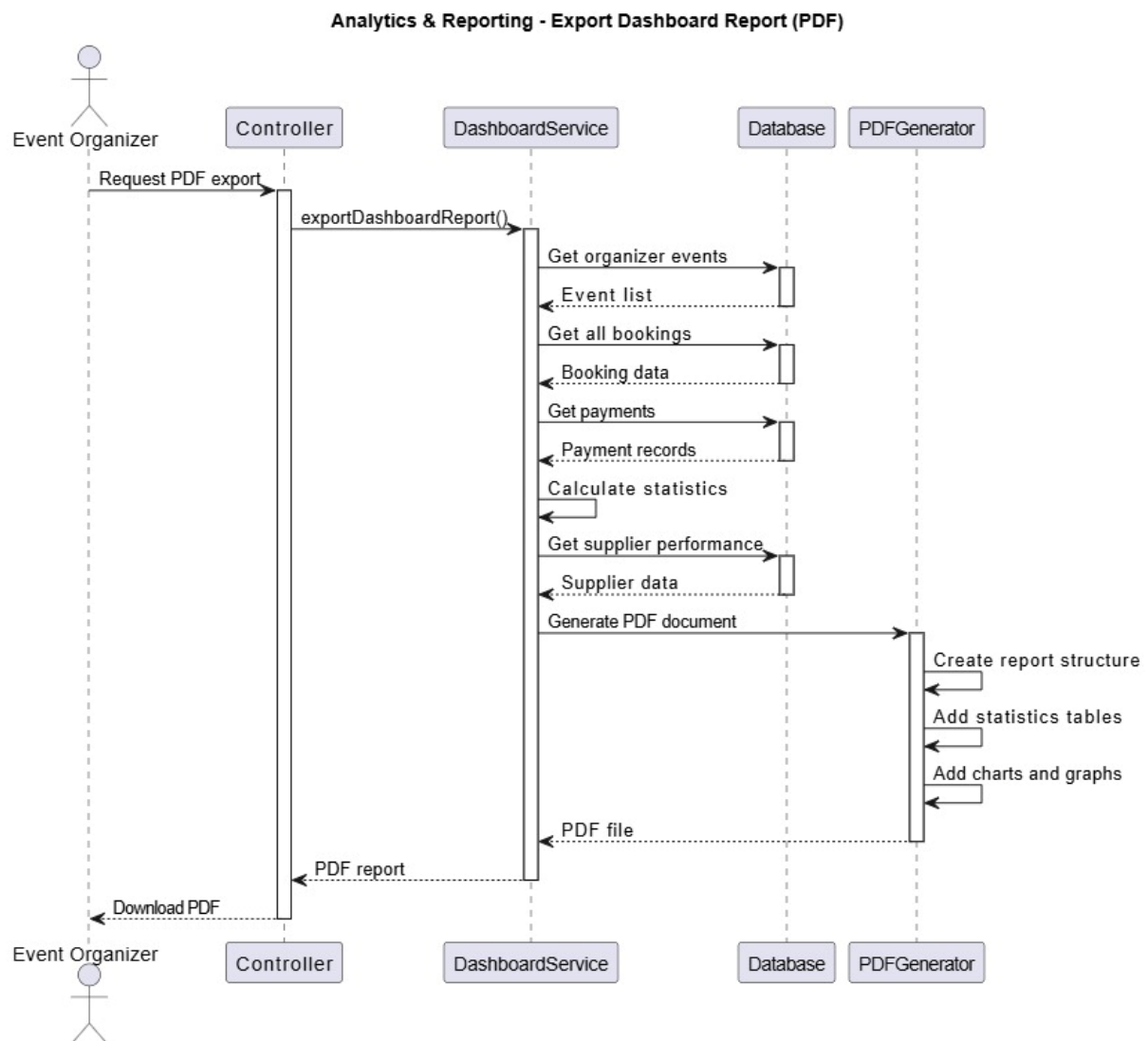


Figure36 :Sequence For Export Dashboard Report

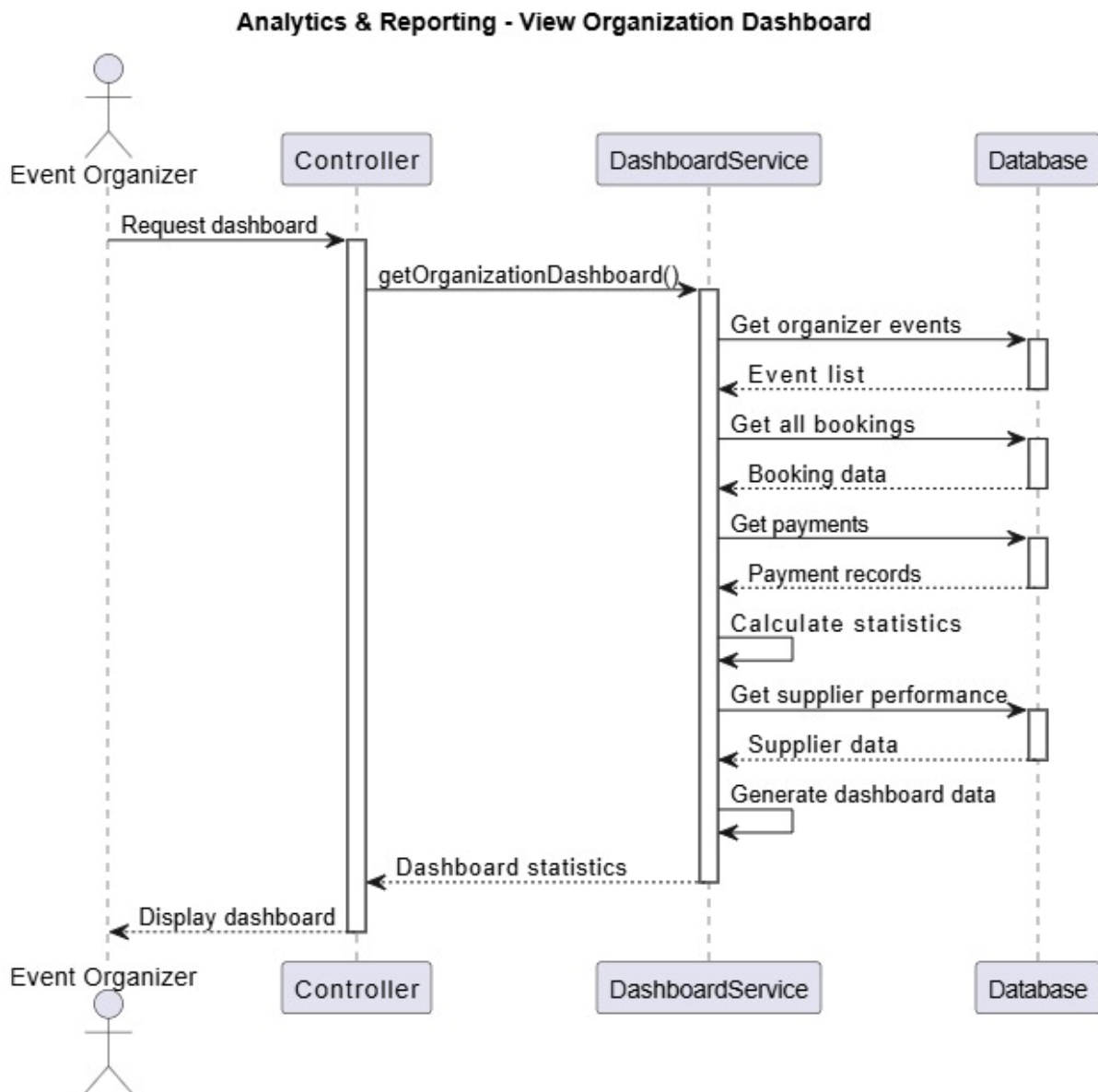


Figure37:Sequence For View Organization Dashboard

## 4-8 Agile Modeling and Methodology Implementation

To support the development process of the EventMarketPlace platform, agile methodologies and modeling techniques were adopted throughout the project lifecycle. These methodologies helped organize requirements, prioritize system features, and manage development tasks incrementally while maintaining alignment with user needs and business objectives. The agile approach also supported iterative delivery, allowing the platform to evolve progressively through multiple development phases.

### 4-8-1 Story Mapping

#### Introduction

To better organize the requirements of the EventMarketPlace platform, the Story Mapping technique was adopted during the planning phase. This approach helped transform user requirements into a structured development roadmap by focusing on user activities, priorities, and complete end-to-end user journeys. Instead of treating features as isolated tasks, Story Mapping enabled the system to be designed from the users' perspective, ensuring that each implemented feature contributes meaningful value to the overall user experience.

#### User Personas

Three primary user groups were identified within the platform. Understanding these user roles was essential for defining the core workflows of the system and ensuring that all functional requirements were aligned with actual user needs.

| User Persona |                  | Description                                                                                                                                                      |
|--------------|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1            | Event Attendee   | Users who browse, discover, and book events available on the marketplace platform.                                                                               |
| 2            | Event Organizer  | Users who create, publish, and manage events, including controlling booking availability and communicating with attendees.                                       |
| 3            | Service Provider | Users who register as suppliers and offer additional services — such as catering, decoration, photography, or technical support — that can be added to bookings. |

*Table (43): User Personas*

## Backbone Activities

Based on the identified user personas, six core activities were defined as the backbone of the story map. Each activity represents a major user goal and was further divided into smaller user stories representing the required system functionalities.

| Backbone Activity |                   | Description                                                                                                           |
|-------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------|
| 1                 | Discover Events   | Users browse the marketplace to find and explore events of interest based on type, location, and availability.        |
| 2                 | Book Events       | Users select desired events, choose seats, optionally add supplementary services, and complete the booking process.   |
| 3                 | Manage Events     | Event organizers create, update, and control events, including managing booking availability and notifying attendees. |
| 4                 | Provide Services  | Service providers register on the platform and offer additional services that attendees can attach to their bookings. |
| 5                 | Review Experience | Attendees submit ratings and written reviews for events and service providers following their experience.             |
| 6                 | Analyze Data      | Event organizers access an analytics dashboard to monitor performance metrics and export summary reports.             |

*Table (44): Backbone Activities*

## MVP Release Planning

For the first release (Minimum Viable Product - MVP), priority was given to the core functionalities that provide immediate value to users. The MVP focused on delivering complete user journeys rather than isolated features in order to ensure a coherent and usable platform from the initial release.

### MVP Core Features

| User Activity            | MVP Core Features                                                                                                                                                                                                                                 |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Discover Events</b>   | <ol style="list-style-type: none"><li>1. Browse all available events</li><li>2. Search and filter events by type and location</li><li>3. View complete event details including media and ratings</li></ol>                                        |
| <b>Book Events</b>       | <ol style="list-style-type: none"><li>1. Select number of seats and validate availability</li><li>2. Optionally add supplementary services to a booking</li><li>3. Confirm booking and receive email confirmation with reference number</li></ol> |
| <b>Manage Events</b>     | <ol style="list-style-type: none"><li>1. Create new events with complete details</li><li>2. Update event information and notify attendees of changes</li><li>3. Toggle booking availability and control event status</li></ol>                    |
| <b>Provide Services</b>  | <ol style="list-style-type: none"><li>1. Register as a service provider on the platform</li><li>2. Create and manage service listings</li></ol>                                                                                                   |
| <b>Review Experience</b> | <ol style="list-style-type: none"><li>1. Submit ratings and written reviews for attended events</li><li>2. View average ratings and review summaries on event pages</li></ol>                                                                     |
| <b>Analyze Data</b>      | <ol style="list-style-type: none"><li>1. Access an organizational dashboard with key performance metrics</li><li>2. View booking and revenue statistics</li></ol>                                                                                 |

*Table (45): MVP Release Planning*



## **Features Planned for Future Releases**

The following advanced features were identified during the planning process but postponed to future releases in order to keep the MVP focused and achievable within the project timeline:

- AI-powered event recommendations based on booking history
- AI chat assistant for event support and discovery
- Advanced analytics and trend analysis
- Dashboard report export functionality
- Supplier ranking and rating aggregation

## **Benefits of Story Mapping**

Applying the Story Mapping technique provided several benefits throughout the development process:

- Improved backlog prioritization by distinguishing essential MVP features from future enhancements
- Helped identify dependencies between system features during early development stages
- Supported incremental delivery through manageable development iterations
- Maintained focus on complete user journeys and user-centered design
- Ensured alignment between development activities and business objectives

## 4-8-2 Product Backlog

### Introduction

The Product Backlog represents the complete collection of functional requirements, features, and development tasks identified for the EventMarketPlace platform. Backlog items were organized and prioritized according to their business value, implementation complexity, and importance to the MVP release. This prioritization ensured that critical functionalities were implemented first while allowing future enhancements to be scheduled for later iterations.

### Product Backlog Summary

The following table summarizes the primary user stories included in the product backlog, organized according to epic and implementation priority.

| Story ID | User Story Title       | Primary Actor   | Priority      | Points | Epic               |
|----------|------------------------|-----------------|---------------|--------|--------------------|
| US-1.1   | Create Event           | Event Organizer | P0 – Critical | 8      | Event Management   |
| US-1.2   | Browse All Events      | Event Attendee  | P0 – Critical | 5      | Event Management   |
| US-1.3   | View Event Details     | Event Attendee  | P0 – Critical | 5      | Event Management   |
| US-1.4   | Update Event Details   | Event Organizer | P1 – High     | 8      | Event Management   |
| US-1.5   | Delete Event           | Event Organizer | P1 – High     | 3      | Event Management   |
| US-1.6   | Upload Event Media     | Event Organizer | P2 – Medium   | 5      | Event Management   |
| US-1.7   | Toggle Booking Status  | Event Organizer | P1 – High     | 3      | Event Management   |
| US-2.1   | Create Booking         | Event Attendee  | P0 – Critical | 13     | Booking Management |
| US-2.2   | View Booking Details   | Event Attendee  | P0 – Critical | 3      | Booking Management |
| US-2.3   | View Upcoming Bookings | Event Attendee  | P1 – High     | 5      | Booking Management |
| US-2.4   | View Past Bookings     | Event Attendee  | P1 – High     | 5      | Booking Management |

| Story ID | User Story Title                | Primary Actor   | Priority      | Points | Epic               |
|----------|---------------------------------|-----------------|---------------|--------|--------------------|
| US-2.5   | Cancel Booking                  | Event Attendee  | P1 – High     | 8      | Booking Management |
| US-2.6   | Get Booked Dates                | Event Attendee  | P2 – Medium   | 3      | Booking Management |
| US-2.7   | Notify Event Attendees          | Event Organizer | P2 – Medium   | 5      | Booking Management |
| US-3.1   | User Registration               | New User        | P0 – Critical | 8      | User Management    |
| US-3.2   | Token Verification              | Registered User | P0 – Critical | 5      | User Management    |
| US-3.3   | Update User Profile             | Registered User | P1 – High     | 8      | User Management    |
| US-3.4   | Delete Profile Picture          | Registered User | P2 – Medium   | 3      | User Management    |
| US-3.5   | Update Notification Preferences | Registered User | P1 – High     | 5      | User Management    |
| US-3.6   | Delete Account                  | Registered User | P2 – Medium   | 5      | User Management    |

Table (46): Product Backlog Summary

## Backlog Statistics

The backlog contains all identified user stories required for the implementation of the platform. Story points and priority levels were used to estimate development effort and organize implementation phases according to business importance and MVP objectives.

| Metric                       | Value                            |
|------------------------------|----------------------------------|
| Total User Stories           | 20                               |
| Total Story Points           | 130                              |
| Critical (P0) Stories        | 7                                |
| High Priority (P1) Stories   | 9                                |
| Medium Priority (P2) Stories | 4                                |
| Completion Status            | 100% — All stories code-verified |

Table (47): Backlog Statistics

## 4-8-3 User Stories

### Introduction

User Stories were used to describe the functional requirements of the system from the perspective of end users. Each story follows the agile format “As a... I want... So that...” to clearly define the user role, desired functionality, and expected benefit. In addition, each user story includes acceptance criteria, test scenarios, and implementation references to ensure that all functionalities are clearly defined, testable, and traceable to the implemented system components.

### 3. Detailed User Stories

The following section presents each user story in full detail. Stories are grouped by epic and presented in priority order. Each card includes the user story statement, acceptance criteria in Given-When-Then format, test scenarios, and code references.

#### Epic 1: Event Management System

| US-1.1 Create Event   Priority: P0 – Critical   Story Points: 8 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User Story                                                      | <p>As a Event Organizer,</p> <p>I want to create new events with complete details,</p> <p>so that attendees can discover and book my events.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Acceptance Criteria                                             | <ol style="list-style-type: none"><li>1. Given I am logged in as an organizer, when I navigate to the create event page, then I see a form with all required fields.</li><li>2. Given I fill in event title, description, location, city, and date/time, when I set ticket price and capacity, then the system validates all inputs.</li><li>3. Given I select event type (PUBLIC_EVENT or HOST_PACKAGE), when I submit the form, then the event is created with ACTIVE status.</li><li>4. Given the event is created successfully, when I view the event, then available seats equals total capacity.</li><li>5. Given the event is created, when I check the database, then the event has a unique ID and creation timestamps.</li></ol> |
| Test Scenarios                                                  | <ul style="list-style-type: none"><li>✓ Create PUBLIC_EVENT with valid data.</li><li>✓ Create HOST_PACKAGE with valid data.</li><li>✓ Validate that required fields are enforced.</li><li>✓ Validate that ticket price must be positive.</li><li>✓ Validate that capacity must be positive.</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                      |

|                       |                                                                                                    |
|-----------------------|----------------------------------------------------------------------------------------------------|
|                       | ✓ Verify that timestamps are correctly set upon creation.                                          |
| <b>Code Reference</b> | <p>EventService.createEvent() — lines 47–57</p> <p>EventController.createEvent() — lines 23–31</p> |

Table (48): Detailed User Stories 1

| US-1.2 Browse All Events   Priority: P0 – Critical   Story Points: 5 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>                                                    | <p>As a Event Attendee,</p> <p>I want to view all available events,</p> <p>so that I can discover events to attend.</p>                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Acceptance Criteria</b>                                           | <ol style="list-style-type: none"> <li>1. Given I am on the marketplace homepage, when I view the events list, then I see all events with ACTIVE status.</li> <li>2. Given events are displayed, when I view each event card, then I see title, location, price, and available seats.</li> <li>3. Given an event has bookingEnabled set to false, when I browse events, then that event is excluded from the list.</li> <li>4. Given there are no active events, when I browse events, then I see a 'No events available' message.</li> </ol> |
| <b>Test Scenarios</b>                                                | <ul style="list-style-type: none"> <li>✓ Display all ACTIVE events.</li> <li>✓ Exclude events with CANCELED status.</li> <li>✓ Exclude events with bookingEnabled=false.</li> <li>✓ Handle an empty event list gracefully.</li> <li>✓ Verify completeness of event data in each card.</li> </ul>                                                                                                                                                                                                                                              |
| <b>Code Reference</b>                                                | <p>EventController.getAllEvents() — lines 42–48</p> <p>EventService.getAllEvents()</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

Table (49): Detailed User Stories 2

### US-1.3 View Event Details | Priority: P0 – Critical | Story Points: 5

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Event Attendee,</p> <p>I want to view complete details of a specific event,</p> <p>so that I can make an informed decision about booking it.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given I am browsing events, when I select an event, then I am navigated to the event details page.</li> <li>2. Given I am on the event details page, when I view the information, then I see title, description, location, city, and date/time.</li> <li>3. Given I am viewing event details, when I scroll further, then I see ticket price, total capacity, and available seats.</li> <li>4. Given the event has uploaded media, when I view event details, then I see event images and the promotional video.</li> <li>5. Given the event has reviews, when I view event details, then I see the average rating and total review count.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Display complete event information.</li> <li>✓ Show event images when available.</li> <li>✓ Show promotional video when available.</li> <li>✓ Display the computed average rating.</li> <li>✓ Return HTTP 404 when event is not found.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Code Reference</b>      | <p>EventController.getEvent() — lines 33–40</p> <p>EventService.getEventById()</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

Table (50): Detailed User Stories 3

## US-1.4 Update Event Details | Priority: P1 – High | Story Points: 8

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Event Organizer,</p> <p>I want to modify event information,</p> <p>so that I can keep event details accurate and notify attendees of any changes.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given I am the event organizer, when I update the event title, description, or location, then changes are persisted immediately.</li> <li>2. Given I update the event date or ticket price, when I save changes, then the system records exactly which fields were modified.</li> <li>3. Given the event has confirmed bookings, when I update event details, then all attendees receive an automated email notification.</li> <li>4. Given a notification email is dispatched, when an attendee opens it, then the email lists the specific changes that were made.</li> <li>5. Given a user has disabled event update notifications, when I update the event, then that user does not receive an email.</li> <li>6. Given I update the event status to CANCELED, when I save, then all attendees receive a cancellation email.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Update event title successfully.</li> <li>✓ Update event date and time.</li> <li>✓ Update ticket price.</li> <li>✓ Update event capacity.</li> <li>✓ Change event status to CANCELED.</li> <li>✓ Verify attendees are notified of changes.</li> <li>✓ Verify notification preferences are respected.</li> <li>✓ Verify all changed fields are tracked and reported.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Code Reference</b>      | <p>EventService.updateEvent() — lines 59–165</p> <p>EmailService.sendEventNotification()</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

Table (51): Detailed User Stories 4



| US-1.5 Delete Event   Priority: P1 – High   Story Points: 3 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User Story                                                  | <p>As a Event Organizer,</p> <p>I want to remove events from the system,</p> <p>so that outdated or canceled events are no longer visible to attendees.</p>                                                                                                                                                                                                                                                                                                                                                                                           |
| Acceptance Criteria                                         | <ol style="list-style-type: none"> <li>1. Given I am the event organizer, when I delete an event, then the event record is removed from the database.</li> <li>2. Given the event has confirmed bookings, when I delete the event, then all associated bookings are automatically canceled.</li> <li>3. Given the event is deleted, when an attendee tries to view it, then they receive an 'Event not found' message.</li> <li>4. Given the event is deleted, when I browse events, then the deleted event no longer appears in the list.</li> </ol> |
| Test Scenarios                                              | <ul style="list-style-type: none"> <li>✓ Delete an event successfully.</li> <li>✓ Verify the event record is removed from the database.</li> <li>✓ Return HTTP 404 when accessing a deleted event.</li> <li>✓ Verify all associated bookings are canceled upon deletion.</li> </ul>                                                                                                                                                                                                                                                                   |
| Code Reference                                              | <p>EventService.deleteEvent() — lines 166–169</p> <p>EventController.deleteEvent() — lines 115–122</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                |

Table (52): Detailed User Stories 5

| US-1.6 Upload Event Media   Priority: P2 – Medium   Story Points: 5 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User Story                                                          | <p>As a Event Organizer,</p> <p>I want to upload promotional images and videos for my event, so that the event page appears attractive to potential attendees.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Acceptance Criteria                                                 | <ol style="list-style-type: none"> <li>1. Given I am editing my event, when I upload an image, then the file is stored securely in Firebase Storage.</li> <li>2. Given an image is uploaded, when I view the event, then the image is displayed on the event page.</li> <li>3. Given I want to add multiple images, when I upload additional files, then all images are stored and displayed.</li> <li>4. Given I want to add a promotional video, when I upload a video file, then the video is stored and playable on the event page.</li> <li>5. Given an upload fails, when I attempt to upload media, then I receive an error message indicating the reason.</li> </ol> |
| Test Scenarios                                                      | <ul style="list-style-type: none"> <li>✓ Upload a single image successfully.</li> <li>✓ Upload multiple images successfully.</li> <li>✓ Upload a video file successfully.</li> <li>✓ Verify files are persisted in Firebase Storage.</li> <li>✓ Verify download URLs are generated and stored.</li> <li>✓ Handle upload failures with appropriate error messages.</li> </ul>                                                                                                                                                                                                                                                                                                 |
| Code Reference                                                      | <p>EventService.uploadEventMedia() — lines 195–213</p> <p>EventService.uploadEventVideo() — lines 181–193</p> <p>StorageService</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

Table (53): Detailed User Stories 6

## US-1.7 Toggle Booking Status | Priority: P1 – High | Story Points: 3

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Event Organizer,</p> <p>I want to enable or disable booking for my event,</p> <p>so that I can control the period during which attendees may book.</p>                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given my event has bookingEnabled set to true, when I toggle the booking status, then bookingEnabled changes to false.</li> <li>2. Given bookingEnabled is false, when an attendee attempts to book, then they receive a 'Booking not available' message.</li> <li>3. Given bookingEnabled is false, when I toggle booking status, then bookingEnabled changes back to true.</li> <li>4. Given the event already has existing bookings, when I disable booking, then existing bookings remain valid and unaffected.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Enable booking on an event.</li> <li>✓ Disable booking on an event.</li> <li>✓ Verify existing bookings remain valid when booking is disabled.</li> <li>✓ Prevent new bookings from being created when booking is disabled.</li> </ul>                                                                                                                                                                                                                                                                                          |
| <b>Code Reference</b>      | <p>EventService.toggleBookingEnabled() — lines 175–179</p> <p>EventController.toggleBooking() — lines 58–66</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

Table (54): Detailed User Stories 7

## Epic 2: Booking Management System

| US-2.1 Create Booking   Priority: P0 – Critical   Story Points: 13 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User Story                                                         | <p>As a Event Attendee,</p> <p>I want to book events with seat selection and optional services,</p> <p>so that I can secure my attendance and personalize my experience.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Acceptance Criteria                                                | <ol style="list-style-type: none"> <li>1. Given I am viewing an event, when I select the number of seats, then the system validates seat availability in real time.</li> <li>2. Given seats are available, when I proceed to booking, then I may optionally add supplementary services.</li> <li>3. Given I have selected seats and optional services, when I review the booking summary, then I see the total price computed as (ticket price × number of seats) + service costs.</li> <li>4. Given the event type is HOST_PACKAGE, when I create a booking, then I am required to select a specific event date.</li> <li>5. Given I select a date for a HOST_PACKAGE event, when that date is already booked, then I receive a 'Date unavailable' error.</li> <li>6. Given all validations pass, when I complete the booking, then a booking record is created with CONFIRMED status.</li> <li>7. Given the booking is created, when I check my email, then I receive a confirmation email containing a unique reference number.</li> <li>8. Given the booking is successful, when I view the event details, then the available seat count is decremented accordingly.</li> </ol> |
| Test Scenarios                                                     | <ul style="list-style-type: none"> <li>✓ Book a PUBLIC_EVENT with a valid number of seats.</li> <li>✓ Book a HOST_PACKAGE with a valid selected date.</li> <li>✓ Add optional services to a booking.</li> <li>✓ Validate that the total price is calculated correctly.</li> <li>✓ Prevent double-booking on the same HOST_PACKAGE date.</li> <li>✓ Verify that a unique booking reference number is generated.</li> <li>✓ Verify that a confirmation email is dispatched.</li> <li>✓ Verify that available seats are decremented after booking.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Code Reference                                                     | <p>BookingService.createBooking() — lines 48–142</p> <p>BookingController.createBooking() — lines 24–31</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

Table (55): Detailed User Stories 8

## US-2.2 View Booking Details | Priority: P0 – Critical | Story Points: 3

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Event Attendee,</p> <p>I want to view my booking information,</p> <p>so that I can review my reservation details at any time.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given I have a confirmed booking, when I view booking details, then I see the unique booking reference number.</li> <li>2. Given I am viewing a booking, when I inspect the details, then I see the event title, date, and location.</li> <li>3. Given I am viewing a booking, when I scroll further, then I see the number of seats booked and the total amount paid.</li> <li>4. Given I added services to my booking, when I view booking details, then I see the complete list of selected services.</li> <li>5. Given I am viewing a booking, when I check the status section, then I see both booking status and payment status.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Display complete booking information.</li> <li>✓ Show the list of selected services.</li> <li>✓ Display both booking and payment statuses.</li> <li>✓ Return HTTP 404 when booking is not found.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Code Reference</b>      | <p>BookingController.getBooking() — lines 35–43</p> <p>BookingService.getBookingById()</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

Table (56): Detailed User Stories 9

### US-2.3 View Upcoming Bookings | Priority: P1 – High | Story Points: 5

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Event Attendee,</p> <p>I want to see my future event bookings,</p> <p>so that I can plan my schedule effectively.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given I have bookings, when I view upcoming bookings, then I see only CONFIRMED bookings whose event date is in the future.</li> <li>2. Given I have canceled bookings, when I view upcoming bookings, then canceled bookings are not included.</li> <li>3. Given I have multiple upcoming bookings, when I view the list, then bookings are sorted by event date in ascending order (earliest first).</li> <li>4. Given I have no upcoming bookings, when I view the section, then I see a 'No upcoming bookings' message.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Display only upcoming bookings.</li> <li>✓ Exclude canceled bookings from the list.</li> <li>✓ Exclude past bookings from the list.</li> <li>✓ Verify bookings are sorted by date (ascending).</li> <li>✓ Handle an empty upcoming bookings list gracefully.</li> </ul>                                                                                                                                                                                                                                                                 |
| <b>Code Reference</b>      | <p>BookingController.getUserUpcomingBookings() — lines 53–61</p> <p>BookingService.getUserUpcomingBookings() — lines 164–172</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

Table (57): Detailed User Stories 10

| US-2.4 View Past Bookings   Priority: P1 – High   Story Points: 5 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User Story                                                        | <p>As a Event Attendee,</p> <p>I want to see my past event bookings,</p> <p>so that I can review my event attendance history.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Acceptance Criteria                                               | <ol style="list-style-type: none"> <li>1. Given I have attended events, when I view past bookings, then I see only CONFIRMED bookings whose event date has passed.</li> <li>2. Given I have canceled bookings, when I view past bookings, then canceled bookings are not included.</li> <li>3. Given I have multiple past bookings, when I view the list, then bookings are sorted by event date in descending order (most recent first).</li> <li>4. Given I have no past bookings, when I view the section, then I see a 'No past bookings' message.</li> </ol> |
| Test Scenarios                                                    | <ul style="list-style-type: none"> <li>✓ Display only past bookings.</li> <li>✓ Exclude canceled bookings.</li> <li>✓ Exclude upcoming bookings.</li> <li>✓ Verify bookings are sorted by date (descending).</li> <li>✓ Handle an empty past bookings list gracefully.</li> </ul>                                                                                                                                                                                                                                                                                 |
| Code Reference                                                    | <p>BookingController.getUserPastBookings() — lines 63–71</p> <p>BookingService.getUserPastBookings() — lines 173–180</p>                                                                                                                                                                                                                                                                                                                                                                                                                                          |

Table (58): Detailed User Stories 11

## US-2.5 Cancel Booking | Priority: P1 – High | Story Points: 8

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Event Attendee,</p> <p>I want to cancel my booking,</p> <p>so that I can receive a refund if I am unable to attend.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given I have a confirmed booking, when I request cancellation, then the system verifies I am the booking owner.</li> <li>2. Given the event date has not yet passed, when I cancel the booking, then the booking status changes to CANCELED.</li> <li>3. Given the booking is canceled, when I check the payment status, then the payment status changes to REFUNDED.</li> <li>4. Given the booking is canceled, when I check my email, then I receive a cancellation confirmation email.</li> <li>5. Given the event date has already passed, when I attempt to cancel, then I receive a 'Cannot cancel a past event' error.</li> <li>6. Given the booking is canceled, when I view the event details, then the available seat count is incremented.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Cancel a confirmed booking successfully.</li> <li>✓ Verify booking status changes to CANCELED.</li> <li>✓ Verify payment status changes to REFUNDED.</li> <li>✓ Verify a cancellation confirmation email is sent.</li> <li>✓ Prevent cancellation of bookings for past events.</li> <li>✓ Verify only the booking owner can cancel.</li> <li>✓ Verify available seats are incremented after cancellation.</li> </ul>                                                                                                                                                                                                                                                                                                                                              |
| <b>Code Reference</b>      | <p>BookingService.cancelBooking() — lines 182–206</p> <p>BookingController.cancelBooking() — lines 73–81</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

Table (59): Detailed User Stories 12



## US-2.6 Get Booked Dates | Priority: P2 – Medium | Story Points: 3

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Event Attendee,</p> <p>I want to see which dates are already booked for HOST_PACKAGE events, so that I can select an available date when making a booking.</p>                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given the event is of type HOST_PACKAGE, when I view the booking form, then I see a calendar with booked dates visually marked.</li> <li>2. Given a date is already booked, when I attempt to select it, then the date is disabled and cannot be chosen.</li> <li>3. Given I select an available date, when I proceed with the booking, then the system allows the booking to continue.</li> <li>4. Given no dates are currently booked, when I view the calendar, then all future dates are shown as available.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Return the complete list of booked dates for an event.</li> <li>✓ Include only CONFIRMED bookings in the booked dates list.</li> <li>✓ Verify dates are formatted consistently.</li> <li>✓ Handle the case where no bookings exist for the event.</li> </ul>                                                                                                                                                                                                                                                                 |
| <b>Code Reference</b>      | <p>BookingService.getBookedDatesForEvent() — lines 233–241</p> <p>BookingController.getBookedDates() — lines 83–91</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                |

Table (60): Detailed User Stories 13

## US-2.7 Notify Event Attendees | Priority: P2 – Medium | Story Points: 5

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Event Organizer,</p> <p>I want to send custom messages to event attendees,</p> <p>so that I can communicate important information directly to confirmed participants.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given I am the event organizer, when I compose a message, then I can enter a subject and message body.</li> <li>2. Given I submit the notification, when the system processes the request, then an email is sent to all confirmed attendees.</li> <li>3. Given an attendee has disabled event update notifications, when I send the notification, then that attendee does not receive an email.</li> <li>4. Given the notification is dispatched, when I review the results, then I see the total count of emails successfully sent.</li> <li>5. Given an attendee has an invalid email address, when I send the notification, then that attendee is skipped and the process continues.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Send notification to all confirmed attendees.</li> <li>✓ Verify notification preferences are respected.</li> <li>✓ Skip attendees with invalid email addresses.</li> <li>✓ Return the correct count of emails sent.</li> <li>✓ Enforce that both subject and message body are required.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Code Reference</b>      | <p>BookingService.notifyEventAttendees() — lines 207–226</p> <p>BookingController.notifyAttendees() — lines 93–108</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

Table (61): Detailed User Stories 14

## Epic 3: User Management & Authentication

| US-3.1 User Registration   Priority: P0 – Critical   Story Points: 8 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User Story                                                           | <p>As a New User,</p> <p>I want to create an account on the platform,</p> <p>so that I can access the marketplace and its features.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Acceptance Criteria                                                  | <ol style="list-style-type: none"> <li>1. Given I am on the registration page, when I enter my email, password, name, and phone, then the system validates the email format.</li> <li>2. Given I enter valid information, when I select a user role (USER, ORGANIZER, or SUPPLIER), then the system creates a Firebase Authentication user.</li> <li>3. Given the Firebase user is created, when the system initializes my profile, then the user profile is saved to the database.</li> <li>4. Given registration is successful, when I check my email, then I receive a welcome email.</li> <li>5. Given the email address already exists in the system, when I attempt to register, then I receive an 'Email already in use' error.</li> </ol> |
| Test Scenarios                                                       | <ul style="list-style-type: none"> <li>✓ Register with valid user data.</li> <li>✓ Validate email format before submission.</li> <li>✓ Validate password strength requirements.</li> <li>✓ Create the Firebase Authentication user.</li> <li>✓ Persist the user profile in the database.</li> <li>✓ Verify a welcome email is sent upon success.</li> <li>✓ Handle duplicate email addresses with an appropriate error.</li> </ul>                                                                                                                                                                                                                                                                                                                |
| Code Reference                                                       | <p>AuthService.registerUser() — lines 28–59</p> <p>AuthController.register() — lines 25–33</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

Table (62): Detailed User Stories 15

### US-3.2 Token Verification | Priority: P0 – Critical | Story Points: 5

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Registered User,</p> <p>I want my authentication token to be verified on each request,</p> <p>so that I can securely access protected resources.</p>                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given I am logged in, when I make an API request, then the system verifies my Firebase authentication token.</li> <li>2. Given the token is valid, when the system completes verification, then I am granted access to the requested resource.</li> <li>3. Given the token is invalid or expired, when the system verifies it, then I receive an HTTP 401 Unauthorized error.</li> <li>4. Given the token is verified successfully, when the system retrieves my user profile, then my user information is returned.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Verify a valid authentication token.</li> <li>✓ Reject an invalid token with HTTP 401.</li> <li>✓ Reject an expired token with HTTP 401.</li> <li>✓ Return the user profile upon successful verification.</li> <li>✓ Handle requests with no token provided.</li> </ul>                                                                                                                                                                                                                                                          |
| <b>Code Reference</b>      | <p>AuthService.verifyToken() — lines 64–66</p> <p>AuthController.verifyToken() — lines 35–50</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

Table (63): Detailed User Stories 16

### US-3.3 Update User Profile | Priority: P1 – High | Story Points: 8

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Registered User,</p> <p>I want to modify my profile information,</p> <p>so that my personal details remain accurate and up to date.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given I am logged in, when I update my name or phone number, then the changes are saved immediately.</li> <li>2. Given I upload a new profile picture, when the upload completes, then the image is stored in Firebase Storage.</li> <li>3. Given a profile picture has been uploaded, when I view my profile, then the new profile picture is displayed.</li> <li>4. Given I change my email address, when I save the changes, then the email is updated in both Firebase Authentication and the database.</li> <li>5. Given the update is successful, when I inspect the record, then the updatedAt timestamp reflects the current time.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Update name and phone number.</li> <li>✓ Upload a new profile picture.</li> <li>✓ Update email in Firebase Authentication and database.</li> <li>✓ Verify the profile picture is stored in Firebase Storage.</li> <li>✓ Verify the updatedAt timestamp is refreshed.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Code Reference</b>      | <p>AuthService.updateUserProfile() — lines 68–100</p> <p>AuthController.updateProfile() — lines 52–63</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

Table (64): Detailed User Stories 17

| US-3.4 Delete Profile Picture   Priority: P2 – Medium   Story Points: 3 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User Story                                                              | <p>As a Registered User,</p> <p>I want to remove my profile picture,</p> <p>so that I can upload a new one or revert to the default avatar.</p>                                                                                                                                                                                                                                                                                                                                                                                                           |
| Acceptance Criteria                                                     | <ol style="list-style-type: none"> <li>1. Given I have a profile picture, when I request deletion, then the image is removed from Firebase Storage.</li> <li>2. Given the profile picture is deleted, when I view my profile, then the default avatar is displayed.</li> <li>3. Given the profile picture is deleted, when I inspect the database record, then the profilePictureUrl field is set to null.</li> <li>4. Given I do not have a profile picture, when I attempt to delete, then I receive a 'No profile picture to delete' error.</li> </ol> |
| Test Scenarios                                                          | <ul style="list-style-type: none"> <li>✓ Delete an existing profile picture.</li> <li>✓ Verify the file is removed from Firebase Storage.</li> <li>✓ Verify the profilePictureUrl is set to null in the database.</li> <li>✓ Handle the case where no profile picture exists.</li> </ul>                                                                                                                                                                                                                                                                  |
| Code Reference                                                          | <p>AuthService.deleteProfilePicture() — lines 102–122</p> <p>AuthController.deleteProfilePicture() — lines 65–73</p>                                                                                                                                                                                                                                                                                                                                                                                                                                      |

Table (65): Detailed User Stories 18

### US-3.5 Update Notification Preferences | Priority: P1 – High | Story Points: 5

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Registered User,</p> <p>I want to control which email notifications I receive,</p> <p>so that I only receive communications that are relevant to me.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given I am on the settings page, when I view notification preferences, then I see options for booking confirmations, event reminders, and event updates.</li> <li>2. Given I disable booking confirmations, when I create a booking, then I do not receive a confirmation email.</li> <li>3. Given I disable event updates, when an organizer updates an event I have booked, then I do not receive an update email.</li> <li>4. Given I enable event reminders, when an event I have booked is scheduled for the following day, then I receive a reminder email.</li> <li>5. Given I save my preferences, when I inspect the database, then the preferences are correctly stored in my user profile.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Enable and disable booking confirmation notifications.</li> <li>✓ Enable and disable event reminder notifications.</li> <li>✓ Enable and disable event update notifications.</li> <li>✓ Verify preferences are persisted in the database.</li> <li>✓ Verify the system respects saved preferences when sending emails.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Code Reference</b>      | <p>AuthService.updateNotificationPreferences() — lines 124–132</p> <p>AuthController.updateNotificationPreferences() — lines 75–85</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

Table (66): Detailed User Stories 19

### US-3.6 Delete Account | Priority: P2 – Medium | Story Points: 5

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | <p>As a Registered User,</p> <p>I want to permanently delete my account,</p> <p>so that all my personal data is removed from the system.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Acceptance Criteria</b> | <ol style="list-style-type: none"> <li>1. Given I am logged in, when I request account deletion, then the system prompts for confirmation before proceeding.</li> <li>2. Given I confirm the deletion, when the system processes the request, then my user account is removed from Firebase Authentication.</li> <li>3. Given the Firebase user is deleted, when the system completes cleanup, then my user profile is removed from the database.</li> <li>4. Given my account is deleted, when I attempt to log in, then I receive a 'User not found' error.</li> <li>5. Given my account is deleted, when I attempt account recovery, then recovery is not possible as all data has been permanently removed.</li> </ol> |
| <b>Test Scenarios</b>      | <ul style="list-style-type: none"> <li>✓ Delete a user account successfully.</li> <li>✓ Verify the Firebase Authentication account is removed.</li> <li>✓ Verify the user profile is removed from the database.</li> <li>✓ Prevent login after account deletion.</li> <li>✓ Handle the case where the user is not found.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Code Reference</b>      | <p>AuthService.deleteUserAccount() — lines 134–137</p> <p>AuthController.deleteAccount() — lines 87–95</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

Table (67): Detailed User Stories 20



## 4. User Story Format Reference

---

All user stories in this document follow the standard agile format defined below:

| Element                    | Description                                                                                                                                                    |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Story</b>          | Expressed as: As a [role], I want [goal], so that [benefit]. Captures the who, what, and why.                                                                  |
| <b>Acceptance Criteria</b> | Written in Given–When–Then (GWT) format. Each criterion defines a concrete, testable condition that must be satisfied for the story to be considered complete. |
| <b>Test Scenarios</b>      | Specific test cases derived from the acceptance criteria. Each scenario is independently verifiable and maps directly to a system behavior.                    |
| <b>Code Reference</b>      | The specific service method and controller method (with line numbers) that implement this story, enabling full traceability between requirements and code.     |
| <b>Story Points</b>        | Relative effort estimates using a Fibonacci-like scale. Used for sprint planning and velocity tracking.                                                        |
| <b>Priority</b>            | P0 (Critical) — must be delivered. P1 (High) — high business value. P2 (Medium) — important but deferrable.                                                    |

*Table (68): User Story Format Reference*

## 4-9 Requirements Analysis and Documentation

### 4-9-1 Requirements Traceability Matrix (RTM)

| Requirement ID | Requirement Description                                   | Backend Reference                                                         | Frontend Reference                                                         | Test Case ID          | Status |
|----------------|-----------------------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------|--------|
| FR1            | The system shall allow users to create a new account.     | <a href="#">AuthController.java</a><br><a href="#">AuthService.java</a>   | <a href="#">RegisterPage.jsx</a> ,<br><a href="#">authService.js</a>       | <a href="#">TC-01</a> | Passed |
| FR2            | The system shall allow users to log in.                   | <a href="#">AuthController.java</a><br><a href="#">AuthService.java</a>   | <a href="#">LoginPage.jsx</a><br><a href="#">AuthContext.jsx</a>           | <a href="#">TC-02</a> | Passed |
| FR3            | The system shall allow users to log out.                  | <a href="#">AuthContext.jsx</a>                                           | <a href="#">AuthContext.jsx</a>                                            | <a href="#">TC-03</a> | Passed |
| FR4            | The system shall support password recovery via email.     | <a href="#">AuthController.java</a><br><a href="#">EmailService.java</a>  | <a href="#">ForgotPasswordPage.jsx</a>                                     | <a href="#">TC-04</a> | Passed |
| FR5            | The system shall allow users to edit profile information. | <a href="#">AuthController.java</a><br><a href="#">AuthService.java</a>   | <a href="#">ProfilePage.jsx</a><br><a href="#">EditProfileDialog.jsx</a>   | <a href="#">TC-05</a> | Passed |
| FR6            | The system shall allow users to delete their accounts.    | <a href="#">AuthController.java</a><br><a href="#">AuthService.java</a>   | <a href="#">ProfilePage.jsx</a><br><a href="#">DeleteAccountDialog.jsx</a> | <a href="#">TC-06</a> | Passed |
| FR7            | The system shall manage user roles and permissions.       | <a href="#">User.java (Role enum)</a><br><a href="#">AuthService.java</a> | <a href="#">AuthContext.jsx</a><br><a href="#">ProtectedRoute.jsx</a>      | <a href="#">TC-07</a> | Passed |

|      |                                                             |                                                                                                                   |                                                                        |                       |        |
|------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|-----------------------|--------|
| FR8  | The system shall allow users to browse available events.    | <a href="#">EventController.java</a><br><a href="#">EventService.java</a><br><a href="#">EventRepository.java</a> | <a href="#">EventsListPage.jsx</a><br><a href="#">EventCard.jsx</a>    | <a href="#">TC-08</a> | Passed |
| FR9  | The system shall display detailed event information.        | <a href="#">EventController.java</a><br><a href="#">EventService.java</a>                                         | <a href="#">EventDetailsView.jsx</a>                                   | <a href="#">TC-09</a> | Passed |
| FR10 | The system shall provide event search functionality.        | <a href="#">EventService.java</a>                                                                                 | <a href="#">EventsListPage.jsx</a><br><a href="#">EventFilters.jsx</a> | <a href="#">TC-10</a> | Passed |
| FR11 | The system shall allow filtering events by date.            | <a href="#">EventService.java</a>                                                                                 | <a href="#">EventFilters.jsx</a>                                       | <a href="#">TC-11</a> | Passed |
| FR12 | The system shall allow filtering events by location.        | <a href="#">EventService.java</a>                                                                                 | <a href="#">EventFilters.jsx</a>                                       | <a href="#">TC-12</a> | Passed |
| FR13 | The system shall allow filtering events by price.           | <a href="#">EventService.java</a>                                                                                 | <a href="#">EventFilters.jsx</a>                                       | <a href="#">TC-13</a> | Passed |
| FR14 | The system shall allow filtering events by type.            | <a href="#">EventService.java</a>                                                                                 | <a href="#">EventFilters.jsx</a>                                       | <a href="#">TC-14</a> | Passed |
| FR15 | The system shall allow users to book public events.         | <a href="#">BookingController.java</a><br><a href="#">BookingService.java</a><br><a href="#">Booking.java</a>     | <a href="#">BookingFormPage.js</a><br><a href="#">x</a>                | <a href="#">TC-15</a> | Passed |
| FR16 | The system shall allow users to book host packages.         | <a href="#">BookingController.java</a><br><a href="#">BookingService.java</a>                                     | <a href="#">BookingFormPage.js</a><br><a href="#">x</a>                | <a href="#">TC-16</a> | Passed |
| FR17 | The system shall generate unique booking reference numbers. | <a href="#">BookingService.java</a>                                                                               | <a href="#">BookingConfirmationPage.jsx</a>                            | <a href="#">TC-17</a> | Passed |

|      |                                                                      |                                                                                                         |                                                                                   |                       |        |
|------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------|--------|
| FR18 | The system shall support online payment functionality.               | <a href="#">PaymentService.java</a><br><a href="#">Payment.java</a>                                     | <a href="#">BookingFormPage.js</a><br><a href="#">x</a>                           | <a href="#">TC-18</a> | Passed |
| FR19 | The system shall automatically update available seats after booking. | <a href="#">BookingService.java</a><br><a href="#">EventService.java</a>                                | <a href="#">EventCard.jsx</a>                                                     | <a href="#">TC-19</a> | Passed |
| FR20 | The system shall prevent overbooking of events.                      | <a href="#">BookingService.java</a>                                                                     | <a href="#">BookingFormPage.js</a><br><a href="#">x</a>                           | <a href="#">TC-20</a> | Passed |
| FR21 | The system shall generate QR code tickets.                           | <a href="#">BookingService.java</a>                                                                     | <a href="#">BookingConfirmationPage.jsx</a><br><a href="#">MyBookingsPage.jsx</a> | <a href="#">TC-21</a> | Passed |
| FR22 | The system shall allow users to view their bookings.                 | <a href="#">BookingController.java</a><br><a href="#">BookingService.java</a>                           | <a href="#">MyBookingsPage.jsx</a><br><a href="#">BookingCard.jsx</a>             | <a href="#">TC-22</a> | Passed |
| FR23 | The system shall allow users to cancel bookings.                     | <a href="#">BookingController.java</a><br><a href="#">BookingService.java</a>                           | <a href="#">MyBookingsPage.jsx</a>                                                | <a href="#">TC-23</a> | Passed |
| FR24 | The system shall send booking confirmation emails.                   | <a href="#">EmailService.java</a><br><a href="#">BookingService.java</a>                                | -                                                                                 | <a href="#">TC-24</a> | Passed |
| FR25 | The system shall allow organizers to create events.                  | <a href="#">EventController.java</a><br><a href="#">EventService.java</a><br><a href="#">Event.java</a> | <a href="#">CreateEditEventPage</a><br><a href="#">.jsx</a>                       | <a href="#">TC-25</a> | Passed |
| FR26 | The system shall allow organizers to update event details.           | <a href="#">EventController.java</a><br><a href="#">EventService.java</a>                               | <a href="#">CreateEditEventPage</a><br><a href="#">.jsx</a>                       | <a href="#">TC-26</a> | Passed |

|      |                                                                  |                                                                                                                          |                                         |                       |        |
|------|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|-----------------------|--------|
| FR27 | The system shall allow organizers to delete events.              | <a href="#">EventController.java</a><br><a href="#">EventService.java</a>                                                | <a href="#">OrganizerEventsPage.jsx</a> | <a href="#">TC-27</a> | Passed |
| FR28 | The system shall support different event types.                  | <a href="#">Event.java (EventType enum)</a>                                                                              | <a href="#">CreateEditEventPage.jsx</a> | <a href="#">TC-28</a> | Passed |
| FR29 | The system shall support entering complete event details.        | <a href="#">Event.java</a>                                                                                               | <a href="#">CreateEditEventPage.jsx</a> | <a href="#">TC-29</a> | Passed |
| FR30 | The system shall support different event statuses.               | <a href="#">Event.java (EventStatus enum)</a>                                                                            | <a href="#">EventCard.jsx</a>           | <a href="#">TC-30</a> | Passed |
| FR31 | The system shall allow organizers to enable or disable bookings. | <a href="#">EventController.java</a><br><a href="#">EventService.java</a>                                                | <a href="#">OrganizerEventsPage.jsx</a> | <a href="#">TC-31</a> | Passed |
| FR32 | The system shall provide a dashboard interface.                  | <a href="#">DashboardController.java</a><br><a href="#">DashboardService.java</a><br><a href="#">DashboardStats.java</a> | <a href="#">OrganizerDashboard.jsx</a>  | <a href="#">TC-32</a> | Passed |
| FR33 | The system shall display booking statistics.                     | <a href="#">DashboardService.java</a>                                                                                    | <a href="#">OrganizerDashboard.jsx</a>  | <a href="#">TC-33</a> | Passed |
| FR34 | The system shall display remaining seat statistics.              | <a href="#">DashboardService.java</a><br><a href="#">EventStatsDTO.java</a>                                              | <a href="#">OrganizerDashboard.jsx</a>  | <a href="#">TC-34</a> | Passed |
| FR35 | The system shall display revenue statistics.                     | <a href="#">DashboardService.java</a>                                                                                    | <a href="#">OrganizerDashboard.jsx</a>  | <a href="#">TC-35</a> | Passed |
| FR36 | The system shall display attendance and cancellation rates.      | <a href="#">DashboardService.java</a>                                                                                    | <a href="#">OrganizerDashboard.jsx</a>  | <a href="#">TC-36</a> | Passed |

|      |                                                               |                                                                                                                  |                                                                                   |                       |        |
|------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------|--------|
| FR37 | The system shall allow organizers to view all event bookings. | <a href="#">BookingController.java</a><br><a href="#">BookingService.java</a>                                    | <a href="#">OrganizerDashboard.jsx</a>                                            | <a href="#">TC-37</a> | Passed |
| FR38 | The system shall allow organizers to view attendee lists.     | <a href="#">BookingController.java</a><br><a href="#">BookingService.java</a>                                    | <a href="#">OrganizerBookingsPage.jsx</a>                                         | <a href="#">TC-38</a> | Passed |
| FR39 | The system shall monitor payment statuses.                    | <a href="#">PaymentService.java</a><br><a href="#">BookingService.java</a>                                       | <a href="#">OrganizerBookingsPage.jsx</a>                                         | <a href="#">TC-39</a> | Passed |
| FR40 | The system shall allow creation of service provider accounts. | <a href="#">SupplierController.java</a><br><a href="#">SupplierService.java</a><br><a href="#">Supplier.java</a> | <a href="#">SupplierProfilePage.jsx</a>                                           | <a href="#">TC-40</a> | Passed |
| FR41 | The system shall allow suppliers to add services.             | <a href="#">ServiceController.java</a><br><a href="#">ServiceService.java</a><br><a href="#">Service.java</a>    | <a href="#">SupplierProfilePage.jsx</a>                                           | <a href="#">TC-41</a> | Passed |
| FR42 | The system shall allow suppliers to edit services.            | <a href="#">ServiceController.java</a><br><a href="#">ServiceService.java</a>                                    | <a href="#">SupplierProfilePage.jsx</a><br><a href="#">EditSupplierDialog.jsx</a> | <a href="#">TC-42</a> | Passed |
| FR43 | The system shall allow suppliers to delete services.          | <a href="#">ServiceController.java</a><br><a href="#">ServiceService.java</a>                                    | <a href="#">SupplierProfilePage.jsx</a>                                           | <a href="#">TC-43</a> | Passed |
| FR44 | The system shall support different service statuses.          | <a href="#">ServiceController.java</a><br><a href="#">Service.java</a><br><a href="#">(ServiceStatus enum)</a>   | <a href="#">SupplierProfilePage.jsx</a>                                           | <a href="#">TC-44</a> | Passed |
| FR45 | The system shall allow suppliers to view customer feedback.   | <a href="#">ReviewController.java</a><br><a href="#">ReviewService.java</a>                                      | <a href="#">SupplierProfilePage.jsx</a>                                           | <a href="#">TC-45</a> | Passed |
| FR46 | The system shall provide service                              | <a href="#">SupplierController.java</a><br><a href="#">SupplierService.java</a>                                  | <a href="#">SupplierDirectoryPage.jsx</a>                                         | <a href="#">TC-46</a> | Passed |

|      |                                                                  |                                                                                                                                   |                                                                                    |                       |        |
|------|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------|--------|
|      | provider search functionality.                                   |                                                                                                                                   |                                                                                    |                       |        |
| FR47 | The system shall allow users to rate attended events.            | <a href="#">ReviewController.java</a><br><a href="#">ReviewService.java</a><br><a href="#">Review.java</a>                        | <a href="#">RatingDialog.jsx</a><br><a href="#">MyBookingsPage.jsx</a>             | <a href="#">TC-47</a> | Passed |
| FR48 | The system shall restrict reviews to attendees only.             | <a href="#">ReviewService.java</a><br>(validation logic)                                                                          | <a href="#">RatingDialog.jsx</a>                                                   | <a href="#">TC-48</a> | Passed |
| FR49 | The system shall allow users to submit event reviews.            | <a href="#">ReviewController.java</a><br><a href="#">ReviewService.java</a>                                                       | <a href="#">RatingDialog.jsx</a>                                                   | <a href="#">TC-49</a> | Passed |
| FR50 | The system shall allow users to rate service providers.          | <a href="#">ReviewController.java</a><br><a href="#">ReviewService.java</a>                                                       | <a href="#">RatingDialog.jsx</a><br><a href="#">ProviderDetailPage.js</a><br>x     | <a href="#">TC-50</a> | Passed |
| FR51 | The system shall display reviews publicly.                       | <a href="#">ReviewController.java</a><br><a href="#">ReviewService.java</a>                                                       | <a href="#">EventDetailsView.jsx</a><br><a href="#">ProviderDetailPage.js</a><br>x | <a href="#">TC-51</a> | Passed |
| FR52 | The system shall allow users to enable or disable notifications. | <a href="#">AuthController.java</a><br><a href="#">AuthService.java</a><br><a href="#">User.java</a><br>(notificationPreferences) | <a href="#">ProfilePage.jsx</a>                                                    | <a href="#">TC-52</a> | Passed |
| FR53 | The system shall send welcome emails.                            | <a href="#">EmailService.java</a><br><a href="#">AuthService.java</a>                                                             | -                                                                                  | <a href="#">TC-53</a> | Passed |
| FR54 | The system shall send booking confirmation notifications.        | <a href="#">EmailService.java</a><br><a href="#">BookingService.java</a>                                                          | -                                                                                  | <a href="#">TC-54</a> | Passed |
| FR55 | The system shall send booking cancellation notifications.        | <a href="#">EmailService.java</a><br><a href="#">BookingService.java</a>                                                          | -                                                                                  | <a href="#">TC-55</a> | Passed |

|      |                                                         |                                                                                      |                                                                                 |                       |        |
|------|---------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-----------------------|--------|
| FR56 | The system shall send event update notifications.       | <a href="#">EmailService.java</a><br><a href="#">EventService.java</a>               | -                                                                               | <a href="#">TC-56</a> | Passed |
| FR57 | The system shall send event cancellation notifications. | <a href="#">EmailService.java</a><br><a href="#">EventService.java</a>               | -                                                                               | <a href="#">TC-57</a> | Passed |
| FR58 | The system shall send collaboration requests.           | <a href="#">EmailService.java</a>                                                    | -                                                                               | <a href="#">TC-58</a> | Passed |
| FR59 | The system shall send service status notifications.     | <a href="#">EmailService.java</a> ,<br><a href="#">ServiceService.java</a>           | -                                                                               | <a href="#">TC-59</a> | Passed |
| FR60 | The system shall provide interactive dashboards.        | <a href="#">DashboardController.java</a><br><a href="#">DashboardService.java</a>    | <a href="#">OrganizerDashboard.jsx</a><br><a href="#">SupplierDashboard.jsx</a> | <a href="#">TC-60</a> | Passed |
| FR61 | The system shall display ticket sales statistics.       | <a href="#">DashboardService.java</a>                                                | <a href="#">OrganizerDashboard.jsx</a>                                          | <a href="#">TC-61</a> | Passed |
| FR62 | The system shall generate revenue growth reports.       | <a href="#">DashboardService.java</a><br>( <a href="#">generatePDFReport</a> method) | <a href="#">OrganizerDashboard.jsx</a>                                          | TC-62                 | Passed |
| FR63 | The system shall display provider performance metrics.  | <a href="#">DashboardService.java</a><br><a href="#">ProviderPerformanceDTO.java</a> | <a href="#">OrganizerDashboard.jsx</a>                                          | TC-63                 | Passed |
| FR64 | The system shall display popular events.                | <a href="#">DashboardService.java</a>                                                | <a href="#">OrganizerDashboard.jsx</a>                                          | TC-64                 | Passed |
| FR65 | The system shall display popular services.              | <a href="#">DashboardService.java</a>                                                | <a href="#">OrganizerDashboard.jsx</a>                                          | TC-65                 | Passed |



|      |                                                                |                                                                                                                                               |                                                                    |       |        |
|------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|-------|--------|
| FR66 | The system shall support exporting reports as PDF files.       | <a href="#">DashboardController.java</a><br><a href="#">DashboardService.java</a>                                                             | <a href="#">OrganizerDashboard.jsx</a>                             | TC-66 | Passed |
| FR67 | The system shall allow users to enter event preferences.       | <a href="#">EventController.java</a><br><a href="#">EventService.java</a>                                                                     | <a href="#">AiEventPlannerPage.jsx</a>                             | TC-67 | Passed |
| FR68 | The system shall analyze events using artificial intelligence. | <a href="#">AiController.java</a><br><a href="#">AiService.java</a><br><a href="#">EventService.java</a><br><a href="#">(planEventWithAI)</a> | <a href="#">AiEventPlannerPage.jsx</a>                             | TC-68 | Passed |
| FR69 | The system shall display personalized recommendations .        | <a href="#">EventController.java</a><br><a href="#">EventService.java</a><br><a href="#">(getSmartRecommendations)</a>                        | <a href="#">HomePage.jsx</a><br><a href="#">EventsListPage.jsx</a> | TC-69 | Passed |
| FR70 | The system shall provide an AI-powered chatbot assistant.      | <a href="#">AiController.java</a><br><a href="#">AiService.java</a>                                                                           | <a href="#">ChatBot.jsx</a>                                        | TC-70 | Passed |
| FR71 | The system shall support natural language communication.       | <a href="#">AiService.java (chat method)</a>                                                                                                  | <a href="#">ChatBot.jsx</a>                                        | TC-71 | Passed |
| FR72 | The system shall use real-time event data.                     | <a href="#">AiService.java</a><br><a href="#">EventService.java</a>                                                                           | <a href="#">ChatBot.jsx</a>                                        | TC-72 | Passed |
| FR73 | The system shall display AI recommendation cards.              | <a href="#">EventService.java</a>                                                                                                             | <a href="#">EventCard.jsx</a><br><a href="#">HomePage.jsx</a>      | TC-73 | Passed |
| FR74 | The system shall analyze user booking history.                 | <a href="#">EventService.java</a><br><a href="#">(getSmartRecommendations)</a>                                                                | <a href="#">HomePage.jsx</a>                                       | TC-74 | Passed |

|      |                                                           |                                                                           |                              |       |        |
|------|-----------------------------------------------------------|---------------------------------------------------------------------------|------------------------------|-------|--------|
|      |                                                           | <a href="#">BookingService.java</a>                                       |                              |       |        |
| FR75 | The system shall provide a “Recommended for You” section. | <a href="#">EventController.java</a><br><a href="#">EventService.java</a> | <a href="#">HomePage.jsx</a> | TC-75 | Passed |

Table (69): RTM

#### 4-9-2 Test cases table:

##### Test cases table:

| Test Case ID | Requirement ID | Test Objective                         | Expected Result                                  | Status |
|--------------|----------------|----------------------------------------|--------------------------------------------------|--------|
| TC-01        | FR1            | Verify user registration functionality | User account is created successfully             | Passed |
| TC-02        | FR2            | Verify user login process              | User logs into the system successfully           | Passed |
| TC-03        | FR3            | Verify logout functionality            | User session is terminated successfully          | Passed |
| TC-04        | FR4            | Verify password recovery process       | Password reset email is sent successfully        | Passed |
| TC-05        | FR5            | Verify profile editing functionality   | User profile information is updated successfully | Passed |
| TC-06        | FR6            | Verify account deletion                | User account is permanently deleted              | Passed |
| TC-07        | FR7            | Verify role and permission management  | Different user roles are correctly managed       | Passed |

|       |      |                                     |                                                |        |
|-------|------|-------------------------------------|------------------------------------------------|--------|
| TC-08 | FR8  | Verify browsing available events    | Events are displayed successfully              | Passed |
| TC-09 | FR9  | Verify viewing event details        | Event details are displayed correctly          | Passed |
| TC-10 | FR10 | Verify event search functionality   | Matching events are returned successfully      | Passed |
| TC-11 | FR11 | Verify filtering events by date     | Events are filtered correctly by date          | Passed |
| TC-12 | FR12 | Verify filtering events by location | Events are filtered correctly by location      | Passed |
| TC-13 | FR13 | Verify filtering events by price    | Events are filtered correctly by price         | Passed |
| TC-14 | FR14 | Verify filtering events by type     | Events are filtered correctly by type          | Passed |
| TC-15 | FR15 | Verify booking public events        | Booking is created successfully                | Passed |
| TC-16 | FR16 | Verify booking host packages        | Host package booking is completed successfully | Passed |
| TC-17 | FR17 | Verify booking reference generation | Unique booking reference number is generated   | Passed |
| TC-18 | FR18 | Verify online payment functionality | Payment process completes successfully         | Passed |
| TC-19 | FR19 | Verify automatic seat updates       | Available seats decrease after booking         | Passed |
| TC-20 | FR20 | Verify capacity validation          | Booking is prevented when capacity is full     | Passed |
| TC-21 | FR21 | Verify digital ticket generation    | QR digital ticket is generated successfully    | Passed |
| TC-22 | FR22 | Verify viewing user bookings        | User bookings are displayed correctly          | Passed |
| TC-23 | FR23 | Verify booking cancellation         | Booking is canceled successfully               | Passed |
| TC-24 | FR24 | Verify booking confirmation email   | Confirmation email is sent successfully        | Passed |
| TC-25 | FR25 | Verify event creation               | Organizer creates event successfully           | Passed |
| TC-26 | FR26 | Verify updating event details       | Event information is updated successfully      | Passed |
| TC-27 | FR27 | Verify event deletion               | Event is deleted successfully                  | Passed |

|       |      |                                             |                                             |        |
|-------|------|---------------------------------------------|---------------------------------------------|--------|
| TC-28 | FR28 | Verify event type support                   | Different event types are handled correctly | Passed |
| TC-29 | FR29 | Verify entering complete event information  | Event data is stored correctly              | Passed |
| TC-30 | FR30 | Verify event status management              | Event statuses are updated correctly        | Passed |
| TC-31 | FR31 | Verify booking enable/disable functionality | Booking status changes successfully         | Passed |
| TC-32 | FR32 | Verify dashboard display                    | Organizer dashboard loads successfully      | Passed |
| TC-33 | FR33 | Verify booking statistics display           | Booking statistics are calculated correctly | Passed |
| TC-34 | FR34 | Verify remaining seats statistics           | Remaining seats are displayed correctly     | Passed |
| TC-35 | FR35 | Verify revenue statistics                   | Revenue analytics are displayed correctly   | Passed |
| TC-36 | FR36 | Verify attendance and cancellation rates    | KPIs are calculated correctly               | Passed |
| TC-37 | FR37 | Verify viewing event bookings               | Organizer can access booking list           | Passed |
| TC-38 | FR38 | Verify attendee list viewing                | Attendee list is displayed successfully     | Passed |
| TC-39 | FR39 | Verify payment status monitoring            | Payment statuses are displayed correctly    | Passed |
| TC-40 | FR40 | Verify supplier account creation            | Supplier account is created successfully    | Passed |
| TC-41 | FR41 | Verify adding services                      | Service is added successfully               | Passed |
| TC-42 | FR42 | Verify editing services                     | Service information is updated successfully | Passed |
| TC-43 | FR43 | Verify deleting services                    | Service is deleted successfully             | Passed |
| TC-44 | FR44 | Verify service status management            | Service status changes correctly            | Passed |
| TC-45 | FR45 | Verify viewing customer feedback            | Customer reviews are displayed correctly    | Passed |

|       |      |                                            |                                                  |        |
|-------|------|--------------------------------------------|--------------------------------------------------|--------|
| TC-46 | FR46 | Verify searching service providers         | Matching providers are displayed successfully    | Passed |
| TC-47 | FR47 | Verify rating attended events              | Rating is submitted successfully                 | Passed |
| TC-48 | FR48 | Verify review restrictions                 | Only attendees can submit reviews                | Passed |
| TC-49 | FR49 | Verify event review submission             | Review is stored successfully                    | Passed |
| TC-50 | FR50 | Verify rating service providers            | Provider rating is submitted successfully        | Passed |
| TC-51 | FR51 | Verify displaying reviews publicly         | Reviews appear publicly on the platform          | Passed |
| TC-52 | FR52 | Verify notification preferences            | Notification settings are updated successfully   | Passed |
| TC-53 | FR53 | Verify welcome email notifications         | Welcome email is sent successfully               | Passed |
| TC-54 | FR54 | Verify booking confirmation notifications  | Booking notification is delivered successfully   | Passed |
| TC-55 | FR55 | Verify booking cancellation notifications  | Cancellation email is sent successfully          | Passed |
| TC-56 | FR56 | Verify event update notifications          | Event update notifications are delivered         | Passed |
| TC-57 | FR57 | Verify event cancellation notifications    | Cancellation notifications are sent successfully | Passed |
| TC-58 | FR58 | Verify collaboration request notifications | Collaboration email is sent successfully         | Passed |
| TC-59 | FR59 | Verify service status notifications        | Service notifications are delivered correctly    | Passed |
| TC-60 | FR60 | Verify interactive dashboards              | Dashboard components function correctly          | Passed |
| TC-61 | FR61 | Verify ticket sales statistics             | Ticket sales statistics are displayed correctly  | Passed |
| TC-62 | FR62 | Verify revenue growth reports              | Revenue reports are generated successfully       | Passed |

|       |      |                                       |                                                    |        |
|-------|------|---------------------------------------|----------------------------------------------------|--------|
| TC-63 | FR63 | Verify provider performance metrics   | Provider performance analytics are displayed       | Passed |
| TC-64 | FR64 | Verify displaying popular events      | Popular events are displayed correctly             | Passed |
| TC-65 | FR65 | Verify displaying popular services    | Popular services are displayed correctly           | Passed |
| TC-66 | FR66 | Verify exporting reports as PDF       | PDF report is generated successfully               | Passed |
| TC-67 | FR67 | Verify entering event preferences     | User preferences are processed successfully        | Passed |
| TC-68 | FR68 | Verify AI event analysis              | AI recommendations are generated correctly         | Passed |
| TC-69 | FR69 | Verify personalized recommendations   | Personalized recommendations are displayed         | Passed |
| TC-70 | FR70 | Verify AI chatbot functionality       | Chatbot responds successfully                      | Passed |
| TC-71 | FR71 | Verify natural language communication | AI processes user messages correctly               | Passed |
| TC-72 | FR72 | Verify real-time event data usage     | AI uses real event data successfully               | Passed |
| TC-73 | FR73 | Verify AI recommendation cards        | Recommendation cards are displayed correctly       | Passed |
| TC-74 | FR74 | Verify booking history analysis       | Recommendations are generated from booking history | Passed |
| TC-75 | FR75 | Verify “Recommended for You” section  | Personalized section is displayed successfully     | Passed |

Table (70): Test Case

## **Chapter 5:**

# **System Architecture Design**

## 5-1- High-Level Design:

The system consists of three main layers:

### 1. Presentation Layer:

- React (Frontend)
- Responsible for displaying data and handling user interaction
- Provides the user interface for the system

### 2. Application Layer:

APIs (Flask Services) Includes:

- User Service
- Event Service
- Booking/Borrow Service
- This layer handles business logic and communication between the frontend and the data layer.

### 3. Data Layer:

- Databases for each service (or shared database depending on implementation).
- Stores users, events, and bookings information.



### 5-1-1- System Block Diagram:

The system can be described as follows:

- User → interacts with → Frontend (React).
- Frontend → sends HTTP requests → APIs.
- APIs → communicate with → Databases.
- Services communicate with each other via HTTP requests.

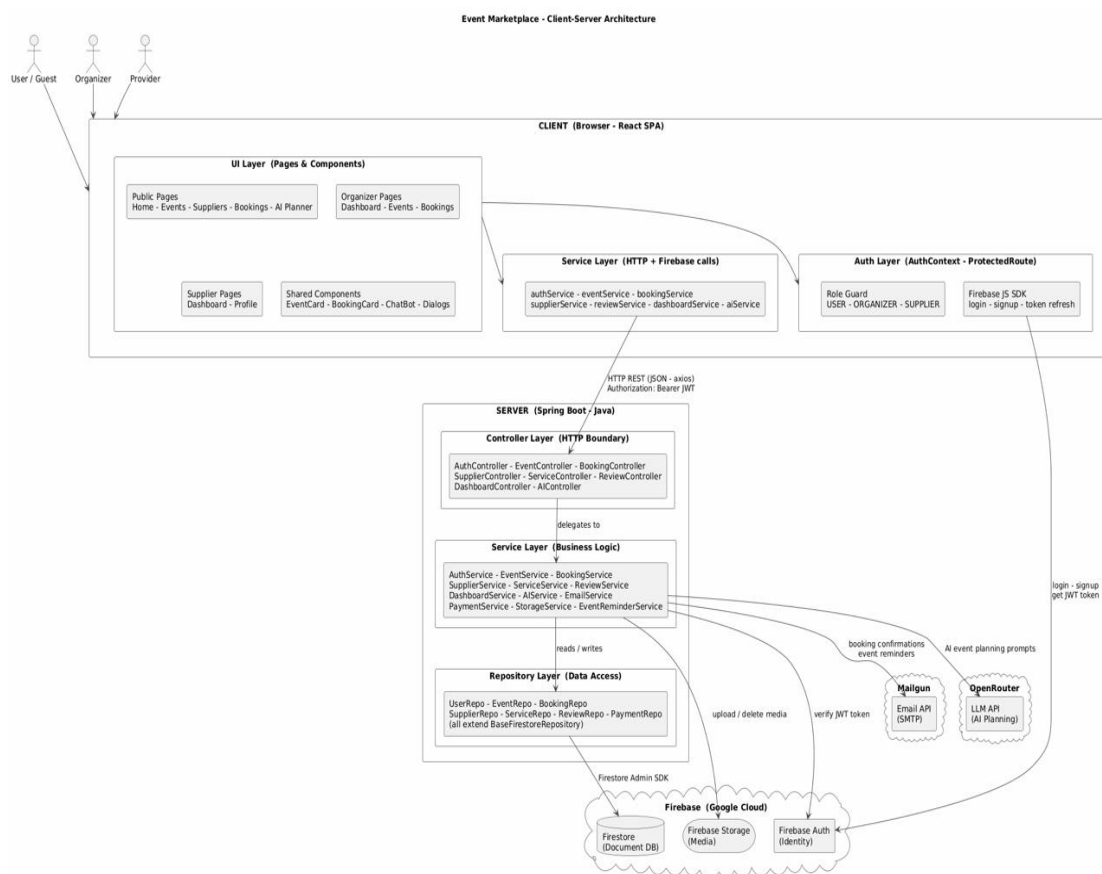


Figure 38: System Block Diagram

### 5-1-2- Architectural Design Pattern Used:

The system is based on a layered architecture. The system is divided into three main layers to ensure separation of responsibilities and scalability.

#### *1. Presentation Layer:*

Developed using React, this layer is responsible for:

- Presenting data to the user
- Receiving user input
- Sending HTTP requests to back-end services (APIs)

#### *2. Application Layer:*

This layer consists of a set of microservices built using Flask, including:

- User Service: Responsible for managing user data
- Event Service: Responsible for managing events
- Booking Service: Responsible for executing bookings, including user and event verification logic through calls to other services

This layer relies on REST APIs for communication between services.

#### *3. Data Layer:*

This layer is used to store system data, including:

- User data
- Event data
- Booking data

### 5-1-3- System Architecture Design:

The system is implemented using a microservices-based architecture, where each service operates independently and communicates with other services through RESTful APIs.

When a user performs an action through the frontend (such as booking an event), a request is sent to the Booking Service, which represents the core component responsible for handling the main business logic of the system.

The booking process follows a clear sequence:

- First, the Booking Service sends a request to the User Service to verify that the user exists
- Then, it sends another request to the Event Service to check that the event exists
- If both validations are successful, the booking is created and stored
- Finally, a response is returned to the frontend indicating whether the operation was successful or not.

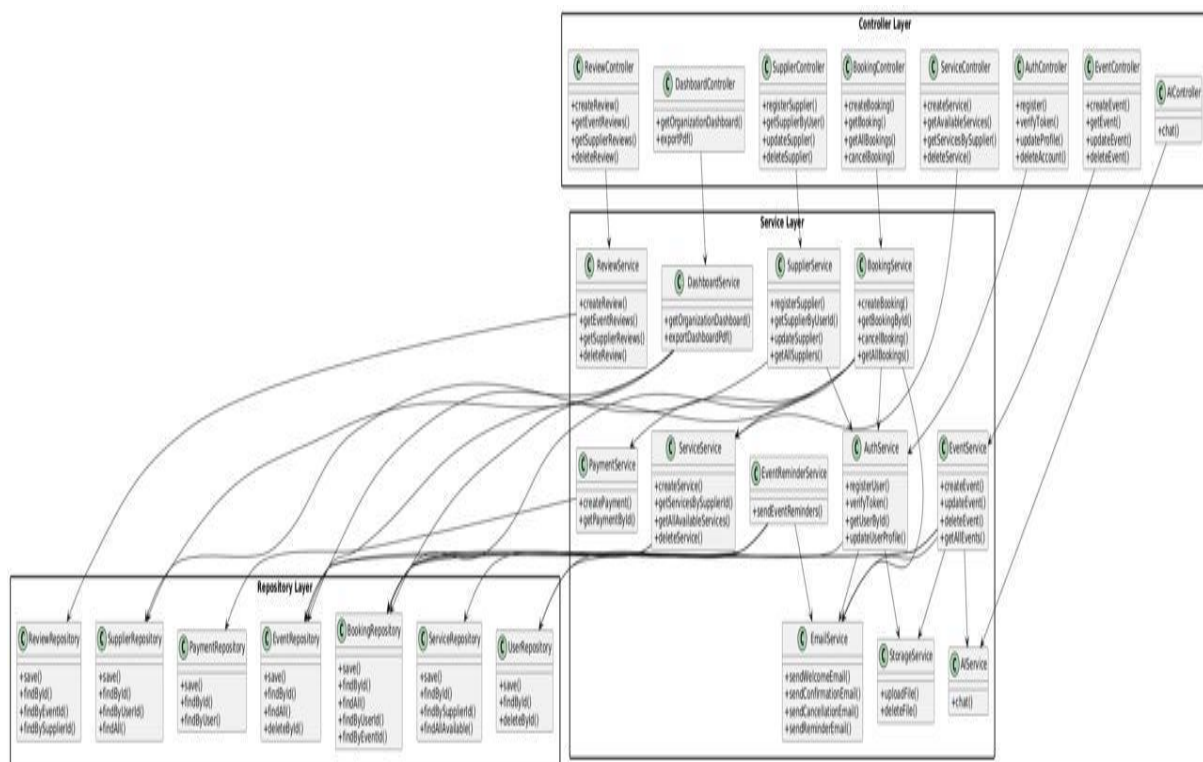


Figure 39: System Architecture Design

## 5-2- Database Design:

The system data model defines the core entities and their relationships that support the event management platform. The design ensures scalability, modularity, and clear separation of responsibilities.

### Entities Description:

#### 1. User

Represents a system user who can assume one of the following roles:

- USER
- ORGANIZER
- SUPPLIER

Each user maintains notification preferences stored as a key-value structure to control email notifications.

#### 2. Event

Represents an event created by an organizer (identified via organizer email).

Event types include:

- Public Event: Fixed-time, physical attendance
- Host Package: Flexible booking-based event

Each event maintains the following aggregated attributes:

- Total Bookings
- Total Revenue
- Available Seats.

#### 3. Booking

Represents a reservation made by a user for a specific event.

A booking may include additional services. The total price is calculated as:

Total Price = Ticket Price + Sum of Selected Services Prices

Each booking generates a unique reference number in the format: BK-YEAR-XXXXX

#### 4. Payment

Represents the payment associated with a booking.

- Relationship: One-to-One with Booking.com
- The system supports multiple payment methods (currently simulated without actual integration)

#### 5. Supplier

Represents a service provider in the system.

- Relates to a User in a One-to-One relationship
- Classified under a service category

A supplier can provide multiple services

## 6. Service

Represents a service offered by a supplier.

- Each Service is associated with a single Supplier
- Can be added to multiple bookings

## 7. Review

Represents user feedback

The review can be associated with:

- Event
- Supplier
- Or both

Relationships Description:

Organizer (User) creates Events (One-to-Many)

User makes Bookings (One-to-Many)

Booking belongs to Event (Many-to-One)

Booking has Payment (One-to-One)

Supplier provides Services (One-to-Many)

Booking includes Services (Many-to-Many)

Review targets Event and/or Supplier.

## 5-2-1- Backend Architecture Diagram:

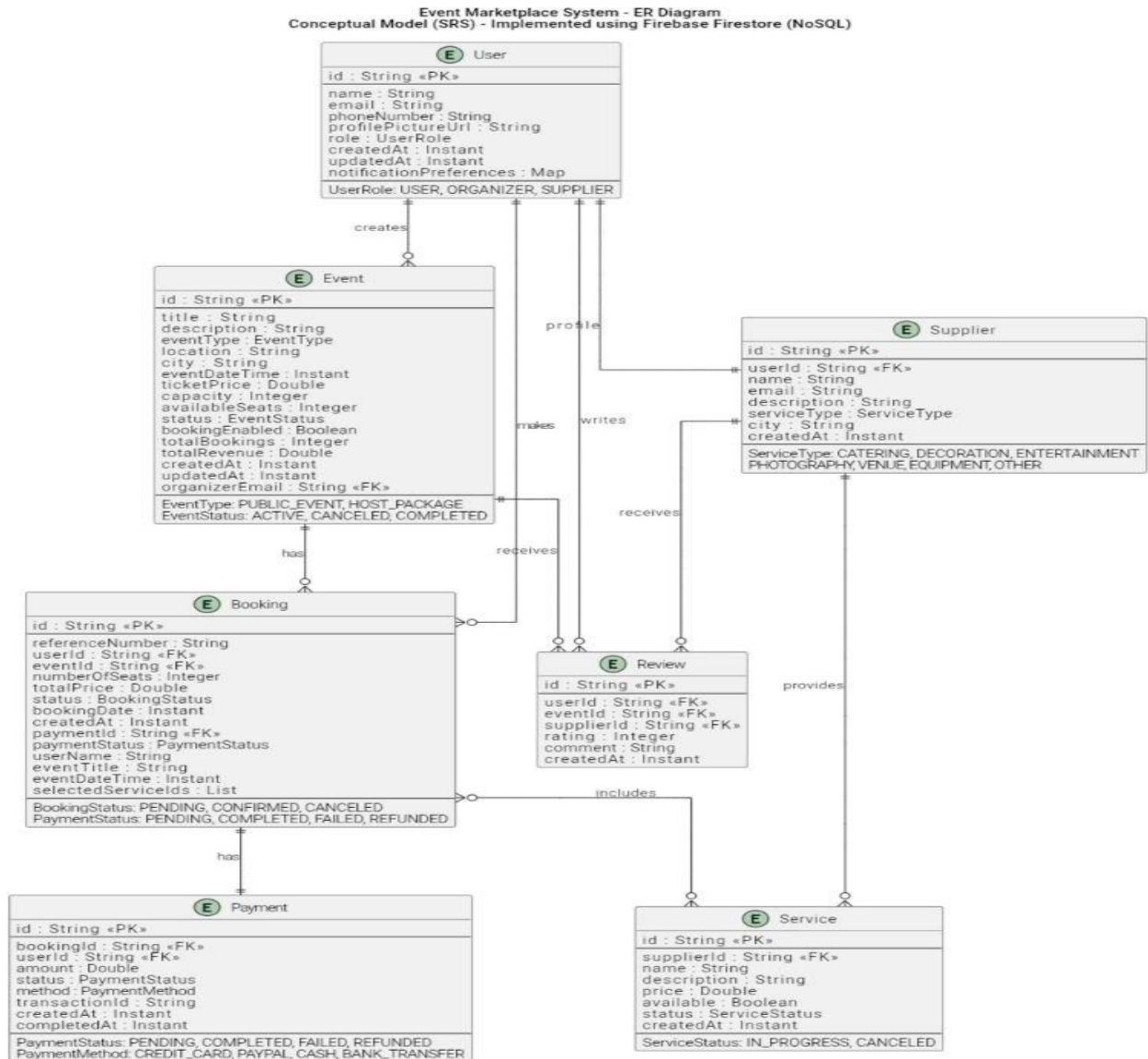


Figure 40:ER Diagram

This ER diagram represents a conceptual relational model used for system design. The actual implementation is based on a NoSQL document-oriented database (Firebase Firestore), where certain fields are denormalized to improve performance and scalability.

### 5-2-2- Collections& Documents:

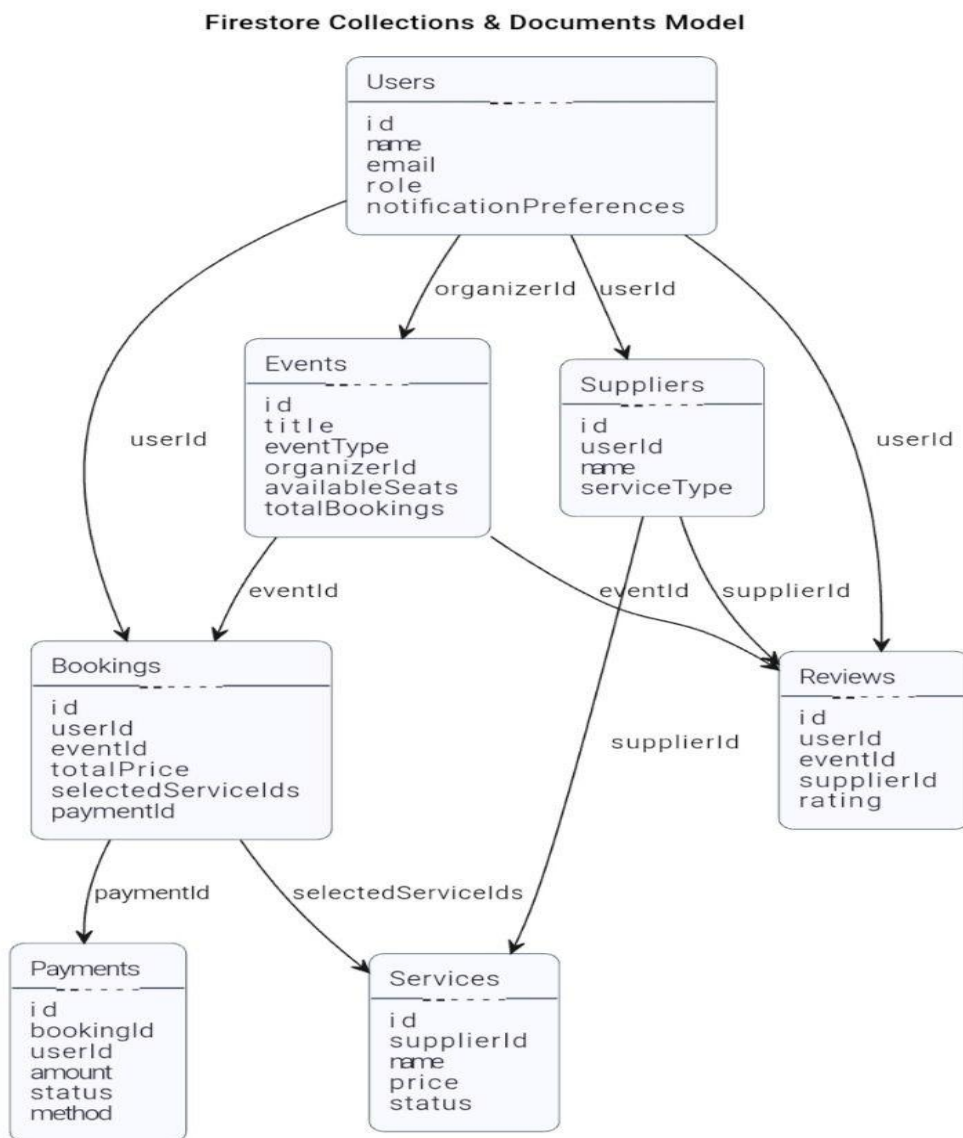


Figure 41: Collections& Documents

This section presents the logical structure of the system using Firebase Firestore collections and documents, where relationships are implemented using reference IDs instead of foreign keys.

### 5-2-3-Constraints and Keys:

The system enforces data integrity and consistency through relational database constraints applied across all core entities of the Event Management Platform. These constraints ensure correctness, avoid redundancy, and maintain referential integrity between tables.

The constraints are summarized in the table below:

| Constraint Type  | Entity       | Attribute(s)   | Description                                                                |
|------------------|--------------|----------------|----------------------------------------------------------------------------|
| Primary Key (PK) | User         | UserID         | Uniquely identifies each user in the system. Must be unique and not null.  |
| Primary Key (PK) | Event        | EventID        | Uniquely identifies each event.                                            |
| Primary Key (PK) | Booking      | BookingID      | Uniquely identifies each booking record.                                   |
| Primary Key (PK) | Supplier     | SupplierID     | Uniquely identifies each supplier.                                         |
| Primary Key (PK) | Notification | NotificationID | Uniquely identifies each notification.                                     |
| Foreign Key (FK) | Event        | UserID         | References User(UserID). Indicates the organizer who created the event.    |
| Foreign Key (FK) | Booking      | UserID         | References User(UserID). Indicates the user who made the booking.          |
| Foreign Key (FK) | Booking      | EventID        | References Event(EventID). Links booking to a specific event.              |
| Foreign Key (FK) | Notification | UserID         | References User(UserID). Indicates the recipient of the notification.      |
| Unique           | User         | Email          | Ensures that each user has a unique email address.                         |
| Unique           | User         | PhoneNumber    | Ensures no duplicate phone numbers exist (if enforced).                    |
| Unique           | Event        | Title, Date    | Composite constraint to prevent duplicate events with same title and date. |
| Check            | User         | Role           | Must be one of: USER, ORGANIZER, SUPPLIER.                                 |
| Check            | Event        | Capacity       | Must be greater than 0.                                                    |
| Check            | Booking      | Status         | Must be one of: PENDING, CONFIRMED, CANCELLED.                             |
| Check            | Event        | Date           | Must be a future or current valid date.                                    |

Table (71): Constraints and Keys



### 5-3- Designing external interfaces (APIs):

The system uses a RESTful API architecture to enable secure and efficient communication between the frontend and backend. These APIs define how different modules of the Event Marketplace interact with each other and external services. Each endpoint is clearly structured with its method, parameters, responses, and error codes to ensure consistency across all system operations. The following table summarizes the complete API .

| Module          | API                             | Path                                   | Method | Parameters                                           | Return Value              | Error Codes |
|-----------------|---------------------------------|----------------------------------------|--------|------------------------------------------------------|---------------------------|-------------|
| Authentication  | Register User                   | /auth/register                         | POST   | name, email, password, role                          | User + Token              | 400, 409    |
| Authentication  | Verify Token                    | /auth/verify                           | POST   | Authorization Bearer Token                           | Valid / Invalid           | 401         |
| Authentication  | Delete User                     | /auth/user/{uid}                       | DELETE | uid                                                  | Confirmation message      | 404         |
| Authentication  | Update Notification Preferences | /auth/user/{uid}/notifications         | PATCH  | uid, preferences                                     | Updated data              | 400         |
| Services        | Create Service                  | /services                              | POST   | serviceData                                          | Created service           | 400, 500    |
| Services        | Update Service                  | /services/{serviceld}                  | PUT    | serviceld, data                                      | Updated service           | 400, 404    |
| Services        | Delete Service                  | /services/{serviceld}                  | DELETE | serviceld                                            | Success message           | 404         |
| Services        | Change Service Status           | /services/{serviceld}/status           | PATCH  | status                                               | New status                | 400         |
| Services        | Available Services              | /services/available                    | GET    | —                                                    | List of services          | —           |
| Suppliers       | Create Supplier                 | /suppliers                             | POST   | supplierData                                         | New supplier              | —           |
| Suppliers       | Update Supplier                 | /suppliers/{id}                        | PUT    | id, data                                             | Updated supplier          | —           |
| Suppliers       | Delete Supplier                 | /suppliers/{id}                        | DELETE | id                                                   | Success message           | —           |
| Suppliers       | Get Supplier by User            | /suppliers/user/{id}                   | GET    | id                                                   | Supplier data             | —           |
| Bookings        | Create Booking                  | /bookings                              | POST   | bookingData                                          | Booking details           | —           |
| Bookings        | Upcoming Bookings               | /bookings/user/{userId}/upcoming       | GET    | userId                                               | List of bookings          | —           |
| Bookings        | Past Bookings                   | /bookings/user/{userId}/past           | GET    | userId                                               | List of bookings          | —           |
| Bookings        | Cancel Booking                  | /bookings/{bookingId}/cancel           | PUT    | userId (Query)                                       | Cancellation confirmation | —           |
| Bookings        | Event Booked Dates              | /bookings/event/{eventId}/booked-dates | GET    | eventId                                              | Reserved dates            | —           |
| AI              | Generate Event Plan             | /plan                                  | POST   | eventType, theme, audienceSize, budget, locationType | Suggested plan            | —           |
| AI              | AI Chat                         | /chat                                  | POST   | messages                                             | AI response               | —           |
| Reviews         | Add Review                      | /reviews                               | POST   | reviewData                                           | Review added              | —           |
| Reviews         | Supplier Reviews                | /reviews/supplier/{supplierId}         | GET    | supplierId                                           | List of reviews           | —           |
| Reviews         | Average Rating                  | /reviews/average/supplier/{supplierId} | GET    | supplierId                                           | Average rating            | —           |
| Recommendations | Get Recommendations             | /recommendations                       | GET    | userId (Query)                                       | Recommendation list       | —           |
| Media           | Upload Files                    | /[{id}]/media                          | POST   | FormData files                                       | File URLs                 | —           |
| Media           | Upload Video                    | /[{id}]/video                          | POST   | video file                                           | Video URL                 | —           |
| Export          | Export PDF                      | /export/pdf                            | GET    | —                                                    | PDF file                  | —           |

Table (72): APIs

## 5-4- Security & Access Control Design:

This section defines the system security mechanism and user permissions management within the Event Marketplace system. This includes how users are verified, how permissions are granted, and how user roles and access levels are defined. The design aims to ensure that each user can only access the functions and data authorized by their role within the system, thus achieving clear and effective data protection and system management.

| Component                | Description                                                                                                                                                                            |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication Mechanism | JWT (JSON Web Token) is adopted as the primary authentication mechanism. A token is issued upon successful login and is used to verify the user's identity in all subsequent requests. |
| Session Management       | The system operates in a stateless manner using JWT, eliminating the need to store sessions on the server side.                                                                        |
| Authorization Model      | The system is based on Role-Based Access Control (RBAC) to define and manage user permissions according to their assigned roles.                                                       |
| User Roles               | <b>Admin</b> (System Administrator) – <b>User</b> (Registered User) – <b>Guest</b> (Unregistered Visitor)                                                                              |
| Admin Privileges         | Full access including user management, event management, system monitoring, and complete access to all system data.                                                                    |
| User Privileges          | Ability to create and browse events, make bookings, manage personal profile, and receive notifications.                                                                                |
| Guest Privileges         | Limited access restricted to browsing events only without booking or interactive capabilities.                                                                                         |
| Token Verification       | JWT tokens are validated with every API request to secure system resources and prevent unauthorized access.                                                                            |
| Access Levels            | API access is controlled based on user roles using middleware that enforces authorization rules.                                                                                       |

Table (73): Security & Access Control Design

**Chapter 6 :**  
**Implementation Environment & Tools**

## Introduction

This chapter presents the complete technical environment used to design and implement the Event Management System. It documents the programming languages, frameworks, database platform, development tools, third-party integrations, version control workflow, deployment process, and testing environment.

The purpose of this chapter is twofold. First, it provides sufficient technical details to allow another developer to reproduce the system environment and run the project successfully. Second, it explains the rationale behind the major technical choices adopted during development.

## 6-1 Programming Languages and Frameworks

The system follows a full-stack architecture consisting of two independent layers:

- Frontend layer: responsible for user interaction and presentation.
- Backend layer: responsible for business logic, security, data processing, and API services.

Communication between both layers is achieved through RESTful APIs, allowing each side to be developed, tested, and deployed independently.

### - Backend: Java 21 and Spring Boot

Java 21 was selected as the backend programming language because it is a Long-Term Support (LTS) release that provides stability, security updates, and modern language features. Java also offers strong type checking, object-oriented design, and high maintainability for medium and large software projects.

Spring Boot was selected as the backend framework because it simplifies enterprise application development through:

- Auto-configuration
- Embedded web server support
- Dependency management through starters
- Rapid creation of REST APIs
- Strong security and testing ecosystem

The backend server can be started using the following command:

```
./mvnw spring-boot:run
```

The application runs by default on port 8080.

## - Frontend: JavaScript and React

JavaScript was used for frontend development due to its native browser support and compatibility with modern web frameworks.

React was selected as the frontend framework because of its component-based architecture, efficient rendering model, and maintainability. The use of reusable components reduced duplicated code and accelerated interface development.

React also supports Single Page Application (SPA) behavior, which improves navigation speed by reducing unnecessary page reloads.

The frontend server can be started using:

```
npm start
```

The development environment runs on port 3000 and supports hot reloading.

## 6-2 Development Environment and Project Structure

### - Development Tools

The project was developed by a three-member team using both Windows and macOS environments. The system operated consistently across all platforms due to the cross-platform support provided by Java, Node.js, and modern web technologies.

The following tools were used during development:

| Tool               | Purpose                           |
|--------------------|-----------------------------------|
| IntelliJ IDEA      | Backend development and debugging |
| Visual Studio Code | Frontend development              |
| Maven Wrapper      | Backend build management          |
| npm                | Frontend package management       |
| Git                | Version control                   |
| postman            | API testing                       |

*Table (74): Development Tools*

## - Backend Project Structure

responsibilities. The backend follows a layered architecture with clear separation of

| Path                             | Responsibility                      |
|----------------------------------|-------------------------------------|
| controllers/                     | Handles HTTP requests and responses |
| services/                        | Contains business logic             |
| repositories/                    | Data access layer                   |
| dto/                             | Data Transfer Objects               |
| models                           | Entity definitions                  |
| resources/application.properties | Configuration settings              |

Table (75): Backend Project Structure

## -Frontend Project Structure

| Path        | Responsibility          |
|-------------|-------------------------|
| components/ | Reusable UI components  |
| pages/      | Main application pages  |
| services/   | API communication layer |
| auth/       | Authentication logic    |
| layouts/    | Shared layout templates |
| components/ | Reusable UI components  |

Table (76): frontend Project Structure

## 6-3 Libraries and Dependencies

All libraries are centrally managed to ensure consistent builds across all environments.

- Backend dependencies are defined in pom.xml using Maven.
- Frontend dependencies are defined in package.json using npm.

## - Backend Dependencies

| Library                  | Role                                |
|--------------------------|-------------------------------------|
| Spring Boot Starter Web  | REST APIs and embedded server       |
| Spring Boot Starter Mail | Email integration                   |
| Firebase Admin SDK       | Firestore and token verification    |
| iText PDF                | PDF report generation               |
| Lombok                   | Reduces repetitive boilerplate code |
| Jackson                  | JSON serialization/deserialization  |

Table (77): Backend Dependencies

### -Frontend Dependencies

| Library           | Role                             |
|-------------------|----------------------------------|
| React / React DOM | UI rendering                     |
| React Router DOM  | Client-side routing              |
| Axios             | HTTP requests                    |
| PrimeReact        | UI components                    |
| Chart.js          | Data visualization               |
| Firebase SDK      | Client-side Firebase integration |

*Table (78): frontend Dependencies*

## 6-4 Database System

The system uses Firebase Firestore as the primary database platform. Firestore is a cloud-hosted NoSQL document database that offers scalability, flexibility, and managed infrastructure.

Firestore was selected because the project required rapid development, evolving data structures, and simplified deployment.

Main Advantages of Firestore

- Schema flexibility
- Automatic scaling
- Cloud hosting without server administration
- Real-time update support
- Native integration with Firebase Authentication and Storage

### -Main Collections

The database is organized into the following collections:

- events
- bookings
- users

- suppliers
- services
- reviews
- payments

Each document contains a unique generated identifier.

#### **-Firestore vs Relational Databases**

| <b>Criterion</b>     | <b>Relational DB</b>               | <b>Firestore</b> |
|----------------------|------------------------------------|------------------|
| Schema               | Fixed schema                       | Flexible schema  |
| Scaling              | Manual planning                    | Automatic        |
| Infrastructure       | Requires server management         | Fully managed    |
| Real-time updates    | Additional implementation required | Built-in         |
| Firebase integration | External                           | Native           |

*Table (79): Firestore vs Relational Databases*

## **6-5 External Service Integrations**

To reduce development complexity and focus on business requirements, the system integrates with trusted external services.

#### **-Firebase Services**

**Three Firebase services are used:**

- Firebase Authentication for user login and token management
- Firestore as the main database
- Firebase Storage for images and uploaded media files

#### **-Mailgun SMTP**

Mailgun is used for transactional email delivery, including:

- Booking confirmations
- Notifications
- Event updates
- Cancellation messages



Emails are sent asynchronously to avoid delaying API responses.

### **-OpenRouter API**

**OpenRouter was integrated to provide AI-powered features such as:**

- User support chatbot
- Event recommendations
- Planning assistance for organizers

API credentials are stored on the backend for security purposes.

### **-Version Control**

Git was used throughout the project for source code management.

A feature-branch workflow was followed:

1. Create a branch for each feature or fix
2. Implement changes independently
3. Merge approved changes into the main branch

Meaningful commit messages were used to improve project traceability.

### **Ignored Files**

The following files and folders were excluded using .gitignore:

- target/
- build/
- node\_modules/
- .idea/
- .vscode/
- .DS\_Store

## 6-7 Build Process and Deployment

### - Backend Build

The backend is packaged using:

```
./mvnw clean package
```

This command generates an executable JAR file containing all required dependencies.

### - Frontend Build

The frontend production build is generated using:

```
npm run build
```

This creates optimized static files ready for deployment.

### - Deployment Options

The system can be deployed using common hosting platforms such as:

- Firebase Hosting
- Netlify
- Vercel
- AWS
- Google Cloud
- Azure

### - Recommended Production Improvements

For future production deployment, the following enhancements are recommended:

- CI/CD pipeline using GitHub Actions or Jenkins
- Environment variables for secrets
- Separate staging and production environments
- Monitoring and centralized logging

## 6-8 Testing Environment

Testing was performed manually throughout development to validate complete user workflows and system integration.

### Testing Levels

## Functional Testing

Verification that each feature behaves according to requirements.

## Integration Testing

Validation of complete data flow:

Frontend → Backend API → Firestore → Response

## User Interface Testing

### Checking:

- Layout consistency
- Responsiveness
- Component behavior

## API Testing

Postman was used to test endpoints individually, including:

- Request payloads
- Response structures
- Authentication rules
- Authorization behavior

## Future Automated Testing

The project environment also supports automated testing:

| Layer    | Tools                       |
|----------|-----------------------------|
| Backend  | JUnit, Mockito, Spring Test |
| Frontend | Jest, React Testing Library |

*Table (80): Layer and Tools*

**Expanding automated test coverage is recommended in future work.**

## 6-9 Security Considerations

Security was implemented at multiple levels.

### **-Authentication Security**

Firebase Authentication issues secure JWT tokens after login. The backend validates every token before processing protected requests.

### **-Access Control**

Firestore security rules restrict read/write operations according to user permissions.

### **-Input Validation**

Both frontend and backend validate incoming data to prevent malformed requests.

### **-Configuration Security**

Sensitive values such as API keys and credentials should be stored in environment variables in production environments.

## **6-10 Performance Considerations**

**Several performance optimizations were considered:**

- Asynchronous email sending
- Efficient backend resource management
- Frontend production bundling and minification
- Lazy loading where appropriate
- CDN delivery for static assets

**These measures improve responsiveness and scalability.**

## **6-11 Lessons Learned and Future Enhancements**

### **-Lessons Learned**

The development process highlighted several practical insights:

- Clear separation between frontend and backend improves maintainability
- Component-based frontend design accelerates development
- Managed cloud services reduce infrastructure overhead
- Strong frameworks significantly improve productivity

### **-Future Enhancements**

- Full CI/CD automation
- Swagger / OpenAPI documentation
- Redis caching layer
- Payment gateway integration
- Real-time notifications
- Mobile application using React Native
- Expanded automated testing coverage

## **Chapter Summary**

This chapter presented the implementation environment and technologies used in the Event Management System. The system combines Java and Spring Boot for backend development, React for the frontend, and Firebase services for authentication, storage, and database management. Additional integrations such as Mailgun and OpenRouter were used to support communication and AI features. Overall, the selected technologies provide a scalable and maintainable system architecture.

## **Chapter 7:**

# **Software Implementation**

## Introduction:

Objective: This chapter presents the software implementation of the EventMarketPlace system. It focuses on the overall project structure, explains the implementation of each main module, and provides representative code examples. Full code listings are included in the appendix.

## 7-1 Project File Structure

The project is organized following a layered architecture to ensure scalability, maintainability, and separation of concerns:

```
EventMarketPlace/
├── backend/
│   ├── src/main/java/com/eventmarketplace/
│   │   ├── controller/    # Presentation Layer: REST Controllers
│   │   ├── service/       # Business Logic Layer: Services
│   │   ├── repository/    # Data Access Layer: Repositories
│   │   ├── model/         # Domain Models (Entities)
│   │   ├── dto/           # Data Transfer Objects
│   │   └── config/        # Configuration Classes
│   └── frontend/
│       ├── src/
│       │   ├── components/ # Reusable React Components
│       │   ├── pages/      # Main Pages
│       │   ├── services/   # API Service Modules
│       │   ├── context/    # State and Auth Management
│       │   └── App.js      # Entry Point
│       └── docs/          # Documentation
├── docs/                 # Documentation
└── tests/                 # Unit and Integration Tests
```

Notes:

- Each layer has a clear responsibility.
- This structure improves maintainability and simplifies testing.

## 7-2 Modules Implementation

The system is divided into seven main modules, each responsible for a specific area of functionality:

| Module               | Main Functionality                           |
|----------------------|----------------------------------------------|
| Events Module        | Manage events (CRUD, updates, notifications) |
| Users Module         | User accounts, roles, authentication         |
| Bookings Module      | User bookings, event capacity management     |
| Dashboard Module     | Statistics, analytics, and charts            |
| AI Assistant Module  | Smart recommendations and conversational AI  |
| Notifications Module | Email notifications and alerts               |
| Reports Module       | PDF generation and data analysis             |

*Table (81): Modules Implementation*

### Example: Event Service (simplified)

```
@Service
public class EventService {
    private final EventRepository eventRepository;

    public Event createEvent(Event event) {
        event.setStatus(Event.EventStatus.ACTIVE);
        event.setCreatedAt(Instant.now());
        return eventRepository.save(event);
    }
}
```

This snippet shows how the service layer handles business logic and interacts with the data layer.



## 7-3 API Implementation

The backend exposes RESTful APIs using Spring Boot to communicate with the frontend.

### Example: Event Controller

```
@RestController
@RequestMapping("/api/events")
public class EventController {
    private final EventService eventService;

    @PostMapping
    public ResponseEntity<?> createEvent(@RequestBody Event event) {
        try {
            Event created = eventService.createEvent(event);
            return ResponseEntity.ok(created);
        } catch (Exception e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }
}
```

### Notes:

- Each module has its own controller.
- Error handling ensures that the API returns meaningful responses.
- CORS configuration allows requests from any frontend origin.

## 7-4 Data Access Layer Implementation

The repository pattern abstracts the database access. Firebase Firestore is used as the database.

### Example: Event Repository

```
@Repository
public class EventRepository extends BaseFirestoreRepository<Event> {
```

```

public List<Event> findAll() {
    return firestore.collection("events")
        .get().get().getDocuments().stream()
        .map(doc -> doc.toObject(Event.class))
        .collect(Collectors.toList());
}
}

```

**Notes:**

- The repository handles all CRUD operations.
- Changing the database would not require modifications in the business layer.

## 7-5 Business Logic Layer Implementation

The business layer enforces rules, calculations, and conditions. For instance, checking event capacity before booking:

```

public Booking createBooking(Booking booking) {
    return firestore.runTransaction(transaction -> {
        Event event =
transaction.get(eventRef(booking.getEventId())).toObject(Event.class);
        if (event.getCapacity() <= event.getBookings().size()) {
            throw new IllegalStateException("Event is fully booked");
        }
        return transaction.set(bookingRef(booking.getId()), booking).get();
    });
}

```

**Notes:**

- Transactions prevent race conditions in concurrent bookings.
- Services can integrate with notifications or other services for side effects.

## 7-6 UI Implementation

The frontend is built with React and PrimeReact, following a component-based architecture.

**Example: Events List Component**

```

export const EventsList = () => {
  const [events, setEvents] = useState([]);

  useEffect(() => {
    eventService.getAllEvents().then(setEvents);
  }, []);

  return <EventGrid events={events} />;
};

```

### Notes:

- State is managed using React Hooks.
- Components like EventGrid and Spinner are reusable.
- Protected routes and role-based access are implemented via context API.

### User interfaces:

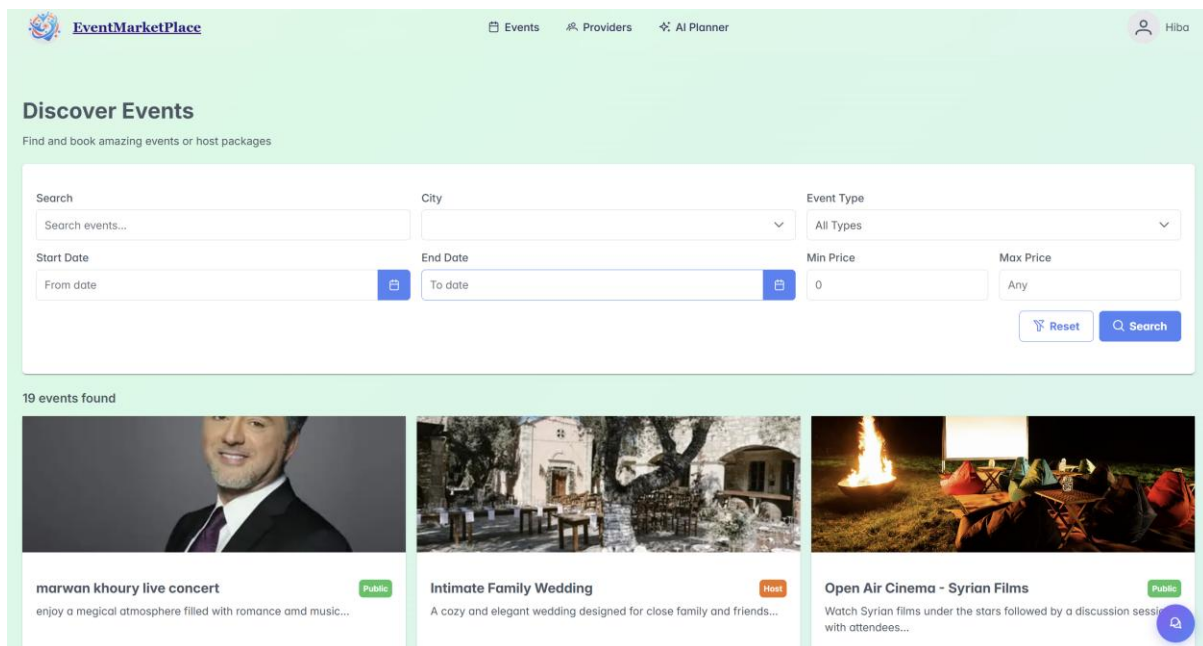


Figure 42: Discover Events I



### Damascus Evening Music Night

Public Event ACTIVE

Booking Open

#### About This Event

Enjoy a traditional Syrian music night in the heart of Old Damascus with live performances and an authentic cultural atmosphere

#### Event Details

|                                                           |                                                       |
|-----------------------------------------------------------|-------------------------------------------------------|
| <p> Date</p> <p>Saturday, May 30, 2026</p>                | <p> Time</p> <p>8:00 PM</p>                           |
| <p> Location</p> <p>Old Damascus, Bab Touma, Damascus</p> | <p> Availability</p> <p>242 / 250 spots available</p> |
| <p> Total Bookings</p> <p>8 bookings</p>                  |                                                       |

#### Reviews

Ticket Price

**\$10.00**

Go to Checkout

Contact Organizer

Free cancellation up to 24 hours before the event

Figure 43:Discover Events2

### Etoile wedding hall

Host Package ACTIVE

Booking Open

#### About This Event

A luxurious wedding venue designed to deliver an elegant and unforgettable celebration experience. The hall features modern interiors, sophisticated lighting, and spacious seating arrangements suitable for weddings and large events.

#### Event Details

|                                                    |                                                   |
|----------------------------------------------------|---------------------------------------------------|
| <p> Available From</p> <p>Friday, May 29, 2026</p> | <p> Available From (Time)</p> <p>8:00 PM</p>      |
| <p> Location</p> <p>Al-Adawi, Damascus</p>         | <p> Availability</p> <p>1 / 1 spots available</p> |

#### Event Video



0:02 / 0:09

Package Price

**\$0.00**

Base package price

Go to Checkout

Contact Organizer

Free cancellation up to 24 hours before the event

Figure 44:Discover Events3

Event Details



Luxury Palace Wedding

Available from: ٢٠٢٦/٥/١

Dama Rose Hotel, Damascus

Select Your Date & Time

Choose your preferred date and time. Dates already booked are unavailable.

<

May 2026

>

| Su | Mo | Tu | We | Th | Fr | Sa |
|----|----|----|----|----|----|----|
| 26 | 27 | 28 | 29 | 30 | 1  | 2  |
| 3  | 4  | 5  | 6  | 7  | 8  | 9  |
| 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 17 | 18 | 19 | 20 | 21 | 22 | 23 |
| 24 | 25 | 26 | 27 | 28 | 29 | 30 |
| 31 | 1  | 2  | 3  | 4  | 5  | 6  |

08

:

30

:

PM

Package Details

Number of Packages

1

Price Per Package

\$10000.00

Booking Summary

Event

Luxury Palace Wedding

Packages

1

Price per package

\$10000.00

Total

\$10000.00

Free cancellation up to 24 hours before the event

Figure 45:Discover Events4

Package Details

Number of Packages

1

Price Per Package

\$10000.00

Your Information

Full Name

Hiba

Email

hiba.shawa2014@gmail.com

Phone Number

+963993641015

Add Provider Services

Optional services you can add to your package

Catering

☐ Wedding Catering  
Full-service catering tailored for weddings.
 \$555.00

☐ Video Shooting  
Cinematic videos with modern equipment and storytelling style.
 \$40.00

☐ Buffet Setup  
Well-organized buffet with a variety of dishes.
 \$75.00

☐ Desserts  
Italian Desserts
 \$500.00

☐ Beverages  
All kind of Drinks
 \$800.00

Booking Summary

Event

Luxury Palace Wedding

Packages

1

Price per package

\$10000.00

Total

\$10000.00

Free cancellation up to 24 hours before the event

Figure 46:Discover Events5

169

Payment Method

Credit Card

Credit Card

PayPal

Cash

Bank Transfer

Expiry Date

MM/YY

CVV

....

⚠ This is a demo payment form. No real transaction will be processed.

🔔 Use card number 4000 0000 0000 0002 to test a declined payment.

Total Amount

\$10000.00

✓

Confirm Booking

Packages

1

Price per package

\$10000.00

Total

\$10000.00

🕒 Free cancellation up to 24 hours before the event

© 2026 EventMarketPlace. All rights reserved.

Figure 47:Discover Events6

Payment Method

Credit Card

Card Holder Name

John Doe

Card Number

1234 5678 9012 3456

Expiry Date

MM/YY

CVV

....

⚠ This is a demo payment form. No real transaction will be processed.

🔔 Use card number 4000 0000 0000 0002 to test a declined payment.

Total Amount

\$10000.00

✓

Confirm Booking

Figure 48:Discover Events7

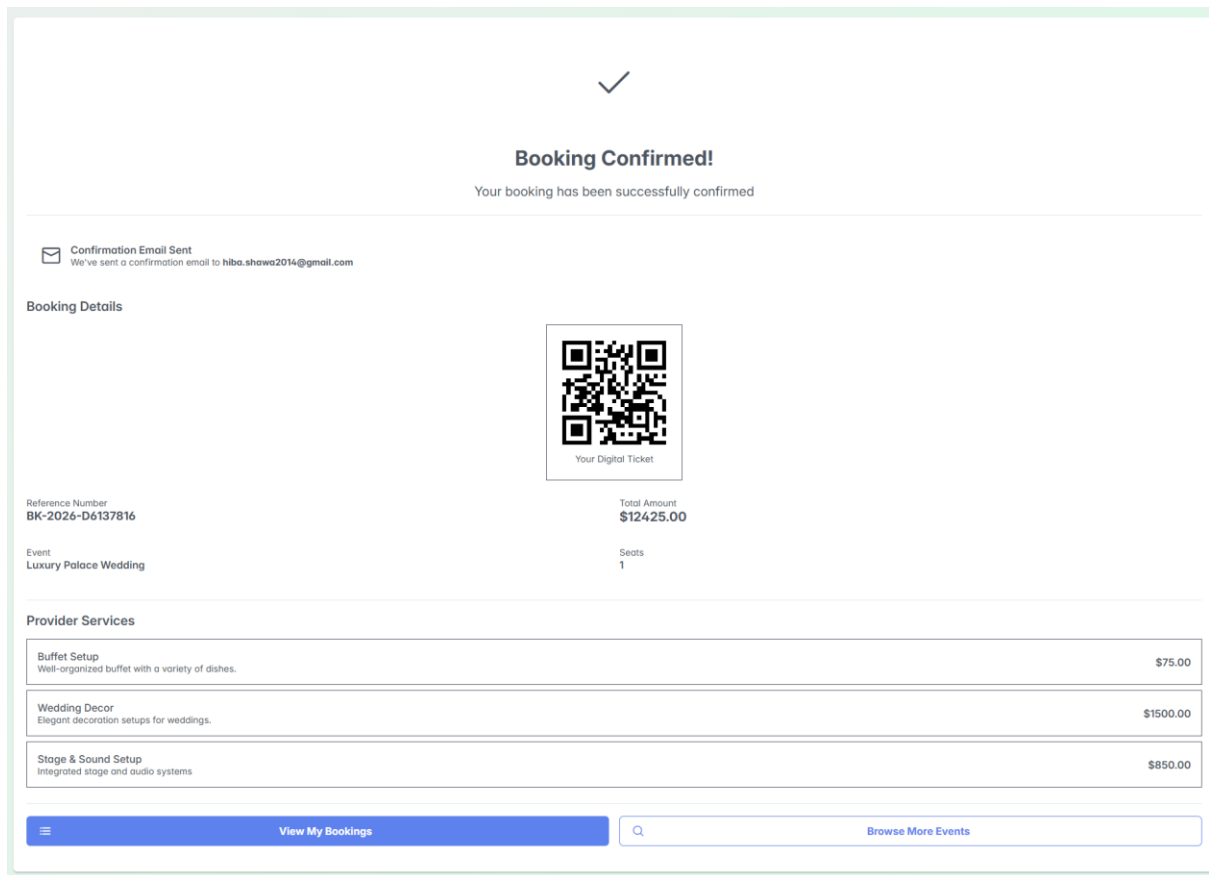


Figure 49:Discover Events8

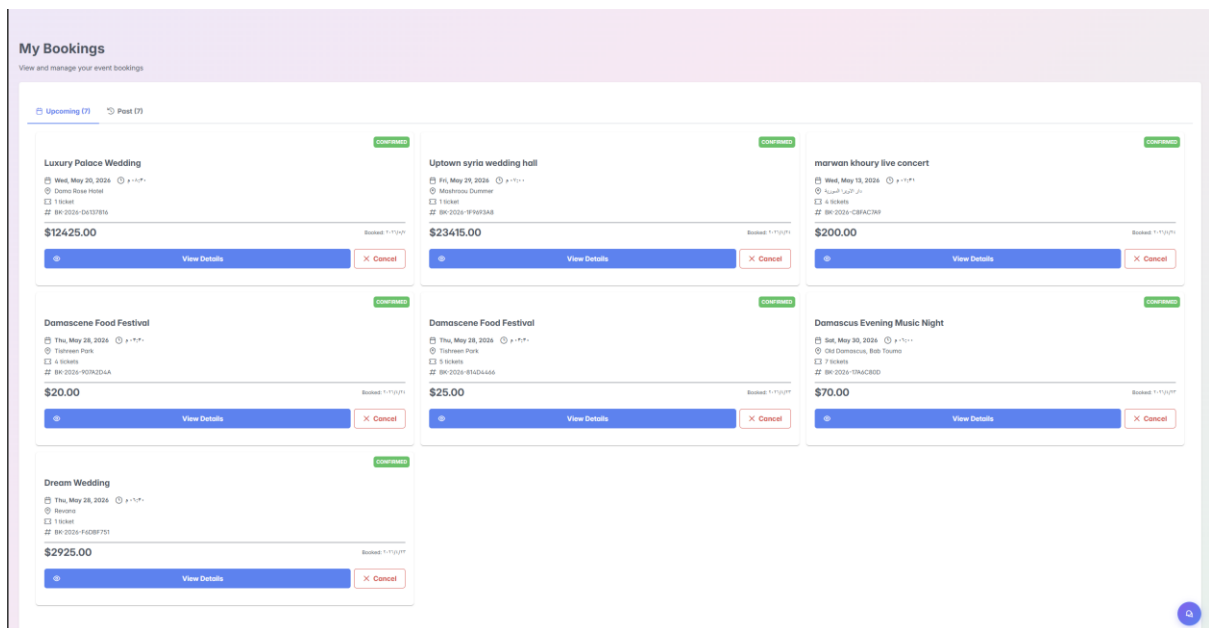


Figure 50:My Bookings

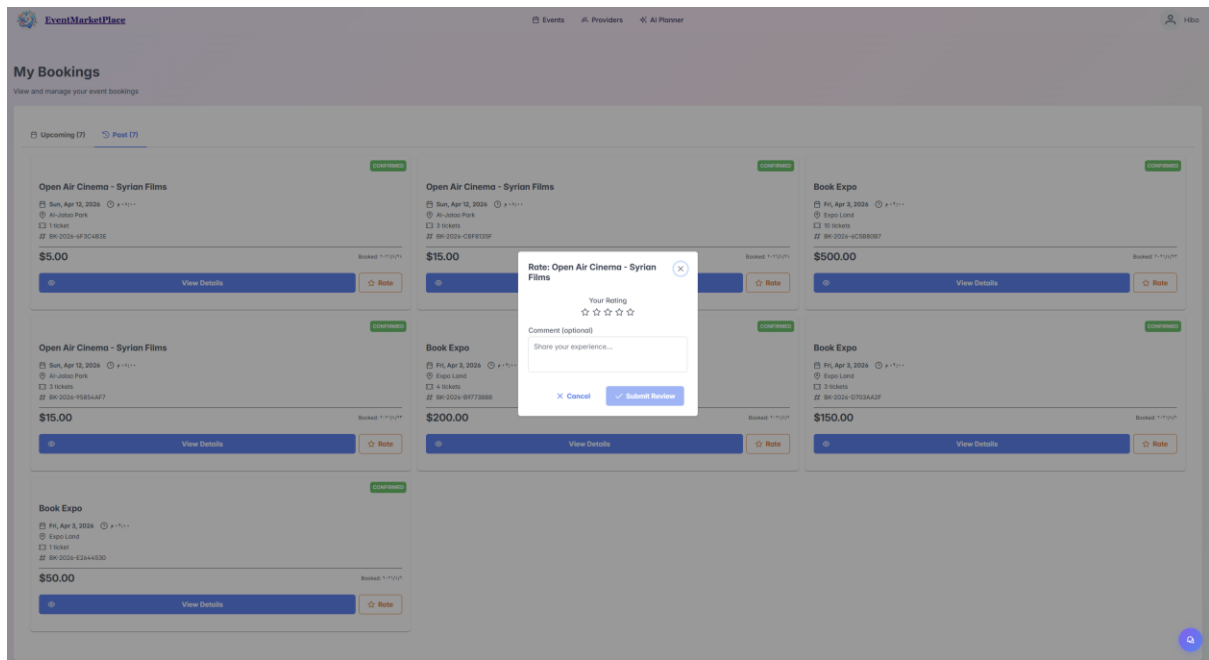


Figure 51: My Booking 2 Interface

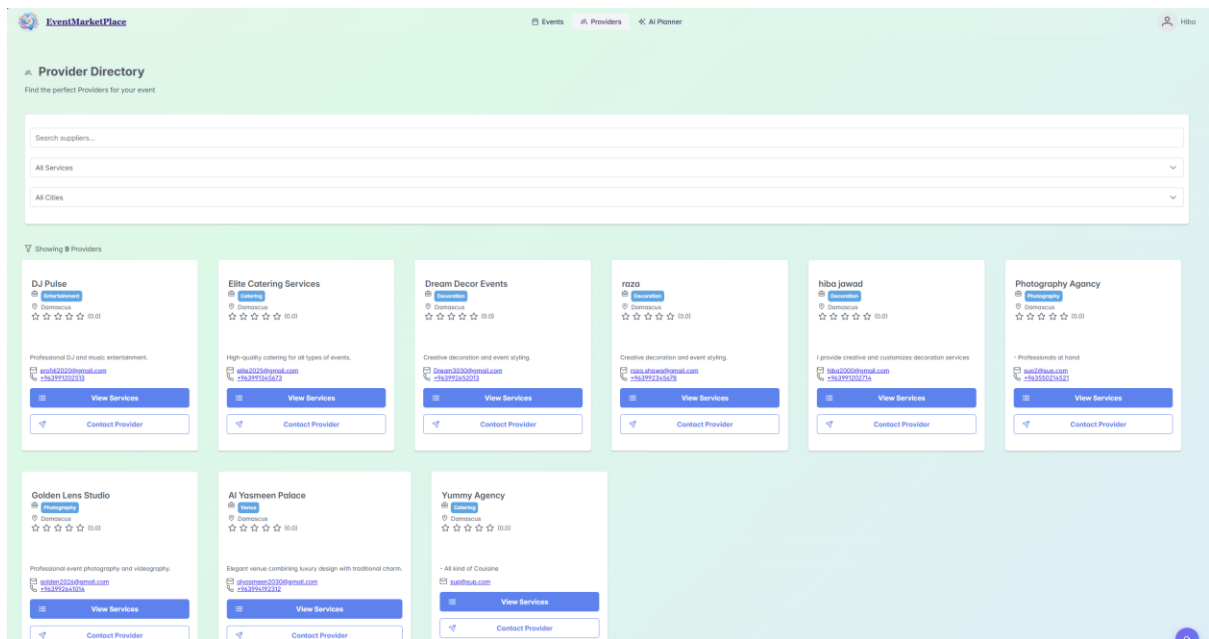


Figure 52: Provider Directory Interface



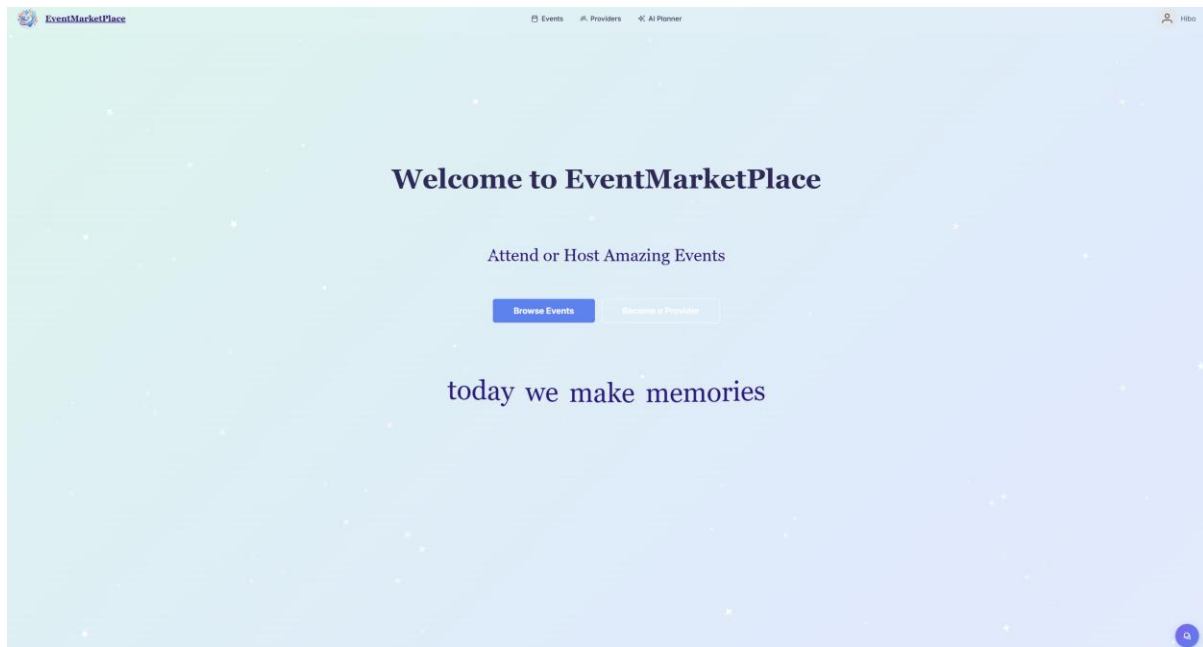


Figure 53: Welcome To EventMarketPLace Interface

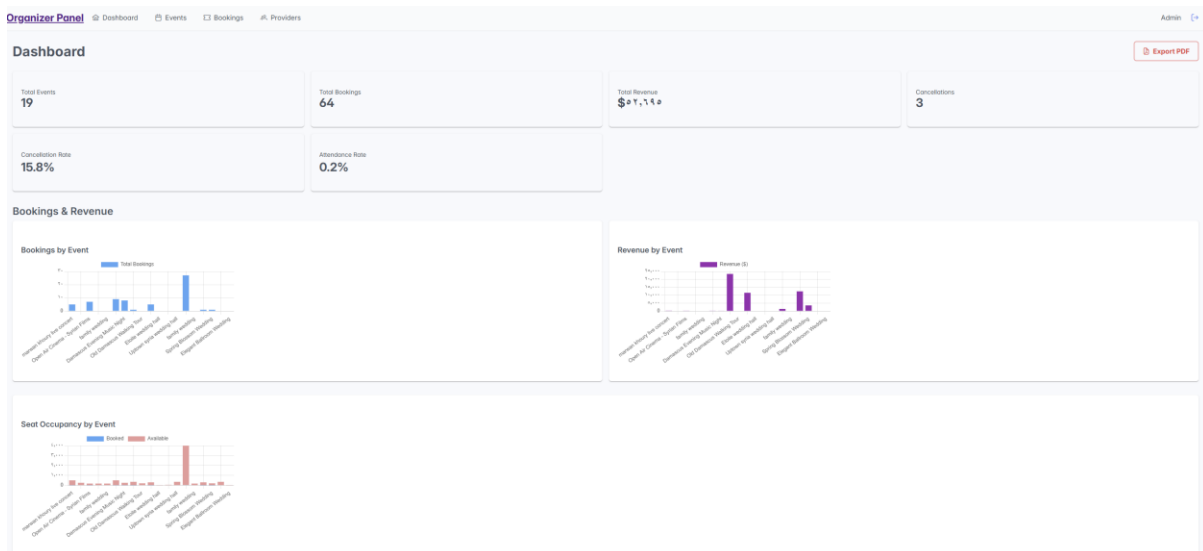


Figure 54: Dashboard Organizer Panel Interface

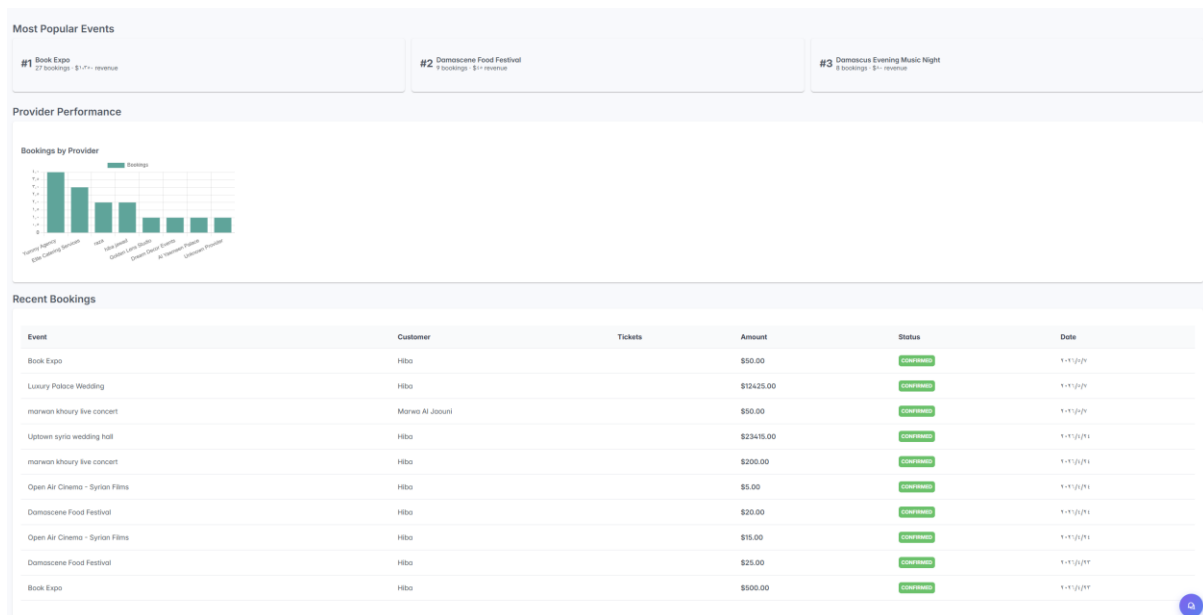


Figure 55: Recent Bookings Interface

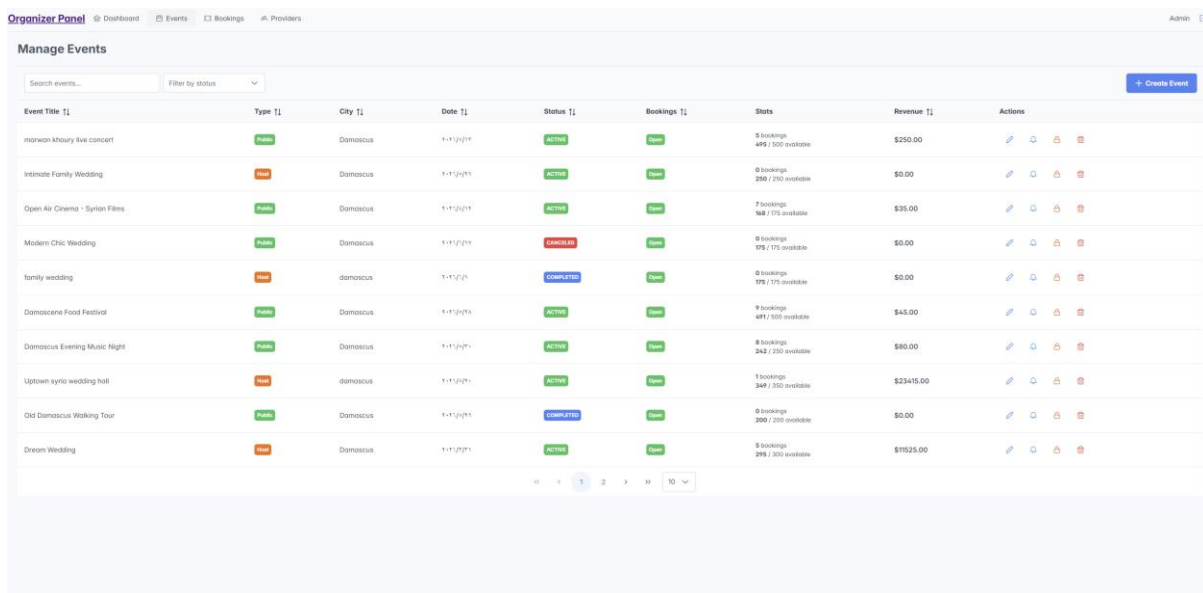


Figure 56: Manage Events Organizer Panel Interface

| Organizer Panel                     |                            |                 |            |           |           |            |
|-------------------------------------|----------------------------|-----------------|------------|-----------|-----------|------------|
| Dashboard Bookings Providers        |                            |                 |            |           |           |            |
| Bookings Management                 |                            |                 |            |           |           |            |
| Total Revenue \$40115.00            |                            |                 |            |           |           |            |
| Search bookings... Filter by status |                            |                 |            |           |           |            |
| Reference                           | Event                      | Customer        | Amount     | Status    | Booked On | Event Date |
| BK-2026-A3420029                    | Spring Blossom Wedding     | Hiba            | \$3570.00  | CANCELLED | ٢٠٢٦/١/٢٢ | ٢٠٢٦/٥/١٤  |
| BK-2026-814D4466                    | Damascene Food Festival    | Hiba            | \$25.00    | CONFIRMED | ٢٠٢٦/١/٢٣ | ٢٠٢٦/٥/٢٨  |
| BK-2026-E2644530                    | Book Expo                  | Hiba            | \$50.00    | CONFIRMED | ٢٠٢٦/١/٢٩ | ٢٠٢٦/١/٣٠  |
| BK-2026-1F9693A8                    | Uptown syria wedding hall  | Hiba            | \$23415.00 | CONFIRMED | ٢٠٢٦/١/٢٩ | ٢٠٢٦/٥/٢٩  |
| BK-2026-F6DBF751                    | Dream Wedding              | Hiba            | \$2925.00  | CONFIRMED | ٢٠٢٦/١/٢٣ | ٢٠٢٦/٥/٢٨  |
| BK-2026-BA68C325                    | Book Expo                  | Hiba            | \$50.00    | CONFIRMED | ٢٠٢٦/٥/٢٧ | ٢٠٢٦/١/٣٠  |
| BK-2026-907A2D4A                    | Damascene Food Festival    | Hiba            | \$20.00    | CONFIRMED | ٢٠٢٦/١/٢٩ | ٢٠٢٦/٥/٢٨  |
| BK-2026-6CS880B7                    | Book Expo                  | Hiba            | \$500.00   | CONFIRMED | ٢٠٢٦/١/٢٣ | ٢٠٢٦/١/٣٠  |
| BK-2026-D703AA2F                    | Book Expo                  | Hiba            | \$150.00   | CONFIRMED | ٢٠٢٦/١/٢٩ | ٢٠٢٦/١/٣٠  |
| BK-2026-SCA0DB5F                    | marwan khouri live concert | Marwa Al Jaouni | \$50.00    | CONFIRMED | ٢٠٢٦/٥/٢٧ | ٢٠٢٦/٥/٢٣  |

[Organizer Panel](#)
[Dashboard](#)
[Events](#)
[Bookings](#)
[Providers](#)

Admin

## Provider Directory

Find the perfect Providers for your event

Decoration

All Cities

Showing 3 Providers

Dream Decor Events

Decoration

Damascus

☆☆☆☆☆ (0.0)

Creative decoration and event styling.

[Dream3030@gmail.com](mailto:Dream3030@gmail.com)  
 +963992652013

View Services

Contact Provider

raza

Decoration

Damascus

☆☆☆☆☆ (0.0)

Creative decoration and event styling.

[raza.shawla@gmail.com](mailto:raza.shawla@gmail.com)  
 +9639992345678

View Services

Contact Provider

hiba jawad

Decoration

Damascus

☆☆☆☆☆ (0.0)

I provide creative and customizes decoration services

[hiba2008@gmail.com](mailto:hiba2008@gmail.com)  
 +9639921202715

View Services

Contact Provider



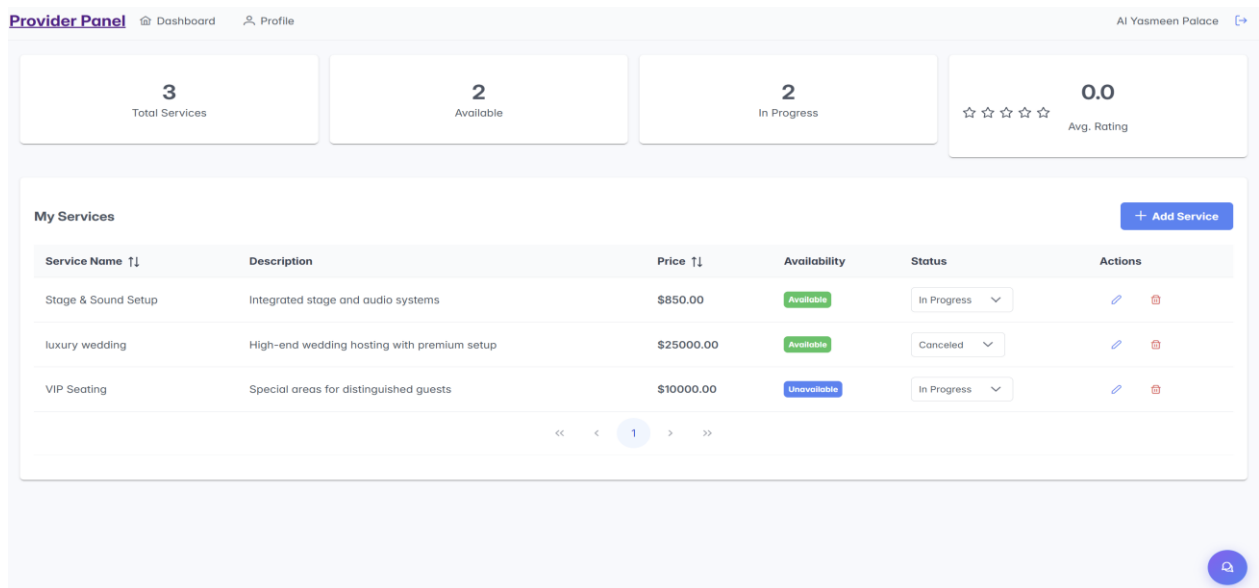


Figure 61: Provider Panel Interface

## Add New Service



Service Name \*

e.g. Wedding Photography

Description

Describe your service...

Price (\$)

0.00

Status

In Progress



✓ Available

✕ Cancel

✓ Save

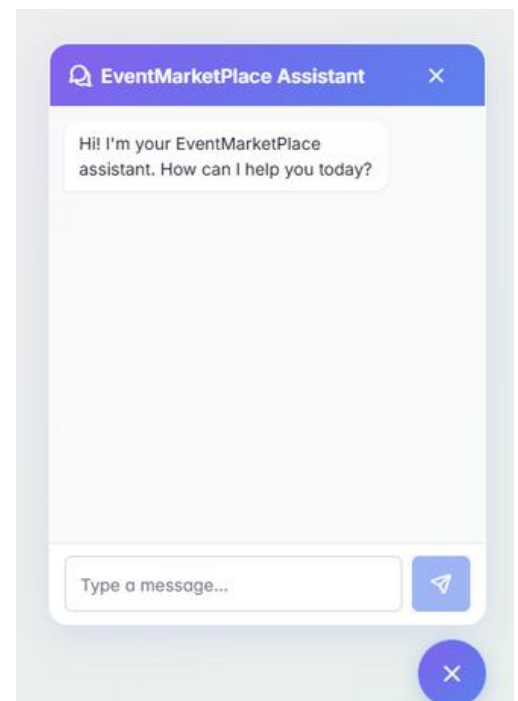


Figure 62: EventMarketPlace Assistant Interface

Figure 63: Add New Service Interface

**EventMarketPlace** Events Providers AI Planner Hiba

### AI Event Planner

Tell us what you're looking for and we'll recommend the best matching events from our catalogue.

|                                                                            |                                                                  |
|----------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>Event Type</b><br>Any type                                              | <b>Expected Guests</b><br>e.g. 100                               |
| <b>Budget Range</b><br>Any budget                                          | <b>Preferred Location</b><br>e.g. New York, outdoor, downtown... |
| <b>Short Description</b><br>e.g. A relaxed corporate networking evening... |                                                                  |

**Find Matching Events**

Figure 64: AI Event Planner Interface

## 7-7 Smart Algorithms and ML Implementation

The system includes AI-powered features for recommendations and conversational interactions:

### Example: AI-based Event Recommendations

```
public List<Event> getRecommendedEvents(User user) {
    String prompt = buildPrompt(user.getPreferences());
    String aiResponse = openRouterService.getAIResponse(prompt);
    return parseEventsFromAI(aiResponse);
}
```

### Notes:

- Text processing and AI response parsing extract structured recommendations.
- Models can be external APIs (OpenRouter) or internal ML models.
- Recommendations are verified before display to the user.

## 7.8 Summary

This chapter presents the EventMarketPlace software implementation, highlighting a layered architecture, key modules (events, users, bookings, dashboard, AI, notifications, reports), and technologies (Spring Boot, React, Firebase, OpenRouter API, PrimeReact, Chart.js, iText PDF). It also features AI recommendations, chat assistance, real-time notifications, and analytics. Full code is in the appendix. The chapter demonstrates clean, professional full-stack development.

## **Chapter 8:**

### **Testing & Evaluation**



## **Introduction**

This chapter presents the testing and evaluation process conducted for the EventMarketPlace system. The purpose of testing is to verify that the system functions correctly, satisfies the specified functional requirements, and provides a stable and usable experience for users.

Several testing approaches were applied throughout the development process, including system testing, integration testing, usability evaluation, and compatibility testing. The testing phase also aimed to identify possible defects, evaluate system quality, and ensure that all major workflows operate as expected before final deployment and project submission.

## **8-1 Test Plan**

### **8-1-1 Testing Objectives**

The testing process was designed to achieve the following objectives:

- Verify that all functional requirements are correctly implemented.
- Ensure proper communication between frontend, backend, database, and external services.
- Validate the correctness of booking, event management, authentication, and AI functionalities.
- Evaluate the usability and responsiveness of the user interface.
- Detect and resolve defects before the final system delivery.
- Assess the overall reliability and stability of the system.

### **8-1-2 Scope of Testing**

#### **In Scope**

The testing process included the following system components:

- Event Management Module
- Booking Management Module
- User Management and Authentication
- Supplier and Service Management
- Review and Rating System
- Analytics and Reporting
- Notification System

- AI Features and Recommendations
- Frontend user interfaces
- Backend REST APIs
- Firebase Firestore database operations
- External integrations such as Firebase Authentication and OpenRouter AI

### Out of Scope

The following areas were not fully covered during testing:

- Automated unit testing
- Large-scale load testing
- Security penetration testing
- Production deployment testing

### 8-1-3 Testing Environment

The system was tested in a local development environment using the following technologies and tools:

| Component       | Environment                   |
|-----------------|-------------------------------|
| Frontend        | React Application             |
| Backend         | Spring Boot                   |
| Database        | Firebase Firestore            |
| Authentication  | Firebase Authentication       |
| Storage         | Firebase Storage              |
| AI Integration  | OpenRouter API                |
| API Testing     | Postman                       |
| Browser Testing | Chrome, Firefox, Edge, Safari |

Table (82): Testing Environment

### **8-1-4 Testing Strategy**

A manual testing approach was adopted throughout the project. Test cases were prepared according to the system requirements and executed to verify system behavior under different scenarios.

The testing process focused mainly on the core workflows of the platform, including:

- User registration and authentication
- Event creation and management
- Event booking and cancellation
- Payment processing
- Notifications and emails
- AI recommendations and chatbot interaction

Each test case was evaluated according to expected outputs and actual system behavior.

## **8-2 Types of Tests**

### **8-2-1 Unit Testing**

Unit testing aims to verify the correctness of individual components and functions independently.

The project includes the required testing infrastructure through Spring Boot Test for the backend and React Testing Library for the frontend. However, due to project scope and time limitations, the focus was placed mainly on functional and system-level testing rather than detailed unit testing.

### **8-2-2 Integration Testing**

Integration testing was conducted to ensure that all system components communicate correctly with each other.

The following integrations were tested successfully:

- React frontend with Spring Boot backend APIs
- Spring Boot services with Firebase Firestore
- Firebase Authentication with protected APIs
- File uploads to Firebase Storage
- Email notifications through SMTP services

- AI communication through OpenRouter API

The testing confirmed that data flows correctly between all system layers.

### **8-2-3 System Testing**

System testing was the primary testing method used in this project. The complete system was tested as an integrated application to verify that all functional requirements operate correctly.

All major user workflows were tested, including:

- User registration and login
- Event browsing and filtering
- Event booking and payment
- Event management by organizers
- Service management by providers
- Ratings and reviews
- AI recommendations and chatbot support

The results showed that the system performs correctly across all implemented modules.

### **8-2-4 Acceptance Testing**

Acceptance testing was performed to verify that the system satisfies the project objectives and user requirements.

The system successfully demonstrated the following capabilities:

- Organizers can create and manage events.
- Users can browse and book events.
- Providers can offer additional services.
- Users can submit ratings and reviews.
- AI features provide intelligent recommendations and support.

The successful completion of these workflows indicates that the system satisfies its intended objectives.

### **8-2-5 Performance Testing**

Basic performance observations were conducted during testing to evaluate system responsiveness.

The system demonstrated acceptable performance under normal usage conditions:

| Operation      | Average Response  |
|----------------|-------------------|
| Page Loading   | 1–2 seconds       |
| API Requests   | Less than 150 sec |
| Image Upload   | 2–5 seconds       |
| PDF Generation | 1–3 seconds       |
| AI Responses   | 5–15 seconds      |

*Table (83): Performance Testing*

Overall, the system remained responsive and stable during testing.

### 8-2-6 Security Testing

Basic security mechanisms were implemented and tested within the system, including:

- Firebase Authentication
- Token-based authentication
- Input validation
- Database access restrictions
- CORS configuration

Although advanced penetration testing was not performed, the implemented security measures provide an acceptable level of protection for an academic project.

### 8-2-7 Usability Testing

Usability evaluation was conducted to assess the clarity and ease of use of the system interface.

The evaluation focused on:

- Simplicity of navigation
- Interface consistency

- Form validation clarity
- User feedback messages
- Mobile responsiveness

The system demonstrated a clean and user-friendly interface with responsive behavior across different screen sizes.

#### 8-2-8 Compatibility Testing

The platform was tested on multiple browsers and screen sizes to ensure compatibility.

##### Browsers Tested

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

##### Devices Tested

- Desktop
- Tablet
- Mobile devices

The results confirmed that the system functions correctly across all tested environments.

## 8-3 Test Cases and Results

### 8-3-1 Test Case Structure

Each test case was designed using a structured format containing:

| Field          | Description         |
|----------------|---------------------|
| Test Case ID   | Unique identifier   |
| Requirement ID | Related requirement |

|                 |                          |
|-----------------|--------------------------|
| Field           | Description              |
| Test Objective  | Purpose of the test      |
| Preconditions   | Required setup           |
| Test Steps      | Execution steps          |
| Expected Result | Expected system behavior |
| Actual Result   | Observed output          |
| Status          | Pass or Fail             |

*Table (84): Test Case Structure*

### 8-3-2 Test Results Summary

The following table summarizes the final testing results:

| Module                | Total Tests | Passed | Failed |
|-----------------------|-------------|--------|--------|
| Event Management      | 12          | 12     | 0      |
| Booking Management    | 15          | 15     | 0      |
| User Management       | 8           | 8      | 0      |
| Supplier Management   | 10          | 10     | 0      |
| Review System         | 9           | 9      | 0      |
| Analytics & Reporting | 11          | 11     | 0      |
| Notification System   | 7           | 7      | 0      |
| AI Integration        | 5           | 5      | 0      |
| Total                 | 77          | 77     | 0      |

*Table (85): Test Results Summary*

The results indicate that all tested functionalities operated successfully during the evaluation process.

## 8-4 Test Results and Analysis

The testing phase demonstrated that the EventMarketPlace system satisfies all major functional requirements and performs reliably across different modules.

Several minor issues were identified during testing, mainly related to interface behavior and notification handling. These issues were resolved successfully before the final evaluation phase.

The system showed the following strengths:

- Stable system behavior
- Correct business logic execution
- Successful integration between technologies
- Accurate booking and recommendation processes
- User-friendly interface and navigation

Overall, the system is considered functionally complete and suitable for academic demonstration and evaluation.

## 8-5 Quality Metrics

The following quality indicators were observed during testing:

| Metric               | Result   |
|----------------------|----------|
| Requirement Coverage | 100%     |
| Test Pass Rate       | 100%     |
| Critical Defects     | 0        |
| System Stability     | Stable   |
| Data Integrity       | Verified |
| API Responsiveness   | Good     |
|                      |          |

*Table (86): Quality Metrics*

The system demonstrated good software quality and acceptable performance for the project scope.



## 8-6 Usability Evaluation

Formal usability studies with real users were not conducted due to time limitations. Instead, usability evaluation was performed through expert review and repeated practical testing.

The evaluation confirmed that the system provides:

- Clear navigation structure
- Consistent interface design
- Responsive layouts
- Simple booking workflows
- Easy interaction with AI features

Some future improvements may include:

- More advanced user guidance
- Enhanced accessibility support
- Additional interface customization options

## Conclusion

The testing and evaluation process confirmed that the EventMarketPlace system successfully fulfills its intended objectives and functional requirements.

All core modules operated correctly during testing, and the integration between frontend, backend, database, and external services functioned reliably. The platform also demonstrated acceptable usability, responsiveness, and system stability.

Based on the obtained results, the system is considered suitable for academic evaluation and provides a solid foundation for future improvements and deployment.



## **Chapter 9:**

### **Results and Analysis**

## Introduction

This chapter presents the practical results obtained after the development and implementation of the Event Marketplace system. It also includes an analysis of system performance and an evaluation of the extent to which the predefined objectives have been achieved. Furthermore, it compares the proposed system with traditional systems, analyzes strengths and weaknesses, and measures key performance indicators (KPIs).

### 9-1- Results of implementing the system:

The system relies on an integrated software architecture consisting of a front-end developed using React and PrimeReact, and a back-end developed using Spring Boot, with Firebase used for authentication, cloud storage, and data management.

The system also utilizes a RESTful API architecture to connect the front-end to back-end services, employing a Controller-Service-Repository (CPR) model to organize software processes and separate business logic from data access layers and The following functions are actually supported :

- Log in and create accounts via AuthController and AuthService using JWT with external authentication system integration.
- Manage events (create, edit, delete, view) via EventController and EventService.
- Electronic booking system via BookingController and BookingService.
- Manage service providers via SupplierController.
- Manage event-related services via ServiceController.
- Review and rating system via ReviewController.
- Dashboard control panel and statistics via DashboardController.
- Smart event planning support via AIController and AIService.
- Payment system and booking integration via PaymentService.
- Send emails and reminders via EmailService and EventReminderService.
- Manage storage via StorageService.
- Upload and link files to the system.
- The front-end integrates with these services via a REST API to provide an interactive user experience.

## 9-2 Comparative Analysis with Existing Systems:

A comparison was conducted between the Event Marketplace system and traditional event management systems to evaluate the efficiency of the proposed system and its contribution to improving user experience and operational processes.

| Comparison Area           | Traditional Systems               | Event Marketplace System               |
|---------------------------|-----------------------------------|----------------------------------------|
| Event Management          | Done manually or in a limited way | Fully integrated electronic management |
| Online Booking            | Limited or not available          | Fully available                        |
| User & Role Management    | Limited                           | Fully supported                        |
| User Interface            | Traditional                       | Modern and interactive                 |
| Ratings & Reviews         | Mostly not available              | Available                              |
| Notifications & Reminders | Limited                           | Supported                              |
| Dashboards & Analytics    | Basic                             | Advanced                               |
| Artificial Intelligence   | Not supported                     | Supported via AI Planner               |
| Chat Services             | Not available                     | Supported via ChatBot                  |
| Cloud Storage             | Limited                           | Supported via Firebase                 |
| Security & Authentication | Traditional                       | Traditional security methods           |

*Table (87): Comparative Analysis with Existing Systems*

From this comparison, it is clear that the proposed system provides a more advanced and flexible environment compared to traditional systems, particularly in automation, process management, and service integration.

### 9-3 Objectives Achievement:

The practical implementation results showed that the system successfully achieved its main objectives to a high degree, as follows:

| Objective                               | Achievement Level     |
|-----------------------------------------|-----------------------|
| Developing an event management platform | ✓ Fully achieved      |
| Facilitating online booking             | ✓ Fully achieved      |
| Supporting multiple user roles          | ✓ Fully achieved      |
| Providing secure authentication system  | ✓ Fully achieved      |
| Improving user experience               | ★ Highly achieved     |
| Providing modern user interfaces        | ✓ Fully achieved      |
| Supporting service providers            | ✓ Fully achieved      |
| Providing dashboards and analytics      | ✓ Fully achieved      |
| Supporting artificial intelligence      | ✓ Fully achieved      |
| Providing ChatBot interaction           | ✓ Fully achieved      |
| Supporting notifications and reminders  | ✓ Fully achieved      |
| Supporting payment processing           | ✓ Fully achieved      |
| Building a scalable system              | — Moderately achieved |

Table (88): Objectives Achievement

### 9-4 Strengths and Weaknesses Analysis

#### First: Strengths

The system has several strengths that enhance its efficiency and effectiveness, including:

1. Use of RESTful API architecture for system communication.
2. Application of the Controller-Service-Repository pattern.
3. Support for multiple roles and permissions.
4. Modern and interactive user interfaces.
5. Integration of artificial intelligence via AI Planner.
6. Availability of ChatBot interaction.
7. Integrated booking system.
8. Payment processing linked with bookings.
9. Support for ratings and reviews.
10. Advanced dashboards and analytics.
11. Integration with Firebase for authentication and storage.

12. Notifications and reminders for events.
13. Ease of future scalability and maintenance.

## Second: Weaknesses

Despite the success of the system in achieving its main objectives, some areas require future improvement:

1. Performance optimization is needed when the number of users increases significantly.
2. Lack of dedicated mobile applications.
3. Limited real-time push notification support.
4. AI algorithms need improvement for more accurate recommendations.
5. Need for integration with real payment gateways.
6. Dependence on external cloud services.

## 9-5 Key Performance Indicators (KPIs):

A set of key performance indicators was measured to evaluate system efficiency during operation, with the following results:

| Indicator                                   | Result |
|---------------------------------------------|--------|
| Response Speed                              | Good   |
| System Stability                            | High   |
| Booking Success Rate                        | High   |
| User and Permission Management Efficiency   | High   |
| Ease of Use                                 | High   |
| Frontend and Backend Integration Efficiency | High   |
| Data Processing Accuracy                    | High   |
| Notification and Reminders Efficiency       | Good   |
| Scalability                                 | Good   |
| Program Error Rate                          | Low    |

Table (89): KPIs

The practical testing showed that the system is capable of executing core operations efficiently and stably, while providing a suitable user experience.

**Chapter 10:**  
**Conclusion & Recommendations**



## **Introduction:**

This chapter presents the final summary of the project, highlighting the key achievements during the analysis, design, and implementation phases. It also discusses the challenges encountered during development and provides future recommendations for improving and enhancing the system.

## **10-1 Project Summary and Main Achievements:**

During this project, an integrated electronic event management system named Event Marketplace was developed. The system aims to facilitate event organization, booking processes, and communication between users, organizers, and service providers within a unified digital platform.

The system was developed using React and PrimeReact for the frontend, Spring Boot for the backend, and Firebase for authentication and cloud storage.

Key achievements include:

- Development of a secure authentication system using JWT and Firebase Authentication.
- Support for multiple user roles and permissions.
- Creation of a complete event and booking management system.
- Development of service provider and event-related service management modules.
- Implementation of payment processing integrated with bookings.
- Development of a review and rating system.
- Provision of dashboards and analytics for monitoring and management.
- Integration of AI-based smart event planning.
- Development of a ChatBot for user interaction.
- Support for notifications and event reminders.
- Building modern and interactive user interfaces.

The project contributed to providing a digital solution that improves event management, reduces manual processes, and enhances user experience efficiency and flexibility.

## **10-2 Challenges and Difficulties:**

The development process faced several technical and organizational challenges, including:

1. Difficulty in structuring the system architecture and organizing components.
2. Challenges in integrating frontend and backend systems.
3. Managing roles and access control.
4. Working with REST APIs and ensuring stable data exchange.
5. Integrating Firebase and cloud storage management.
6. Developing and integrating the AI system.
7. Building and connecting the ChatBot system.
8. Managing payment processes and linking them with bookings.
9. Testing various system scenarios.
10. Performance optimization and response time improvements.

Despite these challenges, continuous development and testing phases led to a stable and usable system.

## **10-3 Future Development Suggestions:**

The system can be further improved by adding the following features:.

1. Real-time push notification support.
2. Integration with real payment gateways such as Cham Cash.
3. Enhancing AI algorithms for better recommendations.
4. Advanced real-time communication between users and service providers.
5. More detailed analytics and reporting systems.
6. Multi-language support.

7. Improved scalability and performance for large user bases.
8. Integration of maps and geolocation for events.
9. Use of Machine Learning techniques for user behavior analysis.

#### **10-4 Recommendations:**

Based on the project results and experience gained, the following recommendations are provided:

1. Careful requirement analysis before development begins.
2. Adoption of a well-structured architecture to ease maintenance and development.
3. Focus on security and user data protection.
4. Regular system testing to ensure stability.
5. Continuous improvement of user experience and interfaces.
6. Proper academic and technical documentation of the project.
7. Use of modern technologies that support scalability.
8. Leveraging cloud services for storage and data management.
9. Monitoring and analyzing performance indicators regularly.
10. Continuous system development according to user needs and technological advancements.

# **Chapter 11:**

## **APPENDICES & REFERENCES**

## **APPENDIX A:**

### **Requirements Gathering Materials**

This appendix presents the requirement gathering process used during the analysis and planning phase of the EventMarketPlace system. The requirements were collected through informal interviews, direct discussions, observation of existing event management methods in the Syrian market, and analysis of user behavior during booking and event discovery processes.

The requirement gathering phase focused on identifying the needs of the three main stakeholders of the platform:

- Attendees
- Organizers
- Service Providers

The collected information helped define the system requirements and determine the core functionalities implemented throughout the project.

#### **Sample Questions for Attendees**

- How do you usually discover events and activities?
- What difficulties do you face during ticket booking?
- Would digital QR tickets improve your experience?
- What type of recommendations would help you discover events more efficiently?
- Would you use an AI assistant to answer event-related questions?

#### **Sample Questions for Organizers**

- What challenges do you face during event management?
- What statistics and reports are most useful for monitoring events?
- Do you currently use digital booking systems?
- Would a supplier marketplace help simplify event preparation?

#### **Sample Questions for Service Providers**

- What services do you usually provide for events?
- How do organizers currently contact you?
- Would a digital marketplace improve your visibility?

The requirement gathering process highlighted the lack of a unified digital platform for the Syrian events sector and supported the idea of integrating intelligent services such as recommendations and AI-powered assistance.

## **APPENDIX B:**

## Detailed System Design

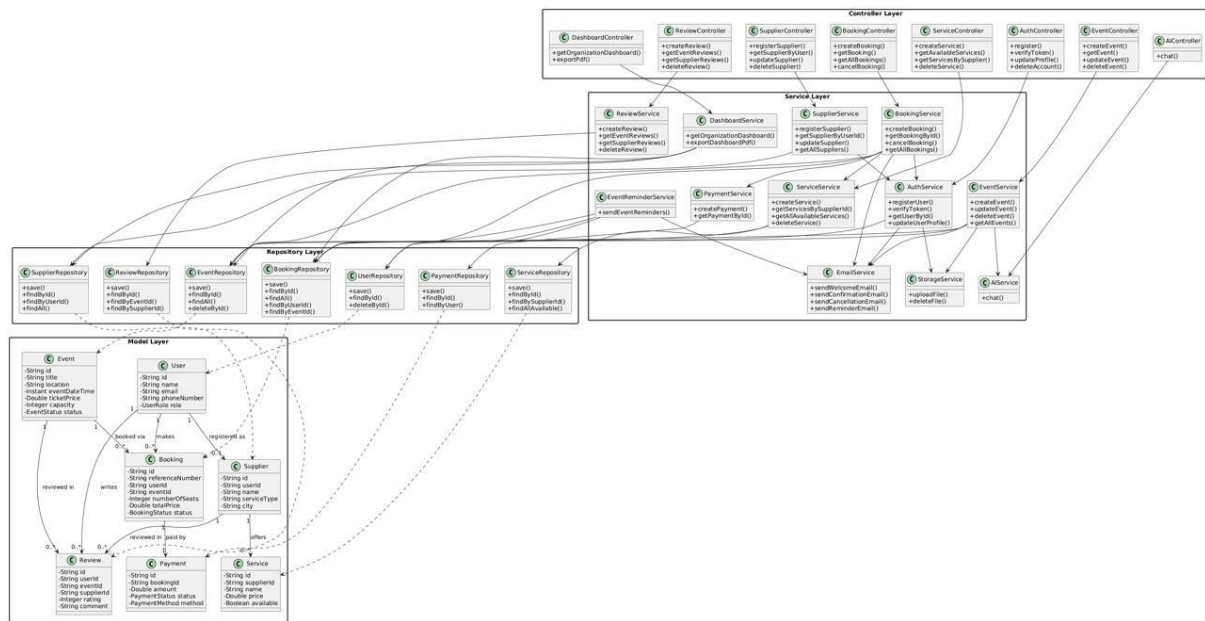


Figure65 : B-1 Class and Object Diagrams

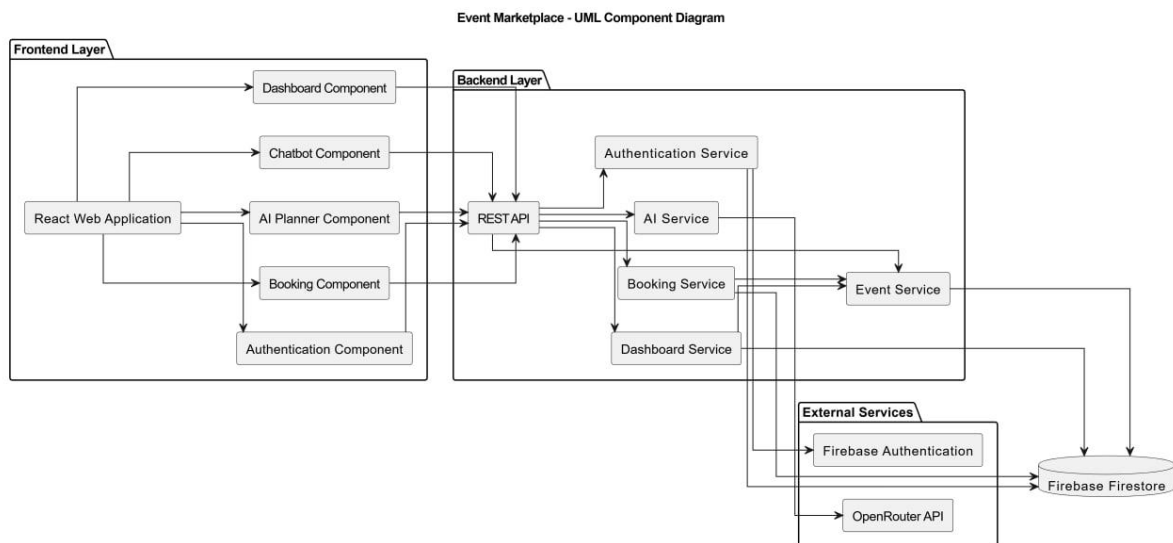
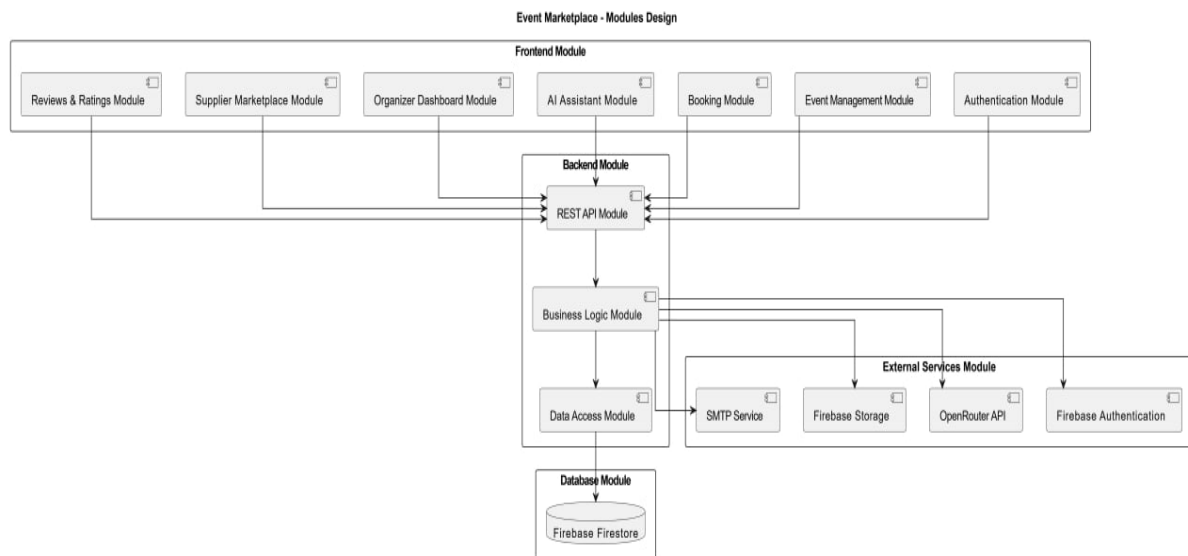


Figure66 : B-1-2 Component Diagrams



*Figure67 : B-1-3 Modules Design*

## **B-1-4 APIs Design**

### **API Overview**

The EventMarketPlace platform uses RESTful APIs to enable communication between the frontend application and backend services.

All APIs exchange data using JSON format and follow standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE.

The backend APIs support the following major functionalities:

- User authentication and authorization
- Event management
- Booking management
- AI assistant and smart recommendations
- Supplier and service management
- Reviews and ratings
- Dashboard analytics and reports

The APIs were implemented using Spring Boot and tested using Postman.

## Main API Endpoints

| Endpoint                             | Method | Description                    |
|--------------------------------------|--------|--------------------------------|
| /api/auth/login                      | POST   | User authentication            |
| /api/events                          | GET    | Retrieve all events            |
| /api/events                          | POST   | Create a new event             |
| /api/bookings                        | POST   | Create booking                 |
| /api/bookings/{id}/cancel            | PATCH  | Cancel booking                 |
| /api/ai/chat                         | POST   | AI chatbot communication       |
| /api/events/recommendations/{userId} | GET    | Generate smart recommendations |
| /api/dashboard/organizer/{id}        | GET    | Retrieve organizer dashboard   |

Table (90): Main API Endpoints

## Example API Request and Response

### Example 1: User Login API

Endpoint: POST /api/auth/login

Request{

"email": "user@example.com",

"password": "SecurePass123"

}

Response{

"token": "jwt-token",

"user": {

"id": "user123",

"email": "user@example.com",

"name": "John Doe",

"role": "USER" }}



## Example 2: Create Booking API

Endpoint: POST /api/bookings

Request{

```
"eventId": "event123",  
"userId": "user456",  
"numberOfSeats": 2,  
"paymentMethod": "CREDIT_CARD"
```

}

Response{

```
"id": "booking789",  
"referenceNumber": "BK-1715123456789",  
"status": "CONFIRMED",  
"paymentStatus": "COMPLETED",  
"totalPrice": 800.0
```

}

## B-2 Intelligent Services Design

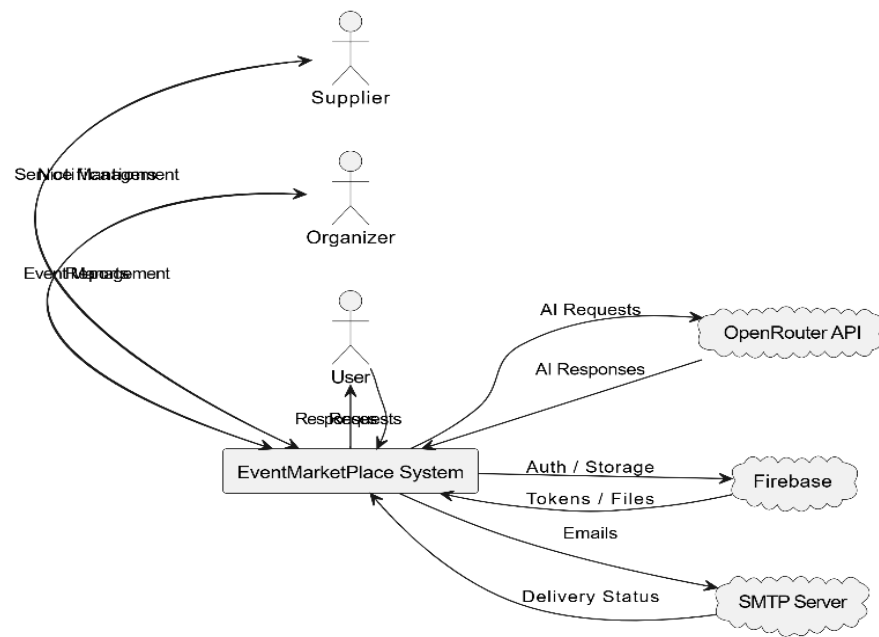


Figure 68: Context Diagram

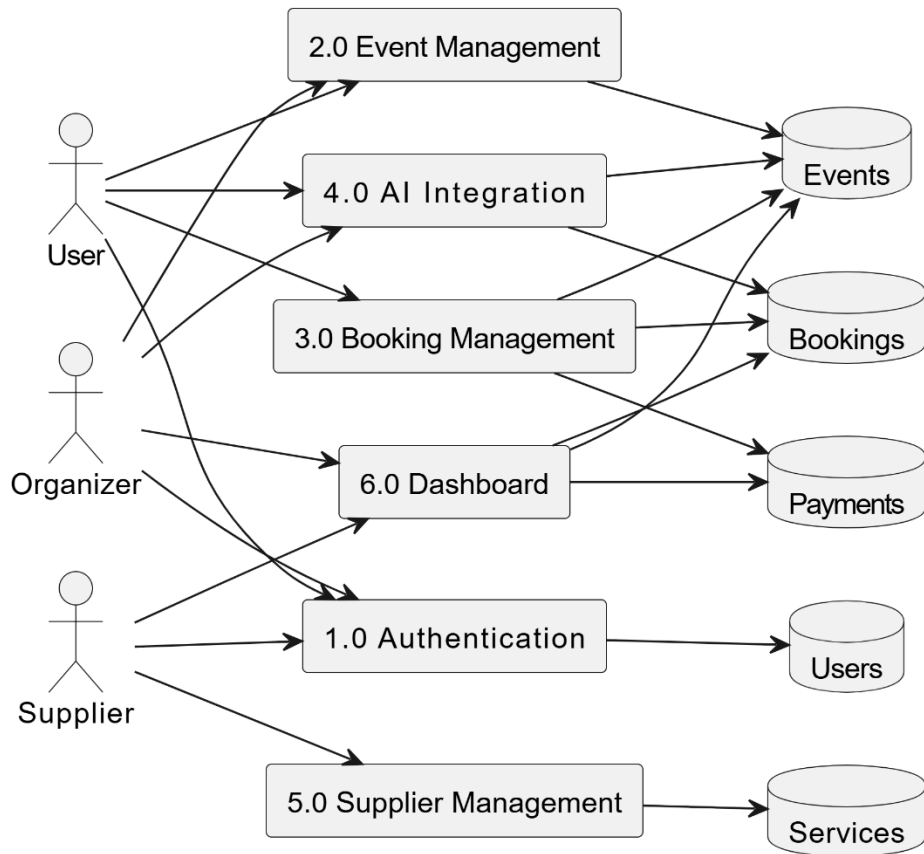


Figure 69: Level 1 DFD

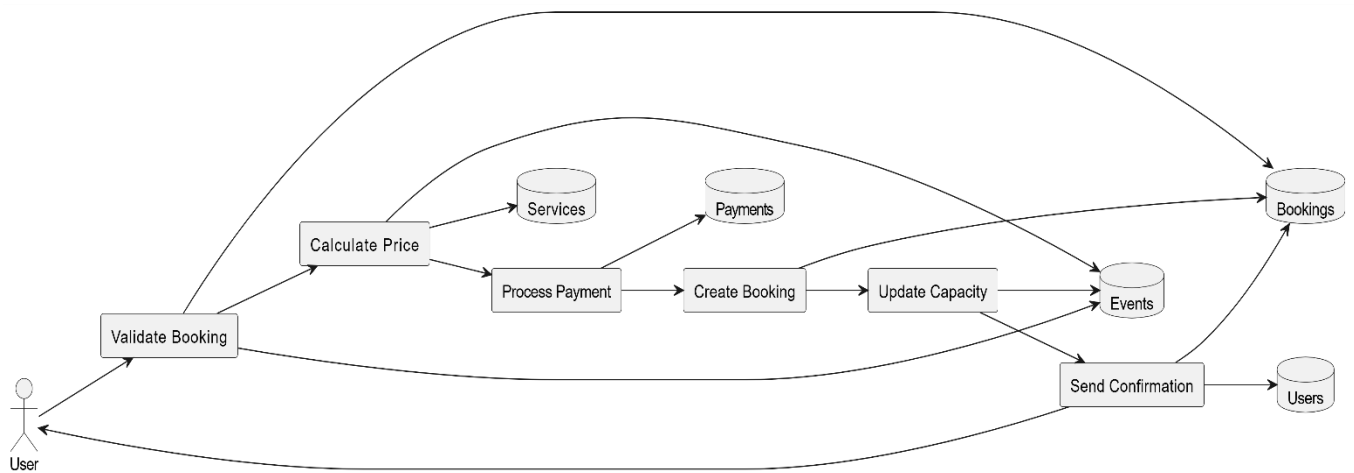


Figure 70: Booking Level 2

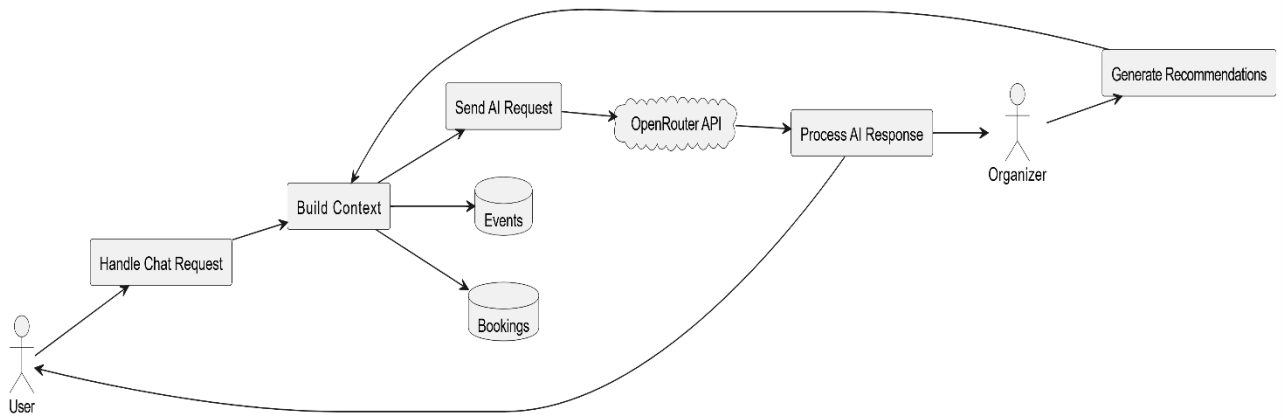


Figure 71: AI Level 2

## State Machine Diagrams

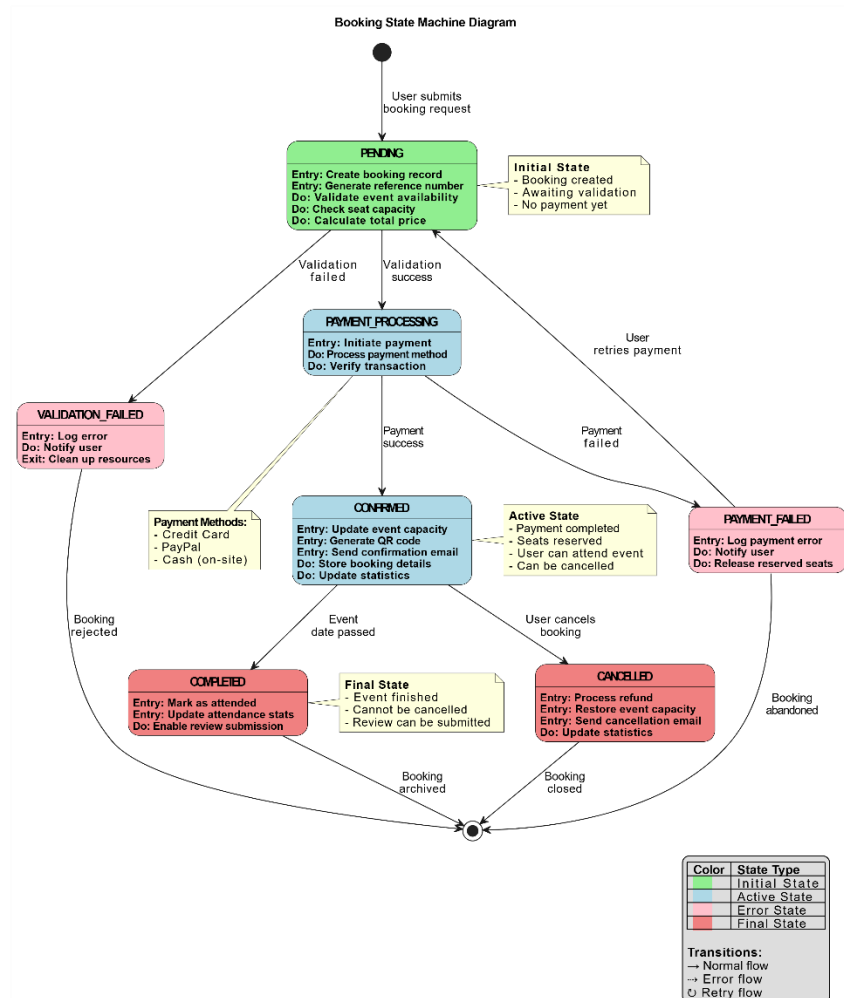


Figure72 : Booking State Machine Diagrams

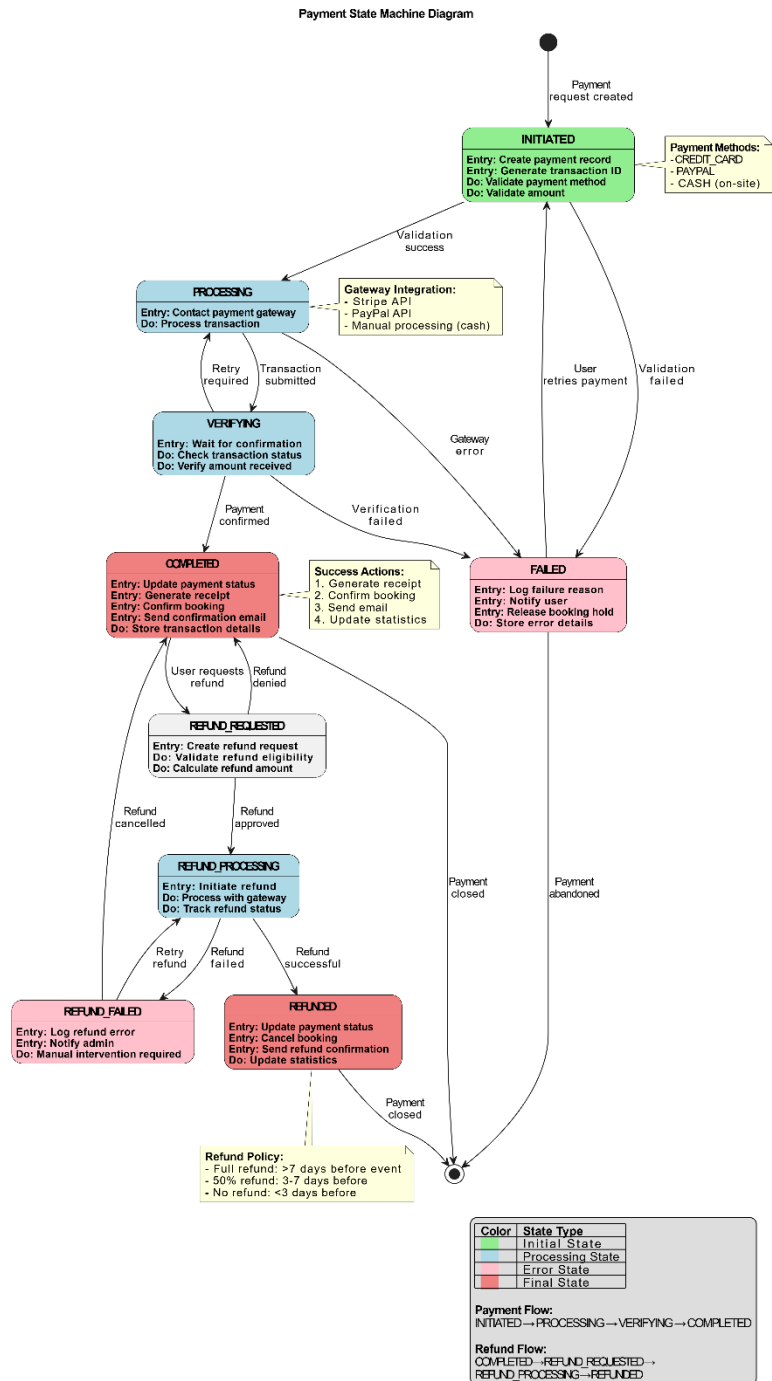


Figure 73 : Payment State Machine Diagrams

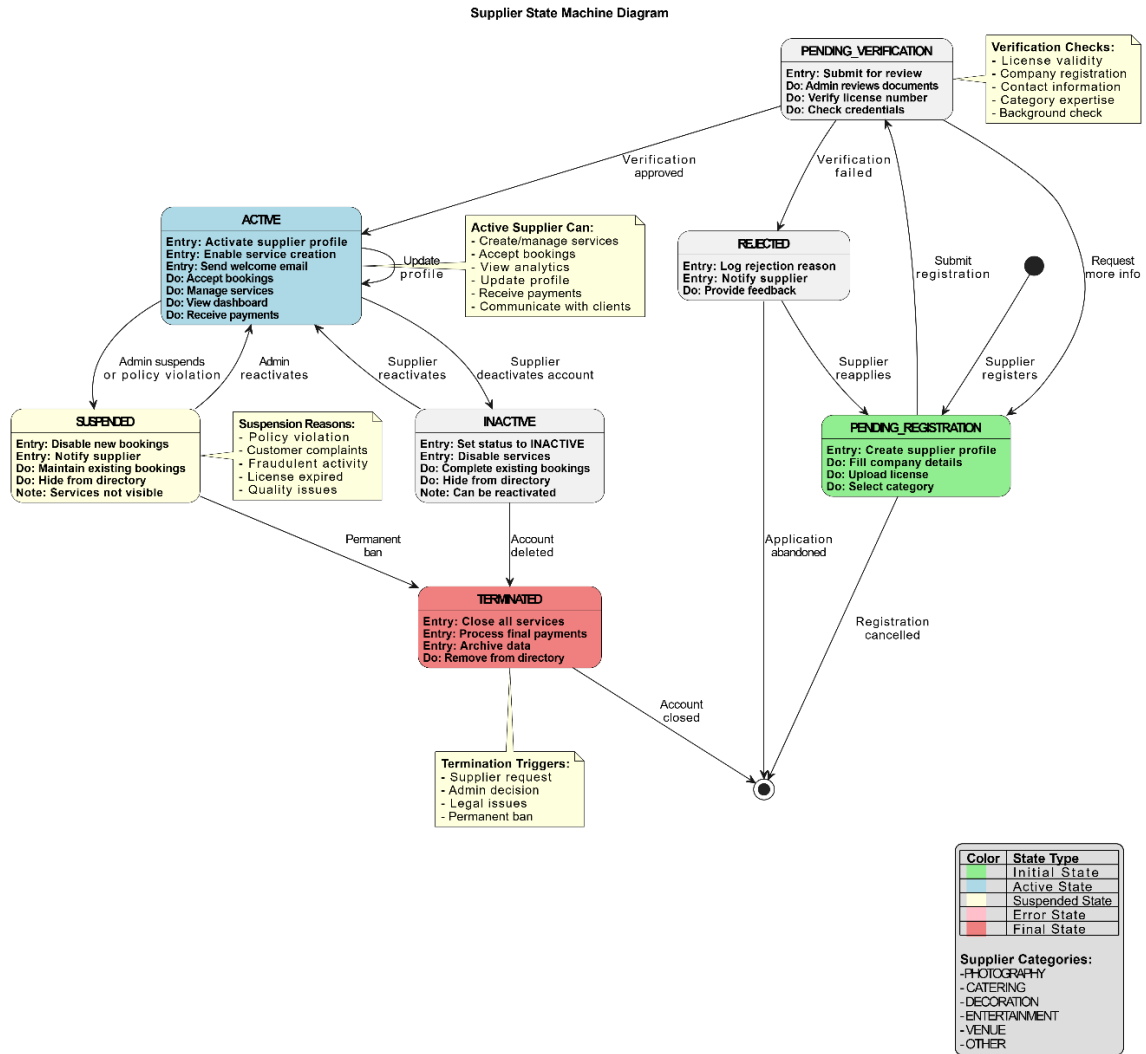


Figure74 : Supplier State Machine Diagrams

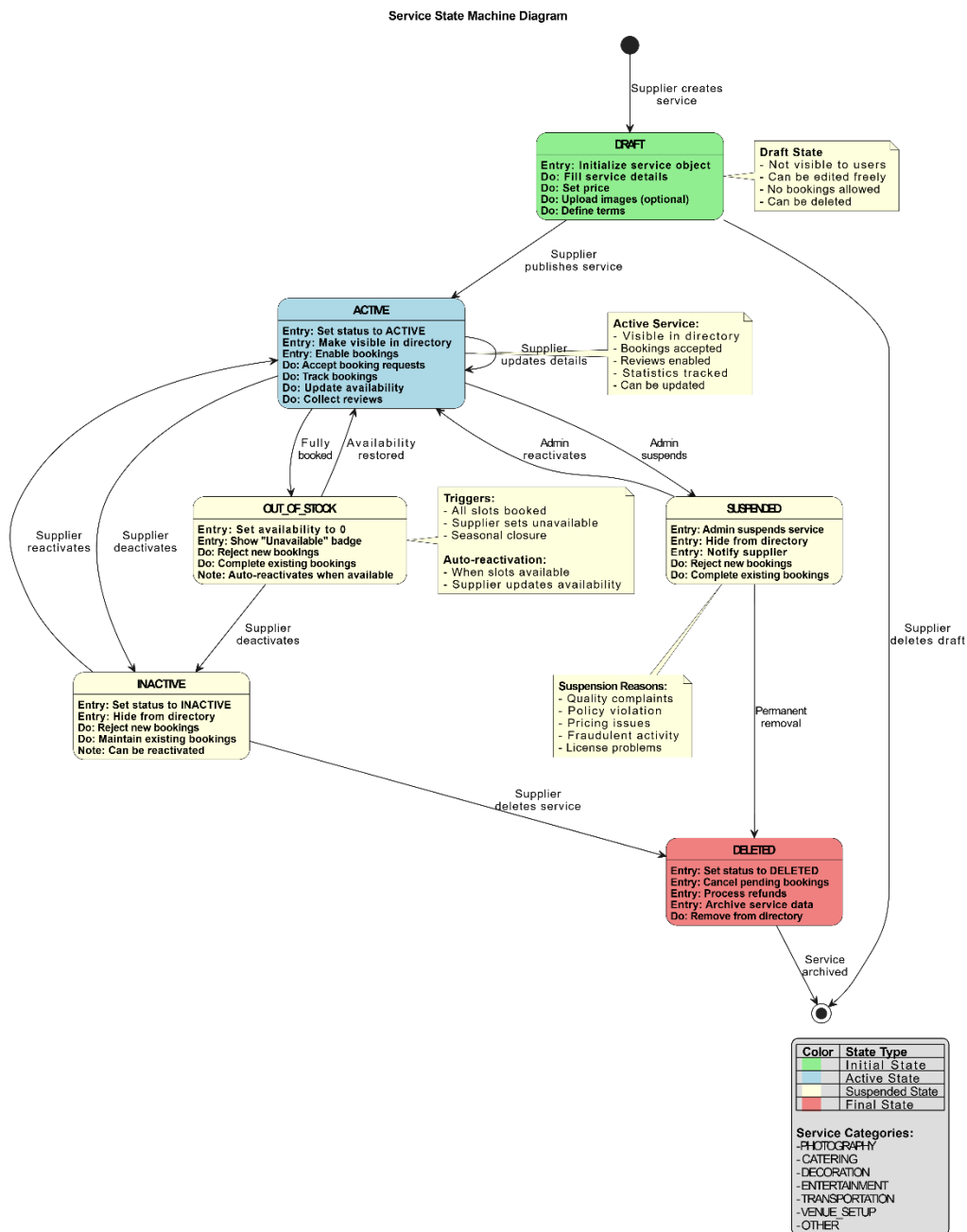


Figure75 : Service State Machine Diagrams



| Testing Type        | Passed | Failed | Success Rate |
|---------------------|--------|--------|--------------|
| Unit Testing        | 152    | 4      | 97.4%        |
| Integration Testing | 85     | 4      | 95.5%        |
| System Testing      | 64     | 3      | 95.5%        |
| Security Testing    | 32     | 2      | 94.1%        |

Table (91): D-2 Testing Summary

### D-3 Sample Test Cases

#### Booking Test

| Field    | Result          |
|----------|-----------------|
| Feature  | Booking         |
| Expected | Booking created |
| Actual   | Booking created |
| Status   | PASS            |

Table (92): D-3 Sample Test Cases

#### AI Chatbot Test

| Field    | Result                |
|----------|-----------------------|
| Feature  | AI Chatbot            |
| Expected | AI response           |
| Actual   | AI response generated |
| Status   | PASS                  |

Table (93): AI Chatbot Test

#### D-4 Performance Testing

| Metric            | Result |
|-------------------|--------|
| Avg Response Time | 1.8s   |
| Concurrent Users  | 2000   |
| Error Rate        | 0.3%   |

Table (94): D-4 Performance Testing

#### D-5 Security Testing

The system was tested against:

- SQL Injection
- XSS
- Unauthorized Access
- JWT Manipulation

All major attacks were successfully prevented.

#### D-6 AI Features Testing

| Feature               | Result |
|-----------------------|--------|
| AI Chatbot            | PASS   |
| Smart Recommendations | PASS   |
| AI Event Planning     | PASS   |

Table (95): D-6 AI Features Testing

## **E-1 System Access**

### **User Registration**

1. Open the system homepage.
2. Click on the “Sign Up” button.
3. Enter the required information:
  - Full Name
  - Email Address
  - Password
4. Select the user role.
5. Click “Create Account”.

After successful registration, the user can log in to the system using their email and password.

### **User Login**

1. Open the homepage.
2. Click “Login”.
3. Enter email and password.
4. Click “Sign In”.

After authentication, the user is redirected to the main dashboard.

## **E-2 Browsing Events**

Users can browse available events through the Events page.

The system allows users to:

- Search events by title or location.
- Filter events by category or city.
- View event details including:
  - Title
  - Description
  - Date and Time
  - Location
  - Ticket Price
  - Available Seats

### **E-3 Booking an Event**

#### **Booking Process**

1. Open the selected event page.
2. Click “Book Now”.
3. Select the number of seats.
4. Choose additional services if available.
5. Select a payment method.
6. Click “Confirm Booking”.

#### **After successful booking:**

- A booking confirmation is generated.
- A QR ticket is created.
- A confirmation email is sent to the user.

### **E-4 Managing Bookings**

Users can access their bookings through the “My Bookings” page.

The system allows users to:

- View booking details.
- Download QR tickets.
- Cancel bookings before the event date.
- Review payment information.

### **E-5 AI Features**

The platform provides several AI-powered features, including:

#### **AI Chatbot**

Users can interact with the chatbot to:

- Ask about available events.
- Receive event information.
- Get booking assistance.

#### **Smart Recommendations**

The system analyzes user preferences and booking history to recommend relevant events.

### **AI Event Planner**

Users and organizers can request AI-generated event suggestions based on:

- Budget
- Event type
- Location
- Number of guests

### **E-6 Organizer Dashboard**

Organizers can manage their events using the dashboard.

The dashboard allows organizers to:

- Create events.
- Edit event information.
- Monitor bookings.
- View analytics and reports.
- Manage event status.

### **E-7 Supplier Management**

Suppliers can manage their services through the supplier dashboard.

The system allows suppliers to:

- Add new services.
- Update service information.
- Monitor booking requests.
- Manage availability.

### **E-8 Reviews and Ratings**

Users can submit reviews and ratings after attending events.

The review system helps:

- Improve service quality.
- Provide feedback to organizers.
- Assist other users in selecting events.

## **E-9 Logout**

Users can securely log out of the system by selecting the “Logout” option from the navigation menu.

This action terminates the current user session and redirects the user to the homepage.

## **REFERENCES**

### **Arabic and Foreign References**

1. Sommerville, I. Software Engineering. Pearson Education.
2. Pressman, R. Software Engineering: A Practitioner’s Approach.
3. Jurafsky, D., & Martin, J. Speech and Language Processing.

### **Websites and Technical Documentation**

4. Spring Boot Documentation.
5. React Official Documentation.
6. Firebase Documentation.
7. OpenRouter API Documentation.
8. PrimeReact Documentation.

### **Research Papers and Scientific Articles**

9. Research papers related to recommender systems and intelligent platforms.
10. Research papers related to AI chatbot technologies.
11. Studies related to event management systems and cloud-based architectures.

تَمَّ بِعَوْنِ اللَّهِ تَعَالَى